



Fakultät Elektronik und Informatik

Studiengang Informatik/Security

Stoneforge RL

Navigation in einer eigenentwickelten prozedural generierten 2D Welt
mittels Reinforcement Learning

Projektarbeit

Vorgelegt von: Florian Merlau, 81775
Laurin Rößler, 3017947

Betreuer: Prof. Lecon

Abgabedatum: 14. 08. 2026

Aalen, 2026

Hinweis zur Verwendung von KI-Werkzeugen

Bei der Erstellung dieser Arbeit wurden KI-gestützte Werkzeuge (GitHub Copilot sowie Claude von Anthropic) als Hilfsmittel eingesetzt. Sie dienten der sprachlichen Ausformulierung und Überarbeitung einzelner Textpassagen sowie als Unterstützung bei der Programmierung. Die inhaltlichen Aussagen, die Konzeption, die Durchführung der Experimente und sämtliche berichteten Ergebnisse wurden eigenständig erarbeitet; alle KI-gestützt formulierten Passagen wurden geprüft und werden inhaltlich verantwortet.

Eidesstattliche Erklärung

Hiermit wird an Eides statt erklärt, dass die vorliegende Projektarbeit mit dem Titel

*»Stoneforge RL — Navigation in einer eigenentwickelten prozedural
generierten 2D Welt mittels Reinforcement Learning«*

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt wurden. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Quellen entnommen sind, werden als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Da die Arbeit von zwei Personen gemeinsam erstellt wurde, wird zusätzlich erklärt, dass die Abschnitte 3.1, 4.1 und 5.1 von Laurin Rößler und die Abschnitte 3.2, 4.2 und 5.2 von Florian Merlau verfasst wurden; die Kapitel 1 und 2 entstanden vollständig gemeinsam. Die Zuordnung von Kapitel 6 und der Anhänge ist Anhang B zu entnehmen.

Aalen, den 14.08.2026

Aalen, den 14.08.2026

Florian Merlau

Laurin Rößler

Kurzfassung

Die vorliegende Arbeit untersucht in zwei aufeinander aufbauenden Teilen, wie eine prozedural generierte, partiell beobachtbare 2D-Welt aufgebaut sein muss, damit sie für einen Reinforcement-Learning-Agenten fair und lösbar ist, und welches Trainingsverfahren darin die besten Navigationsergebnisse liefert, gemessen primär an der Erfolgsquote unter Berücksichtigung der resultierenden Pfadeffizienz. Dazu wird eine C++-basierte Simulationsumgebung mit angebundenem Python-Interface entwickelt, in der der Ausgang jeder Welt ausschließlich über eine Breitensuche vom Spawnpunkt aus platziert wird; unlösbare Weltinstanzen werden dadurch strukturell ausgeschlossen, unabhängig von der Wanddichte. Auf dieser Umgebung werden rekurrente und nicht-rekurrente Proximal-Policy-Optimization-Modelle (PPO) über ein leistungsbasiertes Curriculum trainiert und gegen eine ungelernete ε -Kompass-Heuristik eingeordnet. Über sieben unabhängige Läufe erreicht das LSTM-PPO-Modell eine mittlere Erfolgsquote von 64,6 % ($\pm 10,1$) auf Testset A und 68,8 % ($\pm 15,5$) auf Holdout B und übertrifft damit das MLP-PPO-Modell (33,5 % $\pm 25,5$ auf Testset A) statistisch signifikant. Die Pfadeffizienz bleibt bei beiden Verfahren niedrig. Die ε -Kompass-Heuristik übertrifft bei $\varepsilon = 0,8$ dennoch beide trainierten Modelle sowohl in der Erfolgsquote als auch in der Pfadeffizienz. Dieser Befund wird methodisch eingeordnet; als mögliche Teilerklärung wird herausgearbeitet, dass die auf Erreichbarkeit optimierte Weltgenerierung selbst die einfache Wegstruktur begünstigen könnte, die das ungelernete Verfahren ausnutzt.

Schlagwörter: Reinforcement Learning, Proximal Policy Optimization, POMDP, Curriculum Learning, prozedurale Weltgenerierung

Abstract

This work investigates, in two consecutive and mutually dependent parts, how a procedurally generated, partially observable 2D world must be structured to be fair and solvable for a reinforcement learning agent, and which training procedure yields the best navigation results within it, measured primarily by success rate while accounting for the resulting path efficiency. A custom simulation environment implemented in C++ with a Python interface places the exit of every world exclusively at positions confirmed reachable from the spawn point by a breadth-first search, which structurally excludes unsolvable world instances regardless of wall density. Recurrent and non-recurrent Proximal Policy Optimization (PPO) agents are trained on this environment through a performance-based curriculum and compared against an untrained ε -compass heuristic baseline. Across seven independent runs, the LSTM-PPO model reaches a mean success rate of 64.6 % (± 10.1) on Testset A and 68.8 % (± 15.5) on Holdout B, statistically significantly outperforming the MLP-PPO model (33.5 % ± 25.5 on Testset A). Path efficiency remains low for both trained models. The ε -compass heuristic at $\varepsilon = 0.8$ nonetheless outperforms both trained models in success rate and path efficiency alike. This finding is placed in a broader methodological context: the reachability-optimized world generation itself may favor exactly the simple path structure that the untrained heuristic exploits.

Keywords: Reinforcement Learning, Proximal Policy Optimization, POMDP, Curriculum Learning, Procedural World Generation

Inhaltsverzeichnis

Hinweis zur Verwendung von KI-Werkzeugen	III
Eidesstattliche Erklärung	IV
Kurzfassung	V
Abstract	VI
Abbildungsverzeichnis	X
Tabellenverzeichnis	XII
Quelltextverzeichnis	XIII
Abkürzungsverzeichnis	XIV
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung und Forschungsfrage	1
1.4 Vorgehen und Methodik	2
1.5 Abgrenzung	2
1.6 Verwandte Arbeiten	3
2 Grundlagen	4
2.1 Grundlagen der prozeduralen Weltgenerierung	4
2.1.1 Deterministischer Zufall und Pseudozufallszahlen	4
2.1.2 Gradienten- und Value-Noise-Verfahren	4
2.1.3 Zelluläre Automaten zur Gefügeglättung	6
2.1.4 Graphentheoretische Erreichbarkeitsanalyse	7
2.2 Grundlagen des Reinforcement Learning	8
2.2.1 Partially Observable Markov Decision Processes (POMDP)	9
2.2.2 Potential Based Reward Shaping (PBRs)	10
2.2.3 Curriculum Learning	10
2.2.4 Proximal Policy Optimization (PPO)	11
2.2.5 Rekurrente Neuronale Netze (RNN)	11
2.2.6 Long Short Term Memory (LSTM)	12
2.2.7 Deep Q Networks (DQN)	13
2.2.8 Die Gymnasium-Schnittstelle (ehemals OpenAI Gym)	14

3	Methodik	16
3.1	Methodik der Weltgenerierung	16
3.1.1	Methodische Grundlagen	16
3.1.2	Heuristiken und Regeln	17
3.1.3	Systemkonfiguration	17
3.2	Methodik des RL-Trainings	18
3.2.1	Auswahl der zu vergleichenden Trainingsverfahren	18
3.2.2	Problemformulierung: Zustand, Aktionsraum und Reward	18
3.2.3	Curriculum Konzept und Schnittstelle zur Weltgenerierung	19
3.2.4	Definition der Baselines	20
3.2.5	Trainings- und Evaluationsprotokoll	21
3.2.6	Validierungsstrategie und Umgang mit Trainingsinstabilitäten	22
4	Umsetzung und technische Realisierung	24
4.1	Implementierung der Weltgenerierung	24
4.1.1	Technologie-Stack	24
4.1.2	Implementierung der Rauscherzeugung und des Koordinaten-Hashings	25
4.1.3	Biemeinteilung und Objekt-Placement	26
4.1.4	Topologische Sicherheits-Fallbacks und Spawn-Freiräume	29
4.1.5	Generierung von Gewässern und dynamische Strand-Derivation	31
4.1.6	Chunk-Speicherarchitektur und prozedurales Biome-Blending	33
4.1.7	Prozedurale Struktur-Platzierung und dynamisches Gegner-Spawning	36
4.1.8	Spieler-Bewegung, Kollisionsabfrage und Kampfdynamik	39
4.2	Implementierung des RL-Trainings	42
4.2.1	Technologie-Stack	42
4.2.2	Anbindung der Spielumgebung und Aufbau der Beobachtung	42
4.2.3	Netzwerkarchitekturen: Asymmetrie zwischen LSTM- und MLP-Policy	44
4.2.4	Trainingspipeline und duales Curriculum-Gating	45
4.2.5	Reward-Shaping und Distanzfeld	48
4.2.6	Umsetzung der Baselines: Die wandblinde Kompass-Heuristik	50
4.2.7	Vollständige Hyperparameter-Konfiguration	51
5	Evaluation	52
5.1	Evaluation der Weltgenerierung	52
5.1.1	Visuelle Ergebnisse und Biome-Gradients	52
5.1.2	Synthese der Spielelemente	52
5.1.3	Artefaktanalyse: Das Phänomen des Biome-Swallowing	53
5.1.4	Lösungsansätze für kleine Biome	53
5.1.5	Performance-Evaluation und Benchmarking	54
5.1.6	Kritische Diskussion und Limitationen	55
5.2	Evaluation des RL-Trainings	56
5.2.1	Erfolgsquote der trainierten Verfahren	56
5.2.2	Pfadeffizienz derselben Verfahren	58

5.2.3	Die Determinismus-Lücke	60
5.2.4	Einordnung gegenüber den Referenzpunkten	61
5.2.5	Wirksamkeit der Hyperparameter	62
6	Fazit und Ausblick	64
6.1	Diskussion der Ergebnisse	64
6.2	Fazit	65
6.2.1	Beitrag der Weltgenerierung	65
6.2.2	Beitrag des RL-Trainings	66
6.2.3	Gesamtbild	67
6.3	Ausblick	68
6.3.1	Weltgenerierung	68
6.3.2	Reinforcement Learning	69
	Literaturverzeichnis	70
A	Konfigurationsauszug	73
B	Autorschaftsübersicht	74
C	Rohdaten der statistischen Auswertung	75

Abbildungsverzeichnis

2.1	Value Noise – lineare gegen Smoothstep-Interpolation zwischen denselben Knotenwerten, eigene Darstellung.	5
2.2	Domain Warping – derselbe Rauschausschnitt unverzerrt und verzerrt, eigene Darstellung.	6
2.3	Zellulärer Automat – Rauschgitter vor und nach vier statt der konfigurier- ten zwei Glättungsiterationen, um den Effekt deutlicher zu zeigen, eigene Darstellung.	7
2.4	Breitensuche – Distanzfeld vom Startpunkt und daraus resultierender kür- zester Pfad, eigene Darstellung.	8
2.5	Interaktionsschleife zwischen Agent und Umgebung, eigene Darstellung. . .	9
2.6	Vergleich zwischen vollständiger und partieller Beobachtbarkeit, eigene Dar- stellung.	9
2.7	Veranschaulichung des Potentialfelds $\Phi(s)$ und der daraus resultierenden formenden Belohnung entlang eines Beispielpfads, eigene Darstellung. . . .	10
2.8	Darstellung der Clipped Surrogate Objective von PPO, eigene Darstellung.	11
2.9	Funktionsweise einer rekurrenten Policy im Vergleich zu einer zustandslosen Policy, eigene Darstellung.	12
2.10	Zusammenspiel von Replay Buffer und Target Network in der DQN-Architektur, eigene Darstellung.	13
3.1	Struktur des vierphasigen Curriculum-Trainings mit Steigerung der Exit- Distanz, eigene Darstellung.	20
3.2	Entscheidungsregel und ausgewertete Parameter der ε -Kompass-Heuristik, eigene Darstellung.	21
4.1	Prozedural generierte Seenstruktur mit organischer Uferlinie und Strand- Overlay, eigene Darstellung (Screenshot des Prototyps).	33
4.2	Vergleich von vier der sieben Biome, eigene Darstellung (Screenshots des Prototyps).	35
4.3	Aufbau des 229-dimensionalen Beobachtungsvektors, Konzeptdarstellung, eigene Darstellung.	44
4.4	Erfolgsquote der sieben v12-Läufe über die Trainingsschritte, je Curriculum- Phase, eigene Darstellung.	47
5.1	Generierte Spielwelt mit dynamischer Beleuchtung, Seenstrukturen und biom- abhängigen Kacheltypen, eigene Darstellung (Screenshot des Prototyps, Debug-Overlay entfernt).	52

5.2	Evaluation – Erfolgsquote der trainierten Verfahren und der Referenzpunkte auf Testset A, eigene Darstellung.	57
5.3	Evaluation – Pfadeffizienz der Verfahren in derselben Achsenordnung, eigene Darstellung.	58
5.4	Evaluation – gelaufener Agentenpfad gegen BFS-Optimum auf Seed 7000 (Testset A), eigene Darstellung.	59
5.5	Evaluation – Erfolgsquote der sieben LSTM-PPO-Läufe in deterministischer und stochastischer Auswertung, eigene Darstellung.	60
5.6	Evaluation – Erfolgsquote gegen Pfadeffizienz für alle Verfahren, eigene Darstellung.	61
6.1	Erfolgsquote des LSTM-PPO-Modells (n=7) gegen das Zielkriterium, Testset A und Holdout B, eigene Darstellung.	67

Tabellenverzeichnis

3.1	Konfigurationsstatus der Weltgenerierung	18
3.2	Übersicht der POMDP-Komponenten für das RL-Training	19
3.3	Diagnose- und Interventionsmatrix der aufgetretenen Trainingsinstabilitäten	23
4.1	Eingesetzte Werkzeuge der Weltgenerierung und des Clients	24
4.2	Eingesetzte Werkzeuge des RL-Trainings	42
4.3	Allgemeine Hyperparameter des PPO-Optimierers	51
5.1	Übersicht der Performancemessungen und Benchmarks des Generierungs- und Simulationssystems.	54
5.2	Belegte Wirkung einzelner Parameterentscheidungen	62
B.1	Zuordnung der Kapitel und Abschnitte der Arbeit	74
C.1	LSTM-PPO: Erfolgsquote je Lauf, Testset A und Holdout B, stochastisch und deterministisch	75
C.2	MLP-PPO: Erfolgsquote je Lauf, Testset A und Holdout B, stochastisch . .	75
C.3	Ungelernte Referenzverfahren: Erfolgsquote, mittlere Schrittzahl und Pfadef- fizienz, Testset A und Holdout B	76

Quelltextverzeichnis

4.1	Verwendung des 64-Bit SplitMix-Mixers und Value-Noise-Interpolation in <code>world.cpp</code>	25
4.2	Biom-Tagging und Schwellenwertauswertung in <code>world.cpp</code>	27
4.3	Freiradien-Erzeugung und Manhattan-Path-Carver in <code>world.cpp</code>	29
4.4	Low-Frequency-Value-Noise für zusammenhängende Gewässermasken in <code>world.cpp</code>	31
4.5	Nachbarschaftsbasierte Strandardarstellung im Render-Pfad in <code>render_engine.cpp</code>	32
4.6	Chunk-Speicherstruktur und Lazy-Loading-Mechanismus in <code>world.hpp</code> und <code>world.cpp</code>	34
4.7	Gewichtete Biom-Mischung und Tinting im Render-Pfad (<code>render_engine.cpp</code>)	35
4.8	Biomabhängige Struktur-Platzierung bei der Chunk-Generierung in <code>world.cpp</code>	37
4.9	Lazy Mob-Spawning und lokale Bewegungslogik in <code>simulation.cpp</code>	38
4.10	Diskrete Bewegungs- und Kollisionsprüfung in <code>simulation.cpp</code> und <code>object.cpp</code>	39
4.11	Flächenangriff des Spielers und Schadensabwicklung in <code>simulation.cpp</code> . .	41
4.12	Aufbau des Beobachtungsvektors, <code>python/stoneforge_env.py</code>	42
4.13	Instanziierung der Modelle mit asymmetrischen Parametern, <code>scripts/train_curriculum.py</code>	44
4.14	Curriculum-Phasen mit dualen Abbruchbedingungen, <code>scripts/train_curriculum.py</code>	46
4.15	Berechnung des Rewards und Shaping-Terms, <code>src/core/simulation.cpp</code> .	48
4.16	Aktionswahl der Referenzverfahren, <code>scripts/eval_baselines.py</code>	50
A.1	Weltgenerierungs-Konfiguration (Auszug)	73

Abkürzungsverzeichnis

Abkürzung	Bedeutung
A2C	Advantage Actor-Critic
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BFS	Breitensuche (Breadth-First Search)
CA	Zelluläre Automaten (Cellular Automaton)
CLI	Command Line Interface
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DQN	Deep Q-Network
FPS	Frames per Second
KI	Künstliche Intelligenz
LRU	Least Recently Used
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MPS	Metal Performance Shaders
PBRs	Potential-Based Reward Shaping
PCGRL	Procedural Content Generation via Reinforcement Learning
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
PRNG	Pseudo-Random Number Generator
RAM	Random Access Memory
RL	Reinforcement Learning
RNN	Recurrent Neural Network
SR	Success Rate
TRPO	Trust Region Policy Optimization

1 Einleitung

1.1 Motivation

Die Arbeit verbindet zwei Themenfelder, die sich in der Praxis selten in einem gemeinsamen Projekt begegnen: prozedurale Spielweltgenerierung und Reinforcement Learning. Beide Bereiche greifen ineinander, sobald eine Spielwelt nicht mehr von Hand entworfen, sondern algorithmisch erzeugt wird und ein lernender Agent auf dieser Grundlage navigieren soll.

Als Vorbild dienten Spiele wie *Minecraft*, deren Reiz zu einem großen Teil darin liegt, dass Welten bei jedem Start prozedural neu erzeugt werden. Ziel dieser Arbeit ist es, ein ähnliches Prinzip im Kleinen umzusetzen: Eine einfache, prozedural generierte Spielwelt zu bauen und darauf einen RL-Agenten zu trainieren, der lernen muss, sich in dieser unbekannten Umgebung zurechtzufinden und den Ausgang zu erreichen.

1.2 Problemstellung

Das zentrale Problem in der Spielwelt ist, dass der Agent zu keinem Zeitpunkt die gesamte Karte sieht. Er bekommt immer nur einen kleinen Ausschnitt direkt um seine eigene Position herum angezeigt und muss trotzdem versuchen, den Ausgang zu finden, der irgendwo im unentdeckten Bereich liegt.

Diese Einschränkung wurde bewusst eingebaut: Wenn der Agent immer die komplette Karte auf einmal sehen könnte, wäre die Aufgabe deutlich zu einfach. Das ließe sich dann bereits mit wenigen festen Regeln oder einfachen Algorithmen lösen, ohne dass ein RL-Agent tatsächlich etwas lernen müsste.

Durch die eingeschränkte Sicht muss der Agent jedoch mit unvollständigem Wissen arbeiten, etwa so wie ein Mensch, der sich ohne Karte durch ein völlig unbekanntes Gebäude tastet. Der Agent muss also durch eine ihm unbekannte Welt navigieren, die er erst beim Durchlaufen Schritt für Schritt entdeckt.

1.3 Zielsetzung und Forschungsfrage

Das Ziel der Arbeit ist es, eine eigene prozedurale 2D-Spielwelt zu bauen und darauf verschiedene RL-Agenten für eine Navigationsaufgabe mit eingeschränkter Sicht zu trainieren.

Daraus ergibt sich die zentrale Hauptfrage: Wie muss eine prozedural generierte 2D-Spielwelt aufgebaut sein und welche Kombination aus Policy-Architektur und Trainingsverfahren erzielt darin die besten Ergebnisse bei der Navigation?

Konkret gliedert sich dies in zwei Teilfragen:

- **Weltdesign:** Wie lässt sich eine performante, seedbasierte, abwechslungsreiche 2D-Gridwelt erstellen, die durch ein Ziel lösbar ist und deren Grundmechaniken gut für eine KI lernbar sind?
- **Algorithmen & Parameter:** Welche Kombination aus Policy-Architektur und Trainingsverfahren erreicht die höchste Erfolgsquote und die besten Wege zum Ausgang?

1.4 Vorgehen und Methodik

Das Vorgehen gliedert sich in vier Schritte:

1. **Recherche zu Weltgenerierung und RL-Algorithmen:** Zunächst wird recherchiert, wie eine prozedurale Generierung sinnvoll aufgebaut sein sollte und welche Algorithmen sich eignen, um zufällige, aber immer lösbare 2D-Welten zu erzeugen. Parallel dazu wird untersucht, welche RL-Algorithmen für Navigationsaufgaben mit eingeschränkter Sicht überhaupt infrage kommen.
2. **Prototyping der Spielumgebung:** Auf dieser Basis entsteht ein erster Prototyp der Spielwelt. Dieser wird iterativ verbessert, bis Parameter wie Exit-Distanz, Waddichte und die Sichtweite des Agenten gut zusammenpassen und die Level fair bleiben.
3. **Training der RL-Agenten:** Sobald die Spielumgebung stabil steht, werden die ausgewählten RL-Algorithmen darauf trainiert und die Hyperparameter Schritt für Schritt angepasst.
4. **Evaluation und Auswertung:** Zum Schluss werden die fertig trainierten Agenten auf vielen neu generierten Leveln getestet. Die Ergebnisse, etwa Erfolgsquote und Pfadefizienz, werden systematisch erfasst und die Verfahren miteinander verglichen.

1.5 Abgrenzung

Damit der Rahmen der Projektarbeit überschaubar bleibt, werden folgende Punkte bewusst ausgeklammert:

- **Fokus auf die eigene Spielwelt:** Untersucht wird ausschließlich die selbst entwickelte 2D-Gitterwelt. Eine Übertragbarkeit auf 3D-Umgebungen oder andere externe Benchmarks wird nicht geprüft.
- **Kein RL für die Weltgenerierung:** Reinforcement Learning wird ausschließlich für das Navigationsverhalten des Agenten eingesetzt. Die Spielwelt selbst wird über feste, klassische Algorithmen erzeugt (kein gelerntes Weltdesign).
- **Begrenzte Parameterstudie:** Auf eine erschöpfende Testreihe über alle denkbaren Kombinationen aus Waddichten oder Exit-Distanzen wird verzichtet; der Fokus liegt auf den ausgewählten RL-Verfahren und praxisnahen Welteinstellungen.

1.6 Verwandte Arbeiten

Der eigene Ansatz lässt sich in bestehende Arbeiten einordnen, die sich mit gitterbasierten Spielwelten und prozeduraler Generierung für Reinforcement Learning beschäftigen:

- **Gitterbasierte Testwelten (MiniGrid & Crafter):** Umgebungen wie *MiniGrid* [3] oder *Crafter* [6] verfolgen ein ganz ähnliches Prinzip wie das vorliegende Projekt. Sie bieten leichtgewichtige 2D-Gitterwelten, in denen der Agent oft nur einen begrenzten Ausschnitt der Karte sieht und einfache Navigations- oder Sammelaufgaben lösen muss.
- **Generalisierung durch prozedurale Level (Procgen):** Der *Procgen Benchmark* [4] untersucht gezielt, wie gut RL-Agenten auf immer wieder neu generierten Levels verallgemeinern. Das bestätigt die Grundidee dieser Arbeit, dass prozedurale Welten verhindern, dass ein Agent feste Pfade einfach nur auswendig lernt.
- **Partielle Beobachtbarkeit und Gedächtnis (POPGym):** Besonders nah am RL-Teil dieser Arbeit ist *POPGym* [13]. Dort werden ganz gezielt Umgebungen mit eingeschränkter Sicht (partielle Beobachtbarkeit) untersucht und es wird verglichen, wie sich Modelle mit Gedächtnis (rekurrente Policies) gegen Modelle ohne Gedächtnis schlagen. Das ist exakt der Ansatz, der auch für den Vergleich der Trainingsverfahren in dieser Arbeit übernommen wird.
- **Abgrenzung zu gelerntem Weltdesign (PCGRL):** In Arbeiten wie *PCGRL* [11] wird Reinforcement Learning eingesetzt, um das Generieren von Spielwelten selbst zu erlernen. Davon grenzt sich diese Arbeit klar ab: Die Erzeugung der Spielwelt bleibt fest über eigene Algorithmen programmiert, das RL-Verfahren dient ausschließlich dazu, das Navigationsverhalten des Agenten zu trainieren.

2 Grundlagen

Dieses Kapitel legt die theoretischen und mathematischen Grundlagen für beide Teile der Arbeit. Zunächst behandelt Abschnitt 2.1 die Grundlagen der prozeduralen Weltgenerierung, von deterministischem Zufall über Rauschverfahren bis zur graphentheoretischen Erreichbarkeitsanalyse. Im Anschluss folgen in Abschnitt 2.2 die Grundlagen des Reinforcement Learning.

2.1 Grundlagen der prozeduralen Weltgenerierung

Prozedurale Generierung bezeichnet Verfahren, bei denen digitale Inhalte nicht manuell durch Designer erstellt, sondern algorithmisch zur Laufzeit berechnet werden. Dies ermöglicht die Erzeugung sehr großer oder theoretisch unendlicher Spielwelten bei geringem Speicherbedarf. Die folgenden Abschnitte bauen aufeinander auf: Zunächst wird die deterministische Zufallsgrundlage behandelt (Abschnitt 2.1.1), darauf aufbauend die daraus abgeleiteten Rauschverfahren (Abschnitt 2.1.2) und deren nachträgliche Glättung (Abschnitt 2.1.3). Den Abschluss bildet die graphentheoretische Absicherung der Spielbarkeit (Abschnitt 2.1.4).

2.1.1 Deterministischer Zufall und Pseudozufallszahlen

In der Informatik ist echter Zufall (*True Randomness*) ohne spezielle Hardware-Schnittstellen schwer zu realisieren. Stattdessen kommen Pseudozufallszahlengeneratoren (*Pseudo-Random Number Generators*, PRNG) zum Einsatz. Ein wesentliches Merkmal für die Weltgenerierung ist der Determinismus:

- **Seed (Startwert):** Ein PRNG erzeugt eine deterministische Zahlenfolge basierend auf einem initialen Eingabewert (Seed). Bei identischem Seed erzeugt der Algorithmus jedes Mal exakt dieselbe Zahlenfolge.
- **Hash- und Mix-Funktionen:** Um räumliche Daten (wie zweidimensionale Koordinaten (x, y)) unabhängig von einem fortlaufenden Zustand abzufragen, werden häufig zustandslose Hash-Funktionen genutzt, etwa nach dem Vorbild schneller, splittbarer Mix-Konstruktionen wie *SplitMix* [20]. Diese bilden Raumkoordinaten, den Seed und ein optionales *Salt* (ein zusätzlicher Offset zur Unterscheidung verschiedener Abfragen) auf einen pseudozufälligen Wert im Intervall $[0, 1]$ ab.

2.1.2 Gradienten- und Value-Noise-Verfahren

Die direkte Verwendung unkorrelierter Pseudozufallswerte für benachbarte Raumpunkte führt zu einer ungeordneten Struktur (weißes Rauschen). Für die Simulation natürlicher

Landschaften werden daher *Coherent-Noise*-Verfahren genutzt, bei denen nahe beieinander liegende Punkte ähnliche Werte aufweisen.

Value Noise und Interpolation

Beim *Value Noise* werden an den Knotenpunkten eines regulären Gitters pseudozufällige Skalarwerte bestimmt. Die Werte für Punkte innerhalb einer Gitterzelle berechnen sich anschließend durch Interpolation der umliegenden Knotenwerte. Das Grundprinzip kohärenten, gitterbasierten Rauschens für die Bildsynthese geht auf die Arbeit von Perlin [16] zurück.

Um stetige Übergänge ohne harte Kanten an den Zellgrenzen zu gewährleisten, wird der Interaktionsparameter $t \in [0, 1]$ vor der linearen Interpolation (*Lerp*) durch eine S-förmige Glättungsfunktion (*Smoothstep*) transformiert:

$$S(t) = 3t^2 - 2t^3 \quad (2.1)$$

Abbildung 2.1 stellt beide Interpolationsvarianten an denselben pseudozufälligen Knotenwerten gegenüber. Der Unterschied zeigt sich am deutlichsten an den Knoten selbst: Die lineare Variante bildet dort sichtbare Knicke, während die Smoothstep-Kurve glatt hindurchläuft.

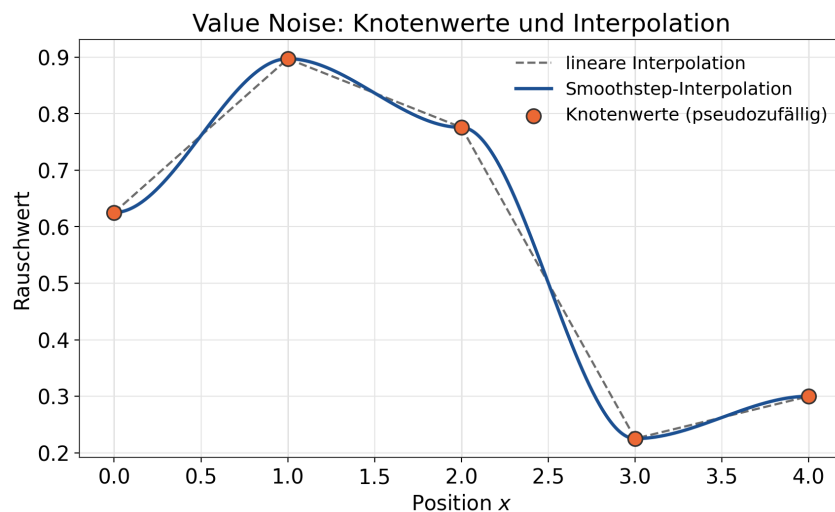


Abbildung 2.1: Value Noise – lineare gegen Smoothstep-Interpolation zwischen denselben Knotenwerten, eigene Darstellung.

Im zweidimensionalen Raum erfolgt die Überlagerung der Interpolationen entlang beider Achsen mittels bilinearer Interpolation.

Domain Warping und fraktale Akkumulation

Um die typischen Raster-Artefakte einfacher Gitterstrukturen zu vermeiden, werden fortgeschrittene Techniken eingesetzt:

- **Multi-Frequency / Fraktaler Noise:** Durch die Addition mehrerer Rauschfunktionen unterschiedlicher Frequenzen (Oktaven) und Amplituden lassen sich fraktal-

ähnliche, hochdetaillierte Strukturen erzeugen. Niedrige Frequenzen steuern großflächige Formen (zum Beispiel Kontinente), während hohe Frequenzen feine Details (zum Beispiel Felsstrukturen) ergänzen.

- **Domain Warping** [17]: Hierbei werden die Eingangskoordinaten (x, y) einer Rauschfunktion selbst durch den Ausgangswert einer anderen Rauschfunktion verzerrt:

$$f_{\text{warped}}(x, y) = f(x + g(x, y), y + h(x, y)) \quad (2.2)$$

Dies führt zu organischen Verformungen und eliminiert visuelle Gitterlinien.

Abbildung 2.2 zeigt denselben Rauschausschnitt mit und ohne Domain Warping. Die verzerrte Variante rechts löst die diagonalen Vorzugsrichtungen des unverzerrten Rauschens auf und wirkt dadurch weniger regelmäßig.

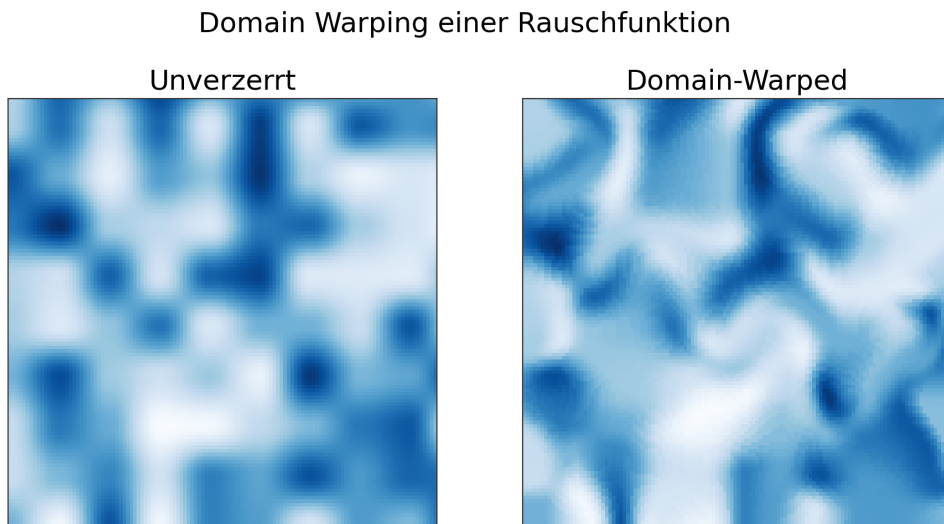


Abbildung 2.2: Domain Warping – derselbe Rauschausschnitt unverzerrt und verzerrt, eigene Darstellung.

2.1.3 Zelluläre Automaten zur Gefügeglättung

Zelluläre Automaten (*Cellular Automata*) bestehen aus Zellen, die diskrete Zustände (zum Beispiel *Wand* oder *Freiraum*) annehmen können. In diskreten Zeitschritten aktualisiert jede Zelle ihren Zustand basierend auf einer lokalen Übergangsfunktion, welche die Zustände ihrer Nachbarschaft berücksichtigt.

Häufig wird hierbei die Moore-Nachbarschaft verwendet, welche die 8 direkt angrenzenden Zellen (horizontal, vertikal und diagonal) umfasst. Durch gezielte Geburts- und Überlebensregeln (*Birth/Survival Rules*) lassen sich aus verrauschten Initialzuständen zusammenhängende, natürliche Höhlen- und Klippenstrukturen erzeugen, wie es für die prozedurale Erzeugung von Höhlenlevels bereits gezeigt wurde [9]. Abbildung 2.3 zeigt diesen Effekt an einem unabhängigen Beispielgitter mit der auch in der Projektkonfiguration hinterlegten Regel B5/S4 (Geburt ab 5, Überleben ab 4 Nachbarwänden): Aus unstrukturiertem Rauschen entstehen nach wenigen Iterationen zusammenhängende Flächen.

Im ausgelieferten Konfigurationsstand ist diese Nachbearbeitung deaktiviert (siehe Tabelle 3.1). Sie wird hier dennoch als Grundlage behandelt, weil sie die naheliegende algorithmische Alternative zum tatsächlich verwendeten Manhattan-Fallback markiert (siehe Abschnitt 2.1.4) und dessen Auswahl erst vor diesem Hintergrund einordbar wird.

Zelluläre Automaten zur Gefügeglättung



Abbildung 2.3: Zellulärer Automat – Rauschgitter vor und nach vier statt der konfigurierten zwei Glättungsiterationen, um den Effekt deutlicher zu zeigen, eigene Darstellung.

2.1.4 Graphentheoretische Erreichbarkeitsanalyse

Ein zentrales Kriterium prozeduraler Welten ist die Sicherstellung der Spielbarkeit, insbesondere die Vermeidung unlösbarer Level-Zustände (*Deadlocks*).

Breitensuche (Breadth-First Search, BFS)

Zur Analyse der Konnektivität wird das Zellgitter als ungerichteter Graph interpretiert, in dem begehbare Zellen als Knoten und Nachbarschaftsbeziehungen als Kanten dargestellt werden. Die Breitensuche (BFS) traversiert den Graphen ebenenweise vom Startknoten aus [5]. Sie eignet sich zur Prüfung der Erreichbarkeit eines Zielknotens sowie zur Bestimmung des kürzesten Pfades auf ungewichteten Gittern. Abbildung 2.4 veranschaulicht das resultierende Distanzfeld an einem Beispielgitter: Jede Ebene entspricht einem BFS-Schritt vom Start aus, der kürzeste Pfad zum Ziel ergibt sich durch Rückverfolgung der jeweils näher liegenden Nachbarzelle.

Im ausgelieferten Konfigurationsstand dient die Breitensuche nicht mehr der nachträglichen Erreichbarkeitsprüfung bei der Weltgenerierung (siehe Tabelle 3.1); diese Validierung ist deaktiviert. Lösbarkeit wird stattdessen vorgelagert erzwungen, indem der Ausgang selbst nur unter Positionen gewählt wird, die eine Breitensuche vom Spawnpunkt aus als erreichbar bestätigt (siehe Abschnitt 4.1.4). Die Breitensuche bleibt damit an mehreren Stellen aktiv: bei der Ausgangsplatzierung, als Distanzfeld für das Reward Shaping im RL-Training (Abschnitt 4.2.5) und als Bezugsgröße der Pfadeffizienz-Metrik in der Evaluation (Abschnitt 5.2.2).

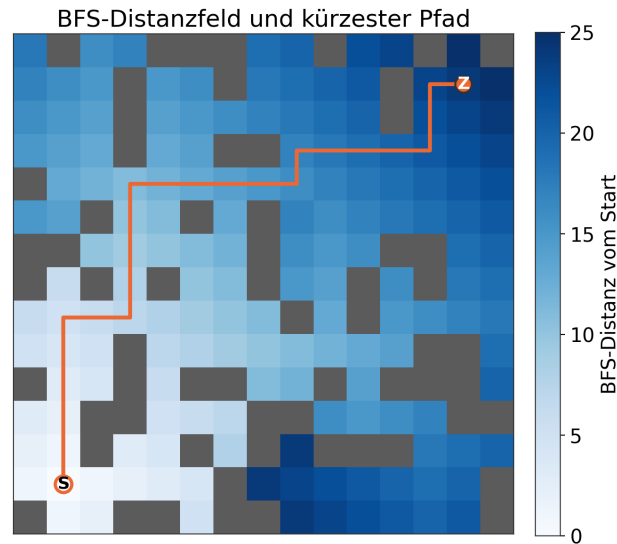


Abbildung 2.4: Breitensuche – Distanzfeld vom Startpunkt und daraus resultierender kürzester Pfad, eigene Darstellung.

Heuristische Validierung und Korrekturverfahren

Da die exakte graphenbasierte Validierung bei großen Chunks rechenintensiv sein kann, werden in der Praxis häufig kombinierte Ansätze gewählt:

- **Manhattan-Distanz:** Die Distanz zwischen zwei Punkten (x_1, y_1) und (x_2, y_2) auf einem orthogonalen Gitter berechnet sich nach:

$$D_{\text{Manhattan}} = |x_1 - x_2| + |y_1 - y_2| \quad (2.3)$$

- **Fallback-Generierung (Carving):** Sollte keine valide Verbindung existieren, stellen geometrische Ausgrabungsalgorithmen (*Carving*) entlang direkter Achsen eine funktionale Wegeverbindung sicher.

2.2 Grundlagen des Reinforcement Learning

Das Reinforcement Learning (RL) ist ein Teilbereich des maschinellen Lernens, bei dem ein Agent lernt, durch Interaktion mit einer Umgebung optimale Entscheidungen zu treffen (siehe Abbildung 2.5). Die formale mathematische Grundlage hierfür bildet der **Markov Decision Process (MDP)**. Ein MDP wird typischerweise als Tupel (S, A, P, R, γ) definiert. Dabei steht S für die Menge aller möglichen Zustände der Umgebung und A für die Menge der möglichen Aktionen des Agenten.

Die Übergangswahrscheinlichkeit $P(s' | s, a)$ beschreibt die Dynamik der Umgebung, also mit welcher Wahrscheinlichkeit der Agent durch die Aktion a vom Zustand s in den Folgezustand s' gelangt. Für jeden dieser Übergänge erhält der Agent ein Feedback in Form eines numerischen Belohnungssignals, dem **Reward** $R(s, a)$. Das übergeordnete Ziel des Agenten ist es, eine Verhaltensregel, die sogenannte **Policy** $\pi(a | s)$, zu erlernen. Diese Policy ordnet jedem Zustand eine Aktion zu und maximiert dabei die erwartete,

kumulierte Belohnung über die Zeit. Der Diskontierungsfaktor $\gamma \in [0, 1]$ bestimmt dabei, wie stark zukünftige Belohnungen im Vergleich zu unmittelbaren gewichtet werden [21].

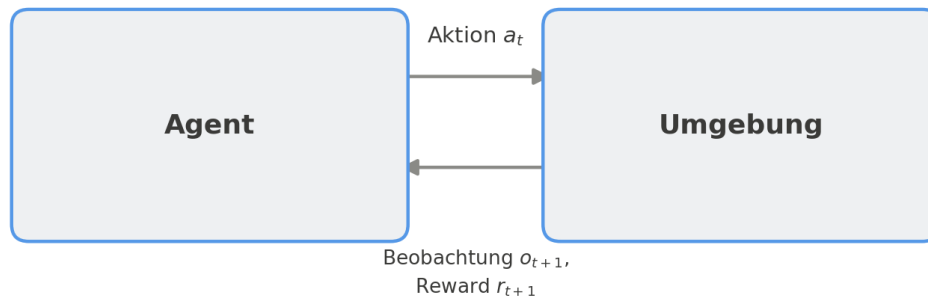


Abbildung 2.5: Interaktionsschleife zwischen Agent und Umgebung, eigene Darstellung.

2.2.1 Partially Observable Markov Decision Processes (POMDP)

Die Standarddefinition des MDP geht von vollständiger Beobachtbarkeit aus. Der Agent kennt stets den exakten aktuellen Zustand s der Umgebung. In vielen realen Szenarien ist diese Annahme jedoch nicht erfüllt, da Sensordaten verrauscht oder Informationen verborgen sein können. Für solche Problemstellungen wird das Konzept zum *Partially Observable Markov Decision Process* (POMDP) erweitert. In einem POMDP hat der Agent keinen direkten Zugriff auf den wahren Zustand $s \in S$, sondern erhält lediglich eine Beobachtung $o \in \Omega$, die gemäß einer Beobachtungswahrscheinlichkeit $O(o | s, a)$ erzeugt wird (siehe Abbildung 2.6). Um dennoch optimale Entscheidungen treffen zu können, muss der Agent einen sogenannten „Belief State“ (Überzeugungszustand) pflegen. Dieser stellt eine Wahrscheinlichkeitsverteilung über alle möglichen wahren Zustände der Umgebung dar und wird nach jeder neuen Aktion und Beobachtung kontinuierlich aktualisiert [10].

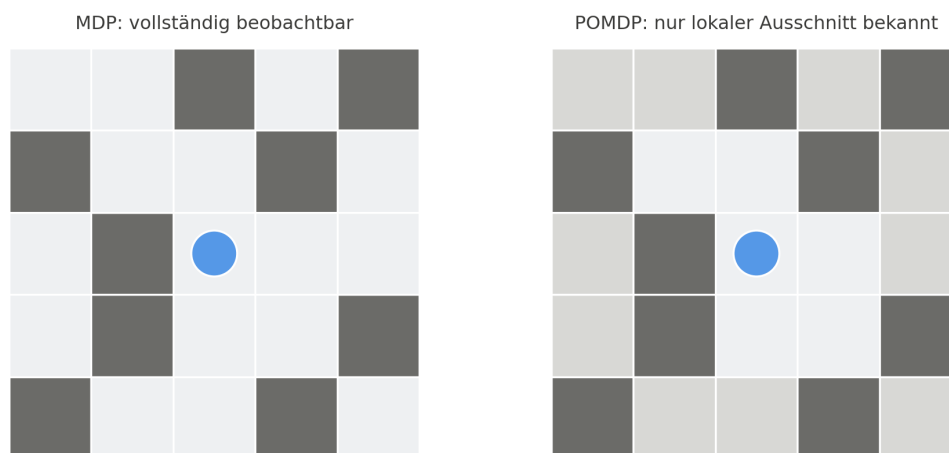


Abbildung 2.6: Vergleich zwischen vollständiger und partieller Beobachtbarkeit, eigene Darstellung.

2.2.2 Potential Based Reward Shaping (PBRs)

Eine zentrale Herausforderung im Reinforcement Learning sind verzögerte oder spärliche Belohnungen (Sparse Rewards), bei denen der Agent nur sehr selten ein positives Feedback erhält. Um den Lernprozess zu beschleunigen, wird häufig *Reward Shaping* angewendet, bei dem das ursprüngliche Belohnungssignal durch zusätzliche, dichtere Zwischenbelohnungen angereichert wird. Die allgemeine Theorie des *Potential Based Reward Shaping* (PBRs) löst dabei das Problem, dass naive Anpassungen der Belohnungsfunktion das eigentliche Ziel des Agenten verfälschen könnten (sogenanntes Reward Hacking). PBRs stellt mathematisch sicher, dass die optimale Policy des ursprünglichen MDPs trotz der zusätzlichen Belohnungen strikt erhalten bleibt. Dies wird erreicht, indem die formende Belohnung $F(s, a, s')$ als Differenz einer Potenzialfunktion Φ zwischen dem Folgezustand und dem aktuellen Zustand definiert wird (vergleiche Abbildung 2.7): $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$. Diese Funktion gibt dem Agenten kontinuierliches Feedback basierend auf seinem „Abstand“ zum Ziel [15].

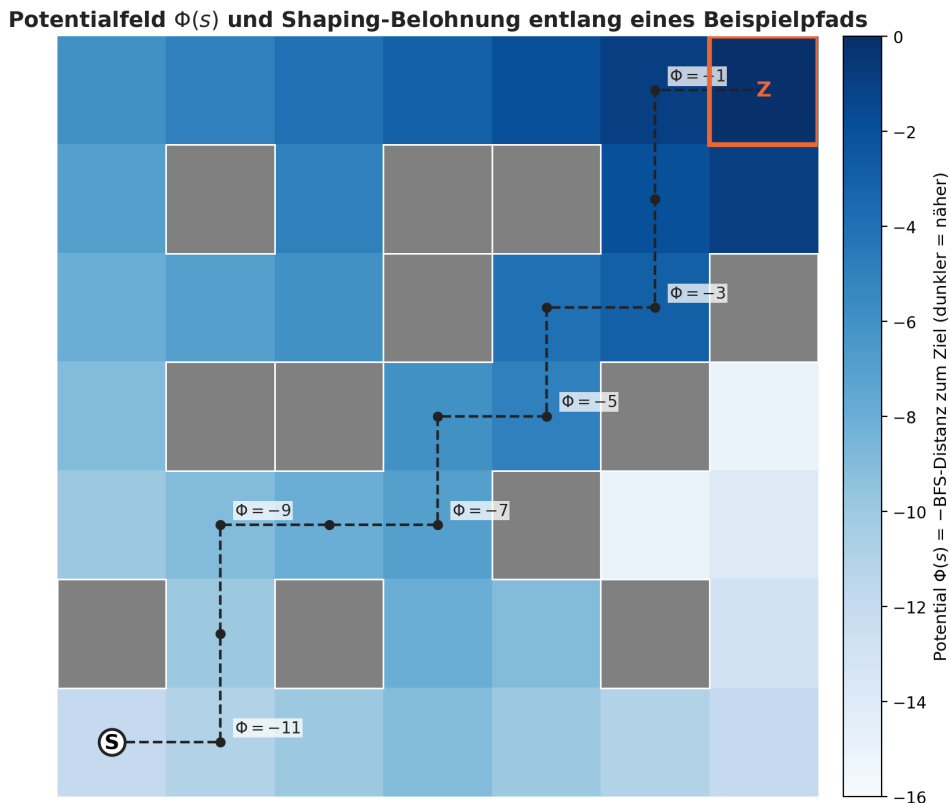


Abbildung 2.7: Veranschaulichung des Potentialfelds $\Phi(s)$ und der daraus resultierenden formenden Belohnung entlang eines Beispielpfads, eigene Darstellung.

2.2.3 Curriculum Learning

Wenn Aufgaben für einen untrainierten RL Agenten zu komplex sind, schlägt das Lernen oft vollständig fehl, da der Agent rein durch zufällige Exploration keine Lösung findet. Das allgemeine Prinzip des *Curriculum Learning* greift dieses Problem auf, indem es sich an der menschlichen Pädagogik orientiert: Anstatt dem Agenten sofort die finale, hochkomplexe

Aufgabe zu stellen, wird der Lernprozess strukturiert. Der Agent trainiert zunächst auf stark vereinfachten Versionen der Aufgabe. Sobald er diese beherrscht, wird der Schwierigkeitsgrad der Umgebung oder der Aufgabenstellung schrittweise erhöht, bis die Zieldomäne erreicht ist. Diese methodische Abfolge (das Curriculum) beschleunigt nicht nur die Konvergenz des Lernalgorithmus erheblich, sondern ermöglicht es oft erst, dass der Agent für sehr komplexe Problemstellungen überhaupt eine optimale Policy findet [1, 14].

2.2.4 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) gehört zur Familie der Policy Gradient Methoden und wurde von Schulman et al. [19] als Weiterentwicklung etablierter Verfahren wie der Trust Region Policy Optimization (TRPO) eingeführt. Der zentrale Vorteil von PPO liegt in der Balance zwischen einfacher Implementierung, hoher Stichprobeneffizienz (Sample Efficiency) und Trainingsstabilität. Während herkömmliche Methoden des Policy Gradient bei zu großen Aktualisierungsschritten destabilisieren und die erlernte Verhaltensregel ruinieren können, führt PPO eine sogenannte Clippingfunktion (*Clipped Surrogate Objective*) ein (vergleiche Abbildung 2.8). Diese Zielfunktion beschränkt die Änderungen der Policy zwischen zwei aufeinanderfolgenden Trainingsschritten auf ein definiertes Intervall, meist parametrisiert durch einen Hyperparameter ϵ (zum Beispiel $[1 - \epsilon, 1 + \epsilon]$). Dadurch wird mathematisch verhindert, dass der Agent seine Strategie in einem einzelnen Updateschritt zu drastisch ändert. Dies führt zu einer monotoneren Verbesserung und einem wesentlich robusteren Lernverhalten, ohne den hohen rechnerischen Aufwand, der bei TRPO für die Berechnung der Kullback Leibler Divergenz anfällt [19].

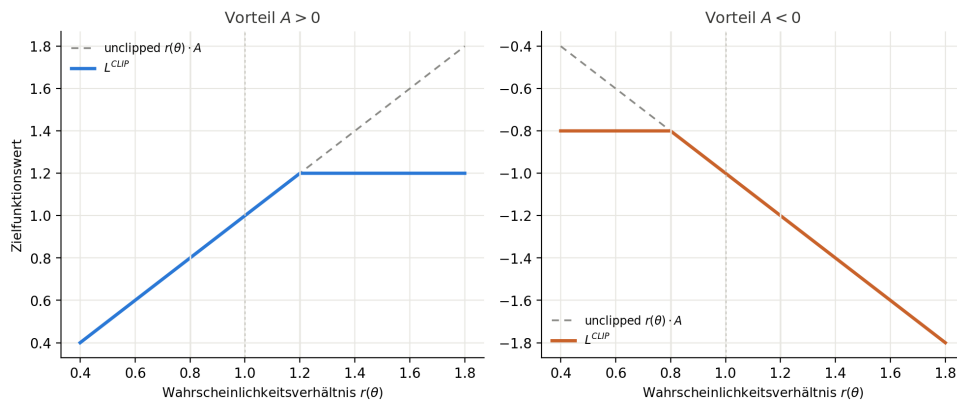


Abbildung 2.8: Darstellung der Clipped Surrogate Objective von PPO, eigene Darstellung.

2.2.5 Rekurrente Neuronale Netze (RNN)

Wie in der Definition der POMDPs beschrieben, kann ein Agent in Umgebungen mit partieller Beobachtbarkeit den wahren Zustand nicht aus einer einzelnen, isolierten Beobachtung ableiten. Um dieses Problem zu überwinden, muss der Agent auf vergangene Beobachtungen zurückgreifen können, wofür rekurrente neuronale Netze (RNNs) eingesetzt werden. Im Gegensatz zu zustandslosen Netzwerken (Feedforward Netzen) besitzen RNNs einen internen Speicher, der bei jedem Zeitschritt basierend auf der aktuellen Eingabe und

dem vorherigen Zustand aktualisiert wird. Durch diesen impliziten Gedächtnisaufbau kann das Netzwerk die verborgenen Zustandsinformationen eines POMDPs aus der Historie der Beobachtungen näherungsweise rekonstruieren. Eine rekurrente Policy ist dadurch in der Lage, auch dann optimale Entscheidungen zu treffen, wenn das aktuelle Beobachtungssignal allein keine eindeutige Zustandsschätzung zulässt [7]. Ein zentrales Problem einfacher RNNs ist jedoch das Phänomen des verschwindenden Gradienten (*Vanishing Gradient*), wodurch das Netzwerk große Schwierigkeiten hat, Abhängigkeiten über längere Zeiträume stabil zu erlernen.

2.2.6 Long Short Term Memory (LSTM)

Um das Problem des verschwindenden Gradienten zu lösen, bildet die *Long Short Term Memory* Architektur die fundamentale Basis für moderne rekurrente Netze. Sie wurde von Hochreiter and Schmidhuber [8] entwickelt und stellt eine spezialisierte Form von RNNs dar. LSTMs nutzen ein System aus dedizierten Speicherzellen und Gatingmechanismen (Input Gates, Output Gates und Forget Gates). Diese Struktur ermöglicht es dem Netzwerk gezielt zu steuern, welche relevanten Informationen über lange Zeitreihen hinweg gespeichert, aktualisiert oder vergessen werden sollen. Für das Reinforcement Learning ist dies von zentraler Bedeutung: Der Agent kann dadurch wichtige Ereignisse, die viele Schritte in der Vergangenheit liegen, verlustfrei in seinem Gedächtnis behalten und für aktuelle Entscheidungen heranziehen (siehe Abbildung 2.9). Die Integration von LSTM-Zellen ermöglicht es dem Agenten dadurch, Sequenzen von Beobachtungen über die Zeit effektiv zu aggregieren und komplexe Aufgaben in unvollständig beobachtbaren Umgebungen erfolgreich zu lösen [7, 8].

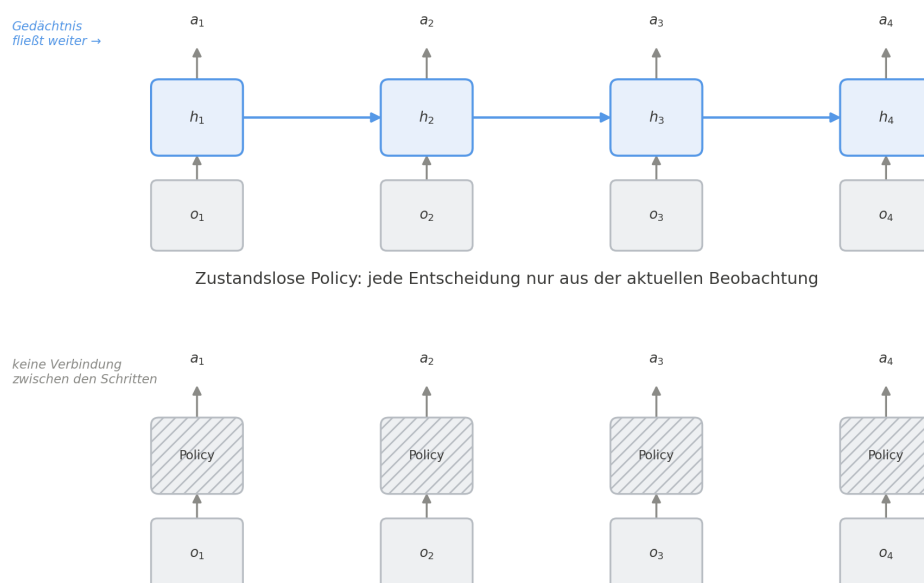


Abbildung 2.9: Funktionsweise einer rekurrenten Policy im Vergleich zu einer zustandslosen Policy, eigene Darstellung.

2.2.7 Deep Q Networks (DQN)

Deep Q Networks (DQN) stellen einen wichtigen Meilenstein in der Entwicklung des modernen Reinforcement Learning dar und wurden maßgeblich durch Mnih et al. [12] geprägt. Im Gegensatz zu den zuvor beschriebenen Methoden des Policy Gradient handelt es sich bei DQN um einen wertebasierten Ansatz. Anstatt die Policy direkt zu parametrisieren, approximiert ein tiefes neuronales Netz die sogenannte Q Funktion (Action Value Funktion). Diese Funktion schätzt die erwartete kumulierte Belohnung für eine spezifische Aktion in einem bestimmten Zustand. Da die Kombination von klassischem Q Learning mit nichtlinearen Funktionsapproximatoren wie neuronalen Netzen historisch oft zu starker Instabilität und Divergenz im Lernverhalten führte, führte DQN zwei zentrale Innovationen ein (siehe Abbildung 2.10).

Zum einen das *Experience Replay*, bei dem vergangene Interaktionen des Agenten kontinuierlich in einem Ringpuffer gespeichert und während des Trainings in zufälligen Batches wiederverwendet werden. Dies bricht die zeitliche Korrelation der Trainingsdaten auf und glättet die Datenverteilung. Zum anderen wird ein separates *Target Network* genutzt, dessen Parameter im Gegensatz zum primären Netzwerk nur periodisch aktualisiert werden. Dies hält die Zielwerte für die Fehlerberechnung über mehrere Trainingsschritte hinweg konstant und verhindert Aufschaukelungseffekte. Durch diese beiden Mechanismen ermöglichte DQN erstmals das stabile Erlernen komplexer Verhaltensstrategien direkt aus hochdimensionalen Beobachtungen [12].

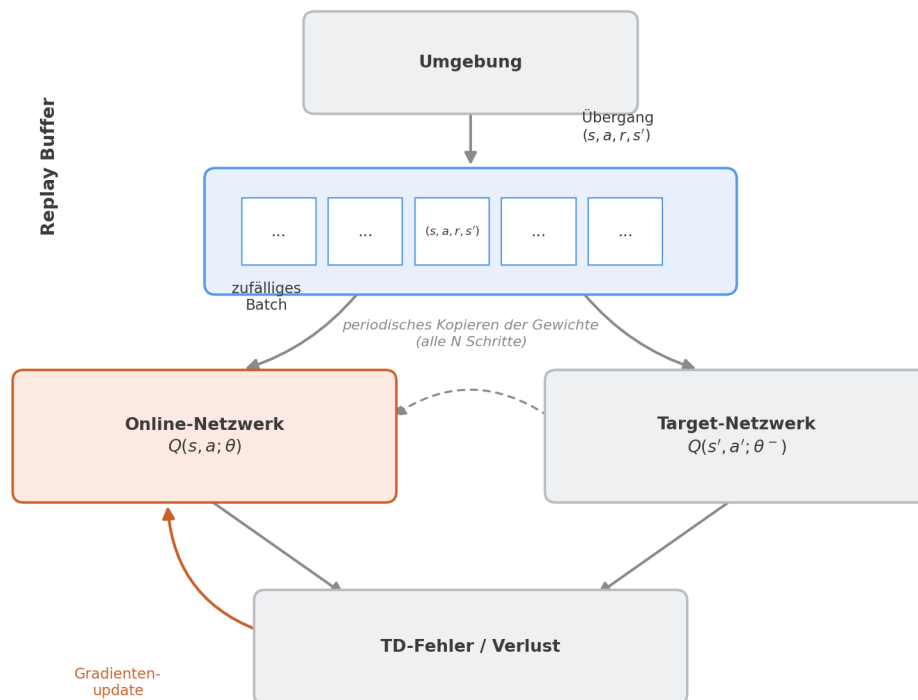


Abbildung 2.10: Zusammenspiel von Replay Buffer und Target Network in der DQN-Architektur, eigene Darstellung.

2.2.8 Die Gymnasium-Schnittstelle (ehemals OpenAI Gym)

Um Reinforcement-Learning-Algorithmen unabhängig von der konkreten Problemstellung entwickeln und testen zu können, bedarf es einer standardisierten Kommunikationsschicht zwischen dem lernenden Agenten und der simulierten Welt. In der Forschung und Industrie hat sich hierfür ein dominanter Standard etabliert, der ursprünglich 2016 von OpenAI unter dem Namen „Gym“ veröffentlicht wurde [2]. Um die langfristige Wartung und Weiterentwicklung zu sichern, wurde das Projekt Ende 2022 an die gemeinnützige Farama Foundation übergeben und wird seitdem unter dem Namen „Gymnasium“ fortgeführt [22].

Die Kernidee von Gymnasium ist die strikte Entkopplung: Der Algorithmus muss nicht wissen, ob er ein Videospiel spielt, einen Roboter steuert oder eine Lagerhalle optimiert. Die Umgebung (**Environment** oder kurz **Env**) kapselt die gesamte interne Logik, Physik und Zustandsspeicherung und gibt nach außen hin nur eine simple, wohldefinierte Programmierschnittstelle (**API**) preis. Diese API modelliert exakt den in der Theorie definierten Markov-Entscheidungsprozess (MDP) über folgende Hauptkomponenten:

Definition der Zustands- und Aktionsräume Bevor das Training beginnt, muss der Agent wissen, welche Sensordaten er empfängt und welche Aktionen ihm zur Verfügung stehen. Gymnasium definiert dafür sogenannte **spaces**:

- **observation_space**: Definiert das Format und die Wertebereiche der Beobachtungen, die die Umgebung zurückgibt (zum Beispiel ein mehrdimensionaler Vektor für Kamerabilder oder Sensordaten, repräsentiert als **Box**-Raum).
- **action_space**: Definiert die erlaubten Aktionen, beispielsweise diskrete Eingaben (hoch, runter, links, rechts) als **Discrete**-Raum oder kontinuierliche Werte für Motoren als **Box**-Raum.

Der Interaktionszyklus (reset** und **step**)** Die eigentliche Interaktion verläuft zyklisch und wird über zwei zentrale Funktionen gesteuert:

- **reset()**: Setzt die Umgebung in einen initialen Startzustand zurück und liefert die allererste Beobachtung an den Agenten. Dies markiert den Beginn einer neuen Episode.
- **step(action)**: Der Agent übergibt seine gewählte Aktion an die Umgebung. Die Umgebung simuliert daraufhin genau einen Zeitschritt und gibt ein Tupel (früher vier, in der aktuellen Gymnasium-Version fünf Werte) zurück:
 - **observation**: Die neue Beobachtung des resultierenden Zustands (s_{t+1}).
 - **reward**: Die skalare Belohnung (r_{t+1}) für die ausgeführte Aktion.
 - **terminated**: Ein boolescher Wahrheitswert, der angibt, ob die Episode durch einen regulären Endzustand (zum Beispiel Ziel erreicht oder „Game Over“) beendet wurde.
 - **truncated**: Ein boolescher Wahrheitswert, der angibt, ob die Episode künstlich von außen abgebrochen wurde (zum Beispiel weil ein Zeitlimit beziehungsweise eine maximale Schrittzahl erreicht wurde).

- **info**: Ein Dictionary für zusätzliche Diagnose- und Metrikinformationen, die für das eigentliche Training irrelevant, aber für die Auswertung nützlich sind.

Durch diese standardisierte Abstraktion ist es möglich, komplexe, eigens in C++ oder anderen performanten Sprachen geschriebene Simulationen über einen Python-Wrapper als `gymnasium.Env` bereitzustellen. Moderne RL-Frameworks wie Stable-Baselines3 [18] erwarten exakt diese Schnittstelle und können so ohne weitere Anpassungen direkt auf der individuellen Umgebung trainieren.

3 Methodik

3.1 Methodik der Weltgenerierung

Dieses Kapitel beschreibt die Architektur der prozeduralen Weltengenerierung. Das System kombiniert deterministische Rauschfunktionen zur Landschaftserzeugung mit regelbasierten Heuristiken zur Gewährleistung der Spielbarkeit.

3.1.1 Methodische Grundlagen

Die Generierung nutzt einen initialen Seed als Basis für alle pseudozufälligen Operationen, um Welten reproduzierbar zu berechnen.

Pseudozufall und Rauscherzeugung

Zur zustandslosen Berechnung von Raumkoordinaten (x, y) transformiert eine Hash-Funktion die Koordinaten, den Welt-Seed und ein Salt in einen normalisierten Zufallswert im Intervall $[0, 1]$.

Für zusammenhängende Strukturen wird ein *Value-Noise*-Verfahren verwendet, dessen theoretische Grundlage bereits vorgestellt wurde (siehe Kapitel 2.1.2). Die Werte der vier Eckpunkte einer Gitterzelle werden ermittelt und bilinear interpoliert. Zur Kantenenglättung wird der Interpolationsparameter $t \in [0, 1]$ zuvor über die bereits eingeführte Smoothstep-Funktion (Gleichung 2.1) transformiert.

Domain Warping und fraktaler Noise

Um sichtbare Rasterstrukturen des quadratischen Gitters zu brechen, verzerrt ein vorgehaltetes Rauschen die Eingangskoordinaten (*Domain Warping*). Zusätzlich werden mehrere Rauschfunktionen unterschiedlicher Frequenzen und Amplituden aufaddiert (fraktaler Noise), um feiner strukturierte Umgebungen zu erzeugen.

Nachbearbeitung und Validierung

Die generierten Chunks können optional mittels zellulärer Automaten nachbearbeitet werden. Die Geburts- und Überlebensregeln der Moore-Nachbarschaft werden dabei über die Konfiguration gesteuert.

Zur topologischen Prüfung kann optional eine BFS eingesetzt werden (siehe Kapitel 2.1.4), die die Erreichbarkeit innerhalb des Chunks sowie valide Ausgänge in einem definierten Distanzring verifiziert. Im ausgelieferten Konfigurationsstand ist diese Prüfung deaktiviert; warum die Deaktivierung die Lösbarkeit unberührt lässt, zeigt Abschnitt 3.1.3.

3.1.2 Heuristiken und Regeln

Biome und Schwellenwerte

Die Welt wird über ein Binning des Rauschergebnisses in festen Intervallen von $\frac{1}{7}$ in sieben Biome unterteilt. Biomspezifische Schwellenwerte steuern das Platzieren von Hindernissen, Erzen und Vegetation.

Gewässer entstehen durch eine gewichtete Linearkombination zweier niederfrequenter Rauschfunktionen:

$$\text{Score} = 0,6 \cdot a + 0,4 \cdot b \quad (3.1)$$

Zellen mit einem Wert über 0,86 werden als Wasser markiert.

Freiräume und Fallback-Pfade

Die eigentliche Lösbarkeitsgarantie entsteht bereits bei der Wahl des Ausgangs: Kandidaten werden per Breitensuche vom Spawnpunkt aus ermittelt, sodass ausschließlich nachweislich erreichbare Positionen als Ausgang infrage kommen. Zusätzlich steht optional ein Fallback-Algorithmus bereit, der bei Bedarf einen direkten Manhattan-Pfad (zuerst X-, dann Y-Achse) in die Karte gräbt; im ausgelieferten Konfigurationsstand ist dieser Fallback deaktiviert (siehe Abschnitt 3.1.3). Feste Radien um Spawn- und Zielpunkte verhindern zusätzlich das Blockieren der Start- und Endpositionen.

Point-of-Interest-Strukturen werden mit einer Wahrscheinlichkeit von $P = 0,10$ pro Chunk platziert. Ihre Form basiert auf biomspezifischen 5×5 ASCII-Matrizen.

3.1.3 Systemkonfiguration

Die früher wegen des festen $\frac{1}{7}$ -Binnings funktional obsoleten Parameter `coldBiomeMax` und `warmBiomeMax` wurden inzwischen vollständig entfernt (Parser, Struct-Felder und JSON-Schlüssel); sie tauchen deshalb auch nicht mehr im Konfigurationsauszug in Anhang A auf. `enableMacroGraphPrecheck` und `macroGraphRadiusChunks` werden dagegen weiterhin eingelesen, aber im Quellcode an keiner Stelle ausgewertet; sie sind damit tote Konfigurationsschlüssel ohne Wirkung.

Der Manhattan-Carver aus Abschnitt 4.1.4 lässt sich über zwei unabhängige Schalter auslösen: `forceGuaranteedPath` ruft ihn unbedingt bei jedem `reset()` auf, während `guaranteedPathFallback` ihn nur als nachgelagerte Korrektur einsetzt, falls der optionale Flood-Fill-Erreichbarkeitstest (`enableFloodFillValidation`) eine unerreichbare Welt meldet. Da Letzterer im ausgelieferten Konfigurationsstand ebenfalls deaktiviert ist, bleibt `guaranteedPathFallback` praktisch wirkungslos. Beide Schalter sind im ausgelieferten Konfigurationsstand deaktiviert.

Der aktuelle Standard-Betriebsmodus laut Konfigurationsdatei:

Die in Tabelle 3.1 aufgeführten Mechanismen sind im ausgelieferten Konfigurationsstand sämtlich deaktiviert. Die Lösbarkeit wird dennoch unbedingt sichergestellt, jedoch nicht durch einen dieser Schalter, sondern durch die Konstruktion der Ausgangsplatzierung selbst: Der Ausgang wird ausschließlich unter Positionen gewählt, die eine Breitensuche

Tabelle 3.1: Konfigurationsstatus der Weltengenerierung

Feature / Algorithmus	Status	Referenz
Zelluläre Automaten (CA)	DEAKTIVIERT	siehe Anhang A
Erreichbarkeitstest (Flood-Fill)	DEAKTIVIERT	siehe Anhang A
Manhattan-Carving-Fallback (<code>forceGuaranteedPath</code>)	DEAKTIVIERT	siehe Anhang A
Manhattan-Carving-Fallback, Flood-Fill- Korrektur (<code>guaranteedPathFallback</code>)	DEAKTIVIERT	siehe Anhang A

vom Spawnpunkt aus als erreichbar bestätigt (siehe Abschnitt 4.1.4). Diese BFS-Exit-Platzierung ist fest im Code verankert und nicht über die Konfiguration abschaltbar.

3.2 Methodik des RL-Trainings

Dieses Kapitel behandelt die methodische Umsetzung der zweiten Forschungsfrage: Wie gut können rekurrente (LSTM) und zustandslose (MLP) PPO-Agenten navigieren, wenn die Umgebung nur teilweise sichtbar ist (partielle Beobachtbarkeit)? Dafür wird im Folgenden das Setup für das Reinforcement Learning (RL) beschrieben, also die Problemformulierung, das Curriculum, die Validierung und der genaue Evaluationsablauf. Die reinen technischen Details zur Implementierung folgen separat in Kapitel 4.2.

3.2.1 Auswahl der zu vergleichenden Trainingsverfahren

Im Mittelpunkt des Vergleichs stehen zwei Varianten von PPO: eine mit rekurrenter LSTM-Policy (im Folgenden LSTM-PPO) und eine mit zustandsloser Multilayer-Perceptron-Policy (MLP-PPO). Da beide Modelle denselben Optimierungsmechanismus nutzen, lässt sich durch diesen Aufbau gut untersuchen, welchen Einfluss das Gedächtnis (Rekurrenz) auf die Navigation in solchen Umgebungen hat.

Frühere Tests mit Deep Q-Networks (DQN) wurden für diese Untersuchung verworfen. Erste Analysen zeigten, dass die gelernten Q-Werte für alle vier Aktionen extrem nah beieinander lagen und sich damit kaum unterschieden. Dadurch konnte der Agent keine klare Richtung lernen und hing oft in endlosen Pendelschleifen zwischen zwei Zellen fest. Ein weiteres Problem war der Replay-Puffer von DQN: Falsch berechnete Rewards aus den ersten Durchläufen blieben darin gespeichert und störten das spätere Training immer wieder. Da PPO als On-Policy-Verfahren solche alten Daten direkt verwirft, treten diese historischen Verzerrungen gar nicht erst auf. Der A2C-Algorithmus wurde theoretisch ebenfalls in Betracht gezogen, wird hier aber nicht praktisch getestet. Um die Leistung der PPO-Agenten stattdessen besser einschätzen zu können, dienen drei ungelernte Baselines (zwei Heuristiken und ein Orakel) als direkter Vergleich.

3.2.2 Problemformulierung: Zustand, Aktionsraum und Reward

Passend zu den theoretischen Grundlagen wird die Navigationsaufgabe als POMDP modelliert (siehe Kapitel 2.2). Die wichtigsten Bausteine dazu sind in Tabelle 3.2 zusammengefasst. Der Agent sieht also nicht die komplette Karte ($s \in S$), sondern bekommt in

jedem Schritt nur einen lokalen Ausschnitt $o \in \Omega$. Das ist konkret ein 15×15 Zellen großes Sichtfeld plus einige Zusatzinformationen, wie die relative Richtung zum Ziel.

Die Größe des Sichtfelds ist dabei ein Zielkonflikt: Ein zu kleiner Ausschnitt lässt den Agenten Hindernisse zu spät erkennen, ein zu großer lässt den Beobachtungsraum unnötig anwachsen und untergräbt zugleich den Charakter der partiellen Beobachtbarkeit, auf den es in dieser Arbeit gerade ankommt. Der konkrete Wert von 15 Feldern liegt bewusst im mittleren Bereich dieses Kompromisses; eine systematische Rastersuche über alternative Radien war im Rahmen dieser Arbeit nicht vorgesehen. Daraus entsteht ein Beobachtungsvektor mit 229 Dimensionen. Als Aktionen gibt es vier diskrete Bewegungsrichtungen. Auf diagonale Schritte wurde verzichtet, da dies im rasterbasierten Labyrinth der Spielumgebung zu Problemen mit den Wänden führt.

Tabelle 3.2: Übersicht der POMDP-Komponenten für das RL-Training

Komponente	Eigenschaften / Struktur
Beobachtungsraum (Ω)	Lokaler 15×15 Grid-Ausschnitt, Trefferpunkte, Zielrichtung, Schrittbudget-Anteil (229 Dimensionen)
Aktionsraum (A)	4 diskrete Aktionen (Auf, Ab, Links, Rechts)
Primärer Reward	Sparse, singulärer hoher Terminal-Reward bei Zielerreichung
Reward Shaping	Potential-Based Reward Shaping (PBRs) basierend auf BFS-Distanzfeld

Die Reward-Funktion wurde bewusst einfach gehalten. Das primäre Signal bleibt „sparse“: Der weitaus größte Ausschlag ist die einzelne, große Belohnung, wenn der Agent den Ausgang erreicht. Auf hohe Schrittstrafen wird verzichtet, damit der Agent nicht durch ständige große Abzüge vom eigentlichen Ziel abgelenkt wird; ein minimaler Zeitabzug von $-0,01$ pro Schritt bleibt dabei bestehen (siehe Abschnitt 4.2.5). Um das Training dennoch etwas zu beschleunigen, kommt das Potential-Based Reward Shaping (PBRs) nach Ng, Harada und Russell zum Einsatz (siehe Kapitel 2.2). Mit der formenden Belohnung $F(s, a, s') = \gamma \Phi(s') - \Phi(s)$ und einem BFS-Distanzfeld als Potenzialfunktion Φ ist die Optimalpolitik-Invarianz nach Ng et al. [15] mathematisch sichergestellt. Diese Garantie gilt jedoch nur für den PBRs-Anteil selbst, nicht für die Gesamtbelohnung: Der tatsächliche Reward enthält mit dem Neuland-Bonus, der Loop-Strafe und der Besuchsstrafe des Python-Wrappers weitere, nicht-potenzialbasierte Terme (siehe Abschnitt 4.2.5), die die optimale Politik durchaus verändern können.

3.2.3 Curriculum Konzept und Schnittstelle zur Weltgenerierung

Damit das Training nicht sofort an der vollen Zieldistanz scheitert, ist der Lernprozess in ein vierphasiges Curriculum unterteilt (siehe Abbildung 3.1). In den Phasen wird die sogenannte Exit-Distanz, also der minimale Laufweg vom Start zum Ausgang, schrittweise erhöht. Das setzt technisch zwingend voraus, dass der Weltgenerator einen Regler bietet, mit dem sich der Ausgang gezielt in einem bestimmten Radius platzieren lässt (siehe Kapitel 3.1). Die Trainingspipeline überschreibt dazu bei jeder Phase die statischen Werte

`exitMinDistance` und `exitMaxDistance` der Konfigurationsdatei (siehe Anhang A) programmatisch mit den jeweiligen Phasengrenzen; der dort hinterlegte Bereich von 35 bis 45 Schritten gilt ausschließlich für den in Abschnitt 3.2.5 beschriebenen Eval-Prozess.

Phase 1 startet mit kurzen Wegen von 5 bis 12 Schritten, die in Phase 2 auf 12 bis 25 und in Phase 3 auf 25 bis 45 Schritte ansteigen. In Phase 4 bleibt die maximale Distanz bei 25 bis 45 Schritten gleich, aber die Explorationsrate des Agenten wird gezielt heruntergefahren. Den vier Phasen kommt dabei jeweils eine eigene Rolle zu: Phase 1 vermittelt das elementare Navigationsprinzip auf kurzen, leicht lösbaren Strecken, Phase 2 überträgt dieses Prinzip auf mittlere Distanzen und prüft damit erstmals die Generalisierungsfähigkeit, Phase 3 stellt die volle Zielverteilung her und beginnt zugleich die Determinisierung der Politik, und Phase 4 dient als reines Greedy Fine-Tuning, das gezielt die Lücke zwischen deterministischem und stochastischem Modus schließen soll.

Sowohl diese Distanzbereiche als auch die phasenspezifischen Ziel-Erfolgsquoten von 85/70/70/60 % (siehe Abschnitt 4.2.4) wurden empirisch so gewählt, dass ein zuverlässiges Meistern von Phase 1 die Voraussetzung für einen vorzeitigen Phasenwechsel bilden sollte; eine systematische Optimierung der exakten Zahlenwerte war im Rahmen dieser Arbeit nicht vorgesehen, wohl aber die hier beschriebene funktionale Aufteilung der Phasen. In der Praxis erreichte dieses Gate jedoch nur einer der sieben LSTM-PPO-Läufe, die übrigen sechs wechselten nach Ausschöpfen des Schrittbudgets in Phase 2, ohne die Zielquote zweimal in Folge erreicht zu haben (siehe Abschnitt 4.2.4). Dadurch soll der Agent aus der anfänglichen Probierphase eine feste, deterministische Strategie entwickeln.

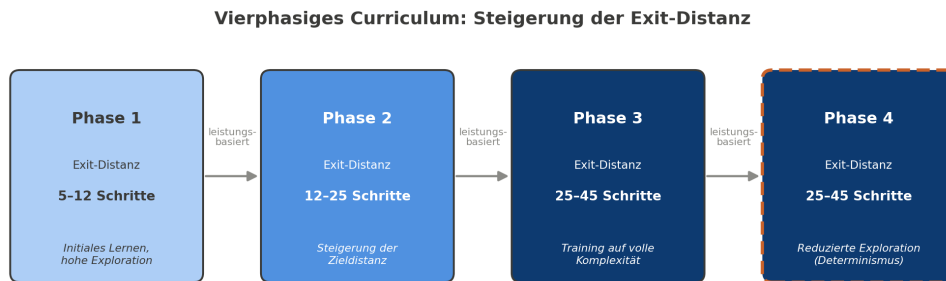


Abbildung 3.1: Struktur des vierphasigen Curriculum-Trainings mit Steigerung der Exit-Distanz, eigene Darstellung.

Wichtig für den Trainingserfolg ist, dass der Wechsel zwischen den Phasen über ein duales Gate abläuft: Die nächste Phase startet entweder beim Erreichen einer bestimmten Erfolgsquote in zwei aufeinanderfolgenden Evaluationen, oder sobald das für die Phase vorgesehene Schrittbudget aufgebraucht ist. Ein rein zeitbasierter Wechsel wird dadurch vermieden, ohne das Training bei ausbleibendem Erfolg unbegrenzt in einer Phase zu belassen.

3.2.4 Definition der Baselines

Um die trainierten Modelle besser einordnen zu können, wurden drei ungelernete Baselines definiert. Sie gehören nicht zum Hauptvergleich zwischen LSTM-PPO und MLP-PPO, sondern dienen als Orientierungshilfe für die Leistung (siehe Abbildung 3.2).

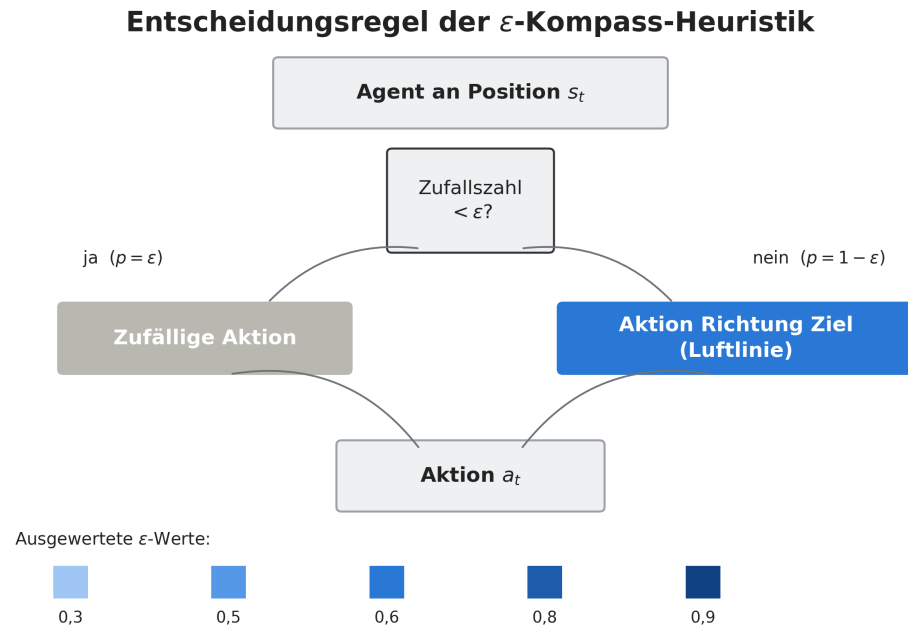


Abbildung 3.2: Entscheidungsregel und ausgewertete Parameter der ε -Kompass-Heuristik, eigene Darstellung.

Das obere Limit bildet das BFS-Optimum. Dieses Orakel nutzt eine Breitensuche, um immer den mathematisch kürzesten Weg zu finden, und zeigt damit das absolute Maximum an Pfadqualität auf. Das untere Limit ist der Random-Agent, der völlig zufällige Aktionen wählt. Dazwischen liegt die ε -Kompass-Heuristik: Sie wählt mit einer Wahrscheinlichkeit von ε einen zufälligen Schritt und bewegt sich ansonsten geradlinig in Richtung Ziel. Diese Heuristik wurde mit verschiedenen Werten getestet ($\varepsilon \in \{0,3; 0,5; 0,6; 0,8; 0,9\}$), um aufzuzeigen, wie Erfolgsquote und Streckenlänge in dieser Umgebung typischerweise zusammenhängen. Die Stützstellen decken dabei einen niedrigen, mittleren und hohen Zufallsanteil ab; sie entstammen keinem systematisch gleichmäßigen Raster, sondern wurden im Projektverlauf schrittweise um zusätzliche Werte ergänzt, ohne dass eine abschließende Rastersuche über das gesamte Intervall durchgeführt wurde.

3.2.5 Trainings- und Evaluationsprotokoll

Um verlässliche und reproduzierbare Ergebnisse zu erhalten, wurde das Vorgehen für das Training und die Evaluation strikt festgelegt. Das Training lief über sieben unabhängige Zufalls-Seeds; ursprünglich war aus Zeitgründen eine engere Grenze von drei Läufen geplant, die sich angesichts des gemessenen Durchsatzes (siehe Abschnitt 4.2.3) als überflüssig erwies, da das LSTM-Training als langsamere der beiden Architekturen sieben Wiederholungen im vorgesehenen Zeitrahmen zuließ. Danach wurden die Modelle auf zwei komplett neuen Umgebungssets getestet, die während des Trainings nicht vorkamen: dem regulären Testset A (Seeds 7000–7049) und einem zusätzlichen Holdout B (Seeds 8000–8049), um die Generalisierungsfähigkeit zu prüfen. Die trainingsbegleitenden Evaluationen, die während des Trainings selbst über das in Abschnitt 4.2.4 beschriebene Gating entscheiden, laufen dagegen auf einem dritten, von beiden Testsets unabhängigen

Seed-Bereich, den sogenannten Validierungs-Seeds (6000–6049). Die sieben resultierenden LSTM-PPO-Modelle werden im weiteren Verlauf unter der Kurzbezeichnung v12 (v12_s1 bis v12_s7) geführt, dem finalen, für diese Arbeit berichts-fähigen Konfigurations- und Trainingsstand.

Jedes der beiden je 50 Seeds umfassenden Testsets wird dabei nicht nur einmal durchlaufen, sondern über fünf unabhängige Wiederholungen mit je eigenem Politik-RNG-Seed ausgewertet, während die Welt-Seeds selbst fix bleiben; die berichteten Prozentwerte sind Mittelwerte über diese fünf Wiederholungen. Die feinere Auflösung mancher Prozentangaben (etwa 65,2 %) erklärt sich dadurch: Jede der fünf Wiederholungen liefert für sich einen Wert im 2-Prozent-Raster von 50 Seeds, der Mittelwert über fünf solcher Werte kann jedoch Vielfache von 0,4 Prozentpunkten annehmen.

Da ein rein zufällig agierender Agent in den seltenen erfolgreichen Fällen im Schnitt 2251,9 Schritte bis zum Ziel benötigt (siehe Abschnitt 5.2.2), war für die Tests ein festes Episodenlimit zwingend erforderlich. Ohne ein solches Limit ließe sich die Erfolgsquote nicht sinnvoll berechnen, da ein schlechter Agent sonst unendlich lange im Labyrinth verweilen würde. Überschreitet ein Agent das Limit, gilt der Versuch strikt als gescheitert. Als zweiter, seltenerer Scheiter-Modus gilt der in Abschnitt 4.2.7 beschriebene Early-Stop nach 256 Schritten ohne positiven Reward: Er ist fest im Umgebungs-Wrapper verankert und greift damit auch in den Testset-A/B-Messungen, nicht nur während des Trainings.

Zusätzlich werden die Modelle zweigleisig betrachtet: einmal deterministisch (der Agent wählt immer die Aktion mit der höchsten Wahrscheinlichkeit) und einmal stochastisch (die Aktion wird anhand der gelernten Verteilung gesampelt). Erst dieser Vergleich zeigt, ob das Netzwerk eine belastbare Richtungsstrategie verstanden hat oder lediglich durch stochastisches Raten ans Ziel gelangt. Die für Testset A und B berichtete Haupteinfolgsquote (siehe Abschnitt 5.2.1) ist dabei durchgehend die stochastische, da sie dem Verhalten der Policy während des eigentlichen Trainings entspricht; das kanonische Eval-Skript deckt ausschließlich diesen Modus ab. Eine ergänzende deterministische Auswertung liegt als separate Zusatzmessung auf denselben Testset-Seeds nur für das LSTM-PPO-Modell vor; für das MLP-PPO-Modell wird ausschließlich die stochastische Erfolgsquote berichtet (siehe Abschnitt 5.2.1).

Bei den Metriken ist die Erfolgsquote (Success Rate, SR) das wichtigste Kriterium, gefolgt von der Pfadqualität. Der Grund dafür liegt darin, dass in ähnlichen POMDP-Benchmarks (wie Procgen, MiniGrid oder POPGym) die Erfolgsquote durchgehend als Standard gilt. Die grundsätzliche Fähigkeit, das Problem überhaupt zu lösen, wird in diesem Kontext höher gewichtet als eine perfekte Routenplanung.

3.2.6 Validierungsstrategie und Umgang mit Trainingsinstabilitäten

Um einen stabilen Trainingsverlauf zu gewährleisten, musste der Prozess durchgehend überwacht und angepasst werden. Da Reinforcement Learning in solchen Umgebungen häufig dazu neigt, sich festzufahren oder den Lernprozess einzustellen, wurde gezielt auf spezifische Fehlerbilder geachtet.

Erstens half das leistungs- und budgetbasierte Curriculum dabei, unkontrollierte Abstürze zu verhindern. Bei einem rein zeitbasierten Wechsel der Phasen besteht die Gefahr des sogenannten *Catastrophic Forgetting*: Der Agent verlernt beim Sprung in die nächste Phase plötzlich zuvor erlernte Fähigkeiten, und die Erfolgsquote stürzt dauerhaft auf nahezu 0 % ab.

Zweitens mussten die Parameter für die rekurrenten Netzwerke sehr genau justiert werden. Vor allem der Entropie-Koeffizient musste sorgfältig gewählt werden, damit der Agent lange genug neue Wege ausprobierte. Gleichzeitig durfte die Schrittzahl pro Update für das LSTM nicht zu hoch angesetzt werden, um das typische Problem der verschwindenden Gradienten (Vanishing Gradient) zu umgehen. Ohne diese Anpassungen bleibt das Training oft komplett stehen, oder der Agent zeigt einen *Mode Collapse*: Er dreht sich beispielsweise nur noch auf der Stelle.

Drittens erwies sich die Puffergröße für das Reward Shaping als kritischer Faktor. Der Distanzfeld-Puffer, ein räumlicher Rand um die Bounding-Box aus Spawn und Exit, in dem die Breitensuche noch valide Distanzwerte liefert (siehe Abschnitt 4.2.5), musste groß genug dimensioniert werden, damit der Agent nicht fälschlicherweise für Rückwärtsschritte belohnt wird. Ein zu klein gewählter Puffer führte häufig zum *Reward Hacking*: Der Agent pendelte nur noch zwischen zwei Feldern hin und her, um dauerhaft kleine Belohnungen zu sammeln, anstatt den Ausgang anzusteuern.

Um diese Risiken systematisch im Blick zu behalten, wurde der Trainingsprozess anhand der Indikatoren aus Tabelle 3.3 überwacht und entsprechend korrigiert.

Tabelle 3.3: Diagnose- und Interventionsmatrix der aufgetretenen Trainingsinstabilitäten

Symptom / Fehlerbild	Indikator in den Metriken	Angewandte Interventionsstrategie
Catastrophic Forgetting (Abrupter Leistungsabfall)	Erfolgsquote stürzt beim Phasenwechsel drastisch ab und erholt sich nicht.	Duales Gate eingeführt: Phasenwechsel erst bei phasenspezifischer Ziel-Erfolgsquote (85/70/70/60 %) in zwei aufeinanderfolgenden Evaluationen oder bei Erschöpfung des Schrittbudgets.
Lernstillstand / Mode Collapse (Zwanghaftes Kreiseln)	Aktions-Entropie fällt schlagartig ab; Agent erreicht stets das Episodenlimit.	Entropie-Koeffizienten erhöht; Schrittzahl pro Update für das LSTM limitiert und angepasst.
Reward Hacking (Pendeln zwischen Zellen)	Steigender Episoden-Reward bei stagnierender oder sinkender Erfolgsquote.	Distanzfeld-Puffer deutlich vergrößert; Potenzialfunktion auf mathematische Korrektheit geprüft.

4 Umsetzung und technische Realisierung

Dieses Kapitel beschreibt die praktische Implementierung der in Kapitel 3 vorgestellten Konzepte, zunächst der Weltgenerierung (Abschnitt 4.1.2 bis 4.1.8), anschließend des RL-Trainings (Abschnitt 4.2). Die Weltgenerierung sowie die zugehörige Client-Anwendung sind in der Programmiersprache C++ realisiert, für die grafische Darstellung des Clients kommt die Bibliothek Raylib zum Einsatz. Das Hauptziel der Weltgenerierung besteht darin, eine performante, deterministische und speichereffiziente Erzeugung von Welt-Chunks zur Laufzeit zu gewährleisten.

Dazu werden nachfolgend die zentralen Softwarebausteine (von der mathematischen Rauscherzeugung über die biomspezifische Zuweisung bis hin zu den topologischen Sicherheits- und Pfad-Algorithmen) anhand relevanter Quellcode-Ausschnitte und deren Datenstrukturen detailliert erläutert.

4.1 Implementierung der Weltgenerierung

4.1.1 Technologie-Stack

Weltgenerierung, Simulationskern und Client sind vollständig in C++ umgesetzt. Tabelle 4.1 gibt einen kurzen Überblick über die verwendeten Werkzeuge.

Tabelle 4.1: Eingesetzte Werkzeuge der Weltgenerierung und des Clients

Werkzeug	Rolle im System	Variante
C++	Implementierungssprache von Kern, Headless-Runner und Client	Standard 20
CMake	Build-System für alle Komponenten und Bindungen	≥ 3.20
Raylib	Grafik- und Eingabebibliothek des spielbaren Clients	Version 5.5
nlohmann/json	Einlesen der Konfigurationsdatei zur Laufzeit	Version 3.11.3
pybind11	Brücke zwischen C++-Simulationskern und Python (siehe Abschnitt 4.2.1)	Version 2.13.6

Der Simulationskern (`stoneforge_core`) ist als eigenständige Bibliothek angelegt und wird von drei getrennten Zielen eingebunden: dem spielbaren, Raylib-basierten Client, einem Headless-Runner für Benchmarks (siehe Abschnitt 5.1.5) und dem über pybind11 erzeugten Python-Modul für das RL-Training. Alle drei greifen damit auf identischen Quellcode zurück, wodurch Divergenzen zwischen der im Client gespielten und der im

Training verwendeten Weltlogik ausgeschlossen sind. Ein früherer SDL2-Client existiert im Quellcode als deaktivierte Rückfalloption, wird jedoch nicht mehr gepflegt.

4.1.2 Implementierung der Rauscherzeugung und des Koordinaten-Hashings

Das Fundament der deterministischen Landschaftserzeugung bildet ein zweistufiger Prozess: Zunächst werden diskrete Raumkoordinaten deterministisch in Pseudozufallswerte transformiert, welche anschließend durch ein bilineares Value-Noise-Verfahren kontinuierlich interpoliert werden (siehe Kapitel 3.1.1).

Deterministischer Hash-Mixer

Um ohne aufwändige Speicherzustände für beliebige Weltkoordinaten (x, y) reproduzierbare Zufallswerte zu erhalten, nutzt das System einen 64-Bit-Hash-Mixer. Listing 4.1 zeigt die konkrete Umsetzung der Abbildung von Raumkoordinaten auf das normierte Intervall $[0, 1]$.

```

1  std::uint64_t World::mix(std::uint64_t value) const {
2      value ^= value >> 30;
3      value *= 0xbf58476d1ce4e5b9ULL;
4      value ^= value >> 27;
5      value *= 0x94d049bb133111ebULL;
6      value ^= value >> 31;
7      return value;
8  }
9
10 double World::noise01(int worldX, int worldY, std::uint64_t salt)
11     const {
12     std::uint64_t value = static_cast<std::uint64_t>(worldX) * 0
13         x9e3779b185ebca87ULL;
14     value ^= static_cast<std::uint64_t>(worldY) * 0
15         xc2b2ae3d27d4eb4fULL;
16     value ^= seed_ * 0x165667b19e3779f9ULL;
17     value ^= salt;
18     value = mix(value);
19
20     constexpr double kScale = 1.0 / static_cast<double>(0
21         xFFFFFFFFFFFFFFFFULL);
22     return static_cast<double>(value & 0xFFFFFFFFFFFFFFFFULL) * kScale;
23 }
24
25 auto sampleValue = [&](double x, double y, std::uint64_t salt) {
26     const int x0 = static_cast<int>(std::floor(x));
27     const int y0 = static_cast<int>(std::floor(y));
28     const int x1 = x0 + 1;
29     const int y1 = y0 + 1;
30     const double tx = smoothstep(x - static_cast<double>(x0));
31     const double ty = smoothstep(y - static_cast<double>(y0));

```

```

28
29     const double v00 = noise01(x0, y0, salt);
30     const double v10 = noise01(x1, y0, salt);
31     const double v01 = noise01(x0, y1, salt);
32     const double v11 = noise01(x1, y1, salt);
33     return lerp(lerp(v00, v10, tx), lerp(v01, v11, tx), ty);
34 };

```

Quelltext 4.1: Verwendung des 64-Bit SplitMix-Mixers und Value-Noise-Interpolation in `world.cpp`

Variablen und Datenstrukturen

Die in Listing 4.1 gezeigten Funktionen bilden die mathematische Basis für die rasterfreie Weltengenerierung:

- **mix:** Eine Implementierung der 64-Bit-SplitMix-Pseudozufallsfunktion. Sie nutzt Bit-Shifting und Multiplikationen mit großen Primzahl-Konstanten, um Bit-Korrelationen zu zerstören und eine gleichmäßige Verteilung zu garantieren.
- **worldX, worldY:** Die diskreten Ganzzahlkoordinaten des abzufragenden Raumpunktes auf dem Weltgitter.
- **seed_:** Die Instanzvariable des globalen Welt-Seeds. Sie stellt sicher, dass dieselben Raumkoordinaten bei identischem Seed exakt reproduzierbare Ergebnisse liefern.
- **salt:** Ein variabler 64-Bit-Wert, der als Salt dient. Er verhindert, dass unterschiedliche Generierungsebenen (zum Beispiel Dichte, Erz- und Baumpopulation) auf derselben Koordinate synchrone Muster aufweisen.
- **kScale:** Ein präziser Skalierungsfaktor ($\frac{1}{2^{48}-1}$), der die untersten 48 Bit des Bit-musters über eine Bitmaskierung (`0xFFFFFFFFFULL`) exakt auf das normierte Intervall $[0, 1]$ abbildet.
- **x0, y0, x1, y1:** Ganzzahlige Eckpunkte der Gitterzelle, in der der kontinuierliche Punkt (x, y) liegt.
- **tx, ty:** Die mittels `smoothstep` transformierten relativen Abstände innerhalb der Zelle, welche als Interpolationsgewichte dienen.
- **v00, v10, v01, v11:** Die an den vier Gitterecken über `noise01` determinierten Zufallswerte, welche abschließend mittels zweifacher linearer Interpolation (`lerp`) verschmolzen werden.

4.1.3 Biomeinteilung und Objekt-Placement

Nach der Bereitstellung der kontinuierlichen Rauschfunktionen erfolgt die Zuweisung konkreter Kacheltypen (`TileType`) zu den Raumkoordinaten (x, y) . Die Funktion `sampleBaseTile` steuert diesen Prozess in Abhängigkeit vom übergeordneten Chunk-Biom und biomspezifischen Schwellenwerten (*Thresholds*).

Implementierung der Kachelauswertung

In Listing 4.2 ist die Realisierung der `sampleBaseTile`-Methode dargestellt. Sie bestimmt zunächst über das Chunk-Raster (`floorDiv`) das zugrundeliegende Biom (`biomeTag`) und wertet anschließend drei unabhängige Noise-Layer (Dichte, Erzvorkommen, Baumdichte) aus.

```

1 TileType World::sampleBaseTile(int worldX, int worldY, const
   WorldGenConfig& cfg) const {
2     const int cx = floorDiv(worldX, kChunkSize);
3     const int cy = floorDiv(worldY, kChunkSize);
4     const int biomeTag = biomeTagForChunk(cx, cy);
5
6     const double density = noise01(worldX, worldY, cfg.densitySalt)
       ;
7     const double ore = noise01(worldX, worldY, cfg.oreSalt);
8     const double trees = noise01(worldX, worldY, cfg.treeSalt);
9
10    double wallThreshold = 0.11;
11    double oreThreshold = 0.03;
12    double treeThreshold = 0.08;
13
14    // Biomspezifische Schwellenwert-Profil
15    switch(biomeTag) {
16        case 0: // Grasland: WENIGER BAEUME
17            wallThreshold = 0.10; oreThreshold = 0.03;
18            treeThreshold = 0.05; break;
19        case 1: // Wald: VIELE BAEUME, WENIG STEINE
20            wallThreshold = 0.07; oreThreshold = 0.015;
21            treeThreshold = 0.23; break;
22        case 2: // Wueste: VIELE STEINE, KEINE BAEUME
23            wallThreshold = 0.19; oreThreshold = 0.08;
24            treeThreshold = 0.0; break;
25        case 3: // Bergland: VIELE STEINE UND ERZE
26            wallThreshold = 0.25; oreThreshold = 0.10;
27            treeThreshold = 0.02; break;
28        case 4: // Steppe: GRASLAND MIT ETWAS MEHR BAEUMEN
29            wallThreshold = 0.11; oreThreshold = 0.03;
30            treeThreshold = 0.09; break;
31        case 5: // Tundra: NORMALE STEINE UND BAEUME
32            wallThreshold = 0.14; oreThreshold = 0.05;
33            treeThreshold = 0.07; break;
34        case 6: // Hoelle: HOHE WAND- UND ERZDICHT
35            wallThreshold = 0.23; oreThreshold = 0.09;
36            treeThreshold = 0.06; break;
37        default: break;
38    }
39
40    TileType tile = TileType::Empty;

```

```

34
35 // Gewaessermaske besitzt hoechste Prioritaet (Freiraum)
36 if(lakeMaskAt(worldX, worldY)) {
37     return TileType::Empty;
38 }
39
40 // Auswertung der Objektgrenzwerte
41 if(density < wallThreshold) {
42     tile = TileType::Wall;
43 }
44 // Erzvorkommen exklusiv im Bergland (Biom 3)
45 if(biomeTag == 3 && ore < oreThreshold) {
46     tile = TileType::Resource;
47 }
48 // Baumpopulation auf verbleibenden Freiflaechen
49 if(treeThreshold > 0.0 && tile == TileType::Empty && trees <
50     treeThreshold) {
51     tile = TileType::Tree;
52 }
53 return tile;
54 }

```

Quelltext 4.2: Biom-Tagging und Schwellenwertauswertung in world.cpp

Logik und Priorisierung der Kachelzuweisung

Die Auswertung in Listing 4.2 folgt einer strikten Prioritätenhierarchie, um ungewollte Überschreibungen von Spielelementen zu vermeiden:

1. **Chunk-Rasterung (floorDiv):** Raumkoordinaten werden zunächst in diskrete Chunk-Koordinaten (cx, cy) transformiert, um das für den gesamten Chunk gültige Biom ($\text{biomeTag} \in [0, 6]$) abzufragen.
2. **Drei unabhängige Noise-Layer:** Durch die Übergabe unterschiedlicher Salt-Werte (cfg.densitySalt , cfg.oreSalt , cfg.treeSalt) an die Funktion `noise01` werden räumlich entkoppelte Zufallswerte für Wand-, Erz- und Baumvorkommen generiert.
3. **Biomspezifische Profile (switch):** Das ermittelte `biomeTag` überschreibt die Standard-Schwellenwerte für Wand-, Erz- und Baumdichte. So erzwingt Biom 2 (Wüste) beispielsweise eine Baumgrenze von `treeThreshold = 0.0` (kein Baumbewuchs), während Biom 1 (Wald) mit `treeThreshold = 0.23` eine hohe Baumdichte aufweist.
4. **Gewässermaske (lakeMaskAt):** Seen fungieren als hindernisfreie Areale. Gibt `lakeMaskAt` den Wert `true` zurück, bricht die Funktion sofort ab und gibt `TileType::Empty` zurück.
5. **Hierarchische Klassifizierung:**
 - Unterschreitet die Dichte den Schwellenwert `wallThreshold`, wird das Tile primär als Wandoberfläche (`TileType::Wall`) definiert.

- Erze (`TileType::Resource`) werden exklusiv im Bergland (`biomeTag == 3`) platziert, sofern der Erz-Noise-Wert unter `oreThreshold` liegt.
- Bäume (`TileType::Tree`) sprießen ausschließlich auf bisher unbesetzten Feldern (`tile == TileType::Empty`) unter Berücksichtigung von `treeThreshold`.

4.1.4 Topologische Sicherheits-Fallbacks und Spawn-Freiräume

Um unabhängig von den pseudozufälligen Rauschwerten und den gewählten Biomschwellenwerten eine unbedingte Spielbarkeit (*Playability*) zu garantieren, implementiert das System mehrere deterministische Sicherheits-Fallbacks. Die primäre Garantie entsteht dabei, wie in Abschnitt 3.1.2 beschrieben, bereits vorgelagert bei der BFS-basierten Wahl des Ausgangs; die im Folgenden beschriebenen Mechanismen sichern zusätzlich die unmittelbare Bewegungsfreiheit um Spawn- und Zielpunkt und stehen als abschaltbarer Fallback für den Pfad selbst bereit.

Implementierung des Manhattan-Carvers und der Freiradien

In Listing 4.3 ist der relevante Ausschnitt aus der Methodenreihenfolge in `reset()` sowie die vollständige Implementierung der Methode `carveGuaranteedPath` dargestellt, deren methodischer Entwurf bereits vorgestellt wurde (siehe Kapitel 3.1.2). Im ausgelieferten Konfigurationsstand ist dieser Fallback deaktiviert; der Carver dient hier der Vollständigkeit der Implementierungsbeschreibung.

```

1 // Ausschnitt aus World::reset()
2 for(int dy = -cfg.spawnClearRadius; dy <= cfg.spawnClearRadius; ++
  dy) {
3     for(int dx = -cfg.spawnClearRadius; dx <= cfg.spawnClearRadius;
      ++dx) {
4         setTile(spawn_.x + dx, spawn_.y + dy, TileType::Empty);
5     }
6 }
7
8 if(cfg.forceGuaranteedPath) {
9     carveGuaranteedPath();
10 }
11
12 for(int dy = -cfg.exitClearRadius; dy <= cfg.exitClearRadius; ++dy)
13 {
14     for(int dx = -cfg.exitClearRadius; dx <= cfg.exitClearRadius;
      ++dx) {
15         setTile(exit_.x + dx, exit_.y + dy, TileType::Empty);
16     }
17 }
18 setTile(exit_.x, exit_.y, TileType::Exit);
19
20 // Deterministische Manhattan-Pfadgenerierung
21 void World::carveGuaranteedPath() {

```

```

21  const auto& cfg = gameConfig().world;
22  int x = spawn_.x;
23  int y = spawn_.y;
24
25  setTile(x, y, TileType::Empty);
26
27  // 1. Horizontale Trassierung (X-Achse)
28  while(x != exit_.x) {
29      x += (x < exit_.x) ? 1 : -1;
30      setTile(x, y, TileType::Empty);
31  }
32
33  // 2. Vertikale Trassierung (Y-Achse)
34  while(y != exit_.y) {
35      y += (y < exit_.y) ? 1 : -1;
36      setTile(x, y, TileType::Empty);
37  }
38
39  // Erneute Durchsetzung der Freiradien nach dem Carving
40  for(int dy = -cfg.spawnClearRadius; dy <= cfg.spawnClearRadius;
41      ++dy) {
42      for(int dx = -cfg.spawnClearRadius; dx <= cfg.
43          spawnClearRadius; ++dx) {
44          setTile(spawn_.x + dx, spawn_.y + dy, TileType::Empty);
45      }
46  }
47
48  for(int dy = -cfg.exitClearRadius; dy <= cfg.exitClearRadius;
49      ++dy) {
50      for(int dx = -cfg.exitClearRadius; dx <= cfg.
51          exitClearRadius; ++dx) {
52          setTile(exit_.x + dx, exit_.y + dy, TileType::Empty);
53      }
54  }
55
56  setTile(exit_.x, exit_.y, TileType::Exit);
57  }

```

Quelltext 4.3: Freiradien-Erzeugung und Manhattan-Path-Carver in world.cpp

Funktionsweise und Ausführungsreihenfolge

Die in Listing 4.3 gezeigte Pfadsicherung folgt einem klaren, mehrstufigen Ablauf:

1. **Freiradien-Erzeugung (Quadratischer Bereich):** Sowohl um die Spawn-Koordinate `spawn_` als auch um die Exit-Koordinate `exit_` wird über verschachtelte Schleifen ein quadratischer Radius der Größe `cfg.spawnClearRadius` beziehungsweise

`cfg.exitClearRadius` vollständig freigeräumt (`TileType::Empty`). Dies stellt sicher, dass der Spieler ohne Kollisionsprobleme starten kann.

2. **Schalterbasierte Pfadfreischaltung (`forceGuaranteedPath`):** Die Pfadgenerierung wird rein konfigurationsbasiert über das Flag `cfg.forceGuaranteedPath` gesteuert, das im ausgelieferten Konfigurationsstand deaktiviert ist (siehe Abschnitt 3.1.3). Der Carver dient damit als abschaltbarer Fallback, nicht als primärer Lösbarkeitsmechanismus.
3. **Orthogonale Manhattan-Trassierung:** Die Funktion `carveGuaranteedPath` startet am Punkt `spawn_` und schlägt schrittweise einen absolut freien Korridor in die Umgebung. Die Abfolge ist fest auf die Orthogonalen festgelegt:
 - Zunächst wird der x -Wert der aktuellen Position schrittweise an `exit_.x` angeglichen und dabei jede betroffene Zelle auf `TileType::Empty` gesetzt.
 - Anschließend erfolgt die vertikale Anpassung der y -Position, bis die Koordinaten von `exit_` erreicht sind.
4. **Redundante Sicherung des Exits:** Nach dem Durchschneiden des Pfades werden die Freiradien um den Spawn- und Exit-Bereich erneut freigeräumt und das Zielfeld explizit mit `TileType::Exit` markiert, damit der Level-Ausgang nicht versehentlich überschrieben wird.

4.1.5 Generierung von Gewässern und dynamische Strand-Derivation

Die Erzeugung von Seen und deren optischer Übergangszone (Strände) erfordert eine Entkopplung der logischen Weltrepräsentation von der visuellen Darstellung. Während Gewässer als harte Kollisions- beziehungsweise Freiräume im Weltmodell definiert sind, entstehen Strände dynamisch über Nachbarschaftsauswertungen im Render-Pfad.

Mathematische Seen-Maskierung (`lakeMaskAt`)

Gewässer dürfen im Gegensatz zu einzelnen Bäumen oder Hindernissen nicht als vereinzelte Pixel-Fehler erscheinen, sondern müssen als zusammenhängende, organische Binnengewässer im Terrain hervortreten. Dazu verwendet die Funktion `lakeMaskAt` niederfrequentes Rauschen (*Low-Frequency Noise*).

```

1  bool World::lakeMaskAt(int worldX, int worldY) const {
2      const auto& cfg = gameConfig().world;
3      if (!cfg.enableLakes) {
4          return false;
5      }
6
7      // Niederfrequente Skalierung für flächige Strukturen
8      const double n1 = sampleValueNoise(
9          static_cast<double>(worldX) * 0.035,
10         static_cast<double>(worldY) * 0.035,
11         cfg.lakeSalt
12     );

```

```

13     const double n2 = sampleValueNoise(
14         static_cast<double>(worldX) * 0.018 + 100.0,
15         static_cast<double>(worldY) * 0.018 - 50.0,
16         cfg.lakeSalt + 100
17     );
18
19     // Mischen beider Rauschschichten
20     const double lakeScore = n1 * 0.6 + n2 * 0.4;
21
22     // Schwellenwertentscheidung fuer seltene, koharente Seen
23     return lakeScore > 0.86;
24 }
25
26 bool World::isLakeAt(int worldX, int worldY) const {
27     // Spawn- und Exit-Zonen dürfen niemals von Seen überdeckt
28     werden
29     if ((worldX == spawn_.x && worldY == spawn_.y) ||
30         (worldX == exit_.x && worldY == exit_.y)) {
31         return false;
32     }
33     return lakeMaskAt(worldX, worldY);
34 }

```

Quelltext 4.4: Low-Frequency-Value-Noise für zusammenhängende Gewässermasken in world.cpp

Wie in Listing 4.4 gezeigt, nutzt die Berechnung zwei transformierte Noise-Funktionen mit kleinen Frequenzfaktoren (0,035 und 0,018). Durch die gewichtete Kombination beider Schichten (lakeScore) und die Wahl eines hohen Schwellenwerts ($> 0,86$) entstehen großflächige, organisch geformte Seenstrukturen („blobs“). Die Hilfsfunktion isLakeAt stellt zudem sicher, dass kritische Positionen wie der Spawn- und der Exit-Punkt niemals durch Gewässer blockiert werden.

Prozedurale Strand-Derivation im Renderer

Anstatt Sand/Strand als eigenständige Kacheltypen (TileType) im Speicher zu verwalten, wird die Strandzone zur Laufzeit im Render-Pfad des Clients anhand einer 3×3 -Nachbarschaftsanalyse abgeleitet.

```

1 bool RenderEngine::lakeOrAdjacentAt(int worldX, int worldY) const {
2     // Überprüfung der 3x3-Nachbarschaft auf angrenzende Seen
3     for (int dy = -1; dy <= 1; ++dy) {
4         for (int dx = -1; dx <= 1; ++dx) {
5             if (world_.isLakeAt(worldX + dx, worldY + dy)) {
6                 return true;
7             }
8         }
9     }
10    return false;

```

```

11 }
12
13 // Auszug aus der Tile-Rendering-Schleife:
14 if (world_.isLakeAt(x, y)) {
15     drawWaterTile(x, y);
16 } else if (lakeOrAdjacentAt(x, y) && isPassable(x, y)) {
17     drawSandSpriteOverlay(x, y); // Strand-Overlay für See-Nachbarn
18     drawBaseTile(x, y);
19 } else {
20     drawBaseTile(x, y);
21 }

```

Quelltext 4.5: Nachbarschaftsbasierte Strandardstellung im Render-Pfad in `render_engine.cpp`

- **Effizienz durch Zustandslosigkeit:** Durch die Trennung von Weltlogik und Darstellung spart die Weltengenerierung Speicherplatz, da keine eigenen Sand-Kacheln in der Chunk-Datenstruktur abgelegt werden müssen.
- **Konditionale Überlagerung:** Die Funktion `lakeOrAdjacentAt` prüft, ob sich im direkten 3×3 -Umfeld ein Gewässer befindet. Ist dies der Fall und handelt es sich nicht um eine blockierte Kachel (zum Beispiel Wand oder Baum), rendert die Engine dynamisch ein Sand-Overlay unter dem Basis-Boden.

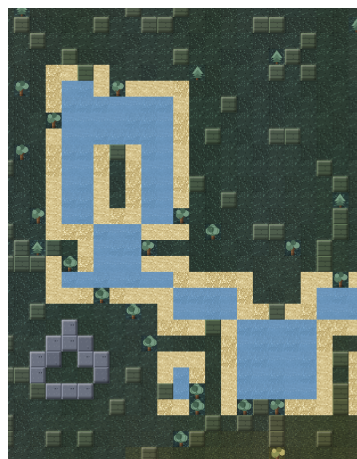


Abbildung 4.1: Prozedural generierte Seenstruktur mit organischer Uferlinie und Strand-Overlay, eigene Darstellung (Screenshot des Prototyps).

4.1.6 Chunk-Speicherarchitektur und prozedurales Biome-Blending

Um speicherhungrige Vorberechnungen großer Welten zu vermeiden, nutzt das System eine bedarfsgesteuerte Chunk-Verwaltung (*Lazy Generation*) gekoppelt mit einer räumlich kontinuierlichen Biom-Farb- und Textursynthese im Render-Pfad.

Bedarfsgesteuerte Chunk-Datenstruktur (Lazy Generation)

Die Weltkarte ist in quadratische Chunks der festen Kantenlänge $kChunkSize = 8$ (insgesamt 64 Tiles pro Chunk) unterteilt. Anstatt gesamte Welten beim Start im RAM abzuliegen, erzeugt die Engine Chunks erst bei der ersten konkreten Zelle- oder Adressabfrage.

```

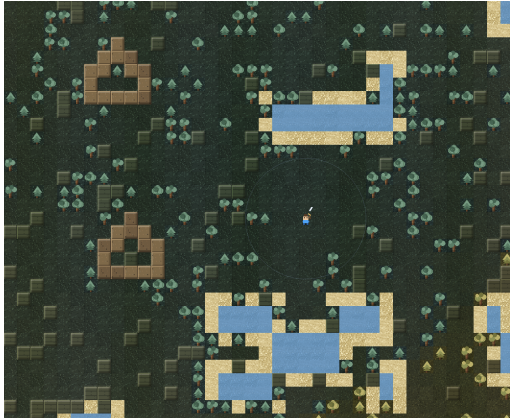
1 // Definition der kompakten Chunk-Struktur (world.hpp)
2 constexpr int kChunkSize = 8;
3
4 struct Chunk {
5     int cx = 0;
6     int cy = 0;
7     bool generated = false;
8     std::array<TileType, kChunkSize * kChunkSize> tiles;
9 };
10
11 // Effiziente Hash-Schlüssel-Generierung für Chunk-Koordinaten
12 inline uint64_t chunkKey(int cx, int cy) {
13     return (static_cast<uint64_t>(cx) & 0xFFFFFFFFFULL) |
14           ((static_cast<uint64_t>(cy) & 0xFFFFFFFFFULL) << 32);
15 }
16
17 // Bedarfsgesteuertes Erzeugen von Chunks (world.cpp)
18 Chunk& World::ensureChunk(int cx, int cy) {
19     const uint64_t key = chunkKey(cx, cy);
20     auto it = chunks_.find(key);
21     if (it != chunks_.end()) {
22         return it->second;
23     }
24
25     // Chunk neu anlegen und initialisieren
26     Chunk& chunk = chunks_[key];
27     chunk.cx = cx;
28     chunk.cy = cy;
29     generateChunkTiles(chunk);
30     chunk.generated = true;
31     return chunk;
32 }

```

Quelltext 4.6: Chunk-Speicherstruktur und Lazy-Loading-Mechanismus in `world.hpp` und `world.cpp`

Wie in Listing 4.6 zu sehen ist, speichert die Klasse `World` alle aktiven Abschnitte in einer `std::unordered_map<uint64_t, Chunk>`. Der 64-Bit-Schlüssel wird bitweise aus den 32-Bit-Signed-Chunk-Koordinaten (cx, cy) zusammengesetzt. Wird über `tileAt()` oder `setTile()` auf ungeneriertes Terrain zugegriffen, erzeugt `ensureChunk()` den Ziel-Chunk instantan und kapselt die Generierungslogik. Ein Aufruf von `reset()` leert die Map mittels `chunks_.clear()`, um den Speicher vollständig freizugeben.

Abbildung 4.2 vergleicht vier der sieben implementierten Biome anhand von Screenshots des Prototyps. Die unterschiedlichen Untergrund-Farbpaletten sowie Objekt- und Hindernisdichten ergeben sich aus den biomspezifischen Schwellenwerten (`wallThreshold`, `oreThreshold`, `treeThreshold`) aus Abschnitt 4.1.3.



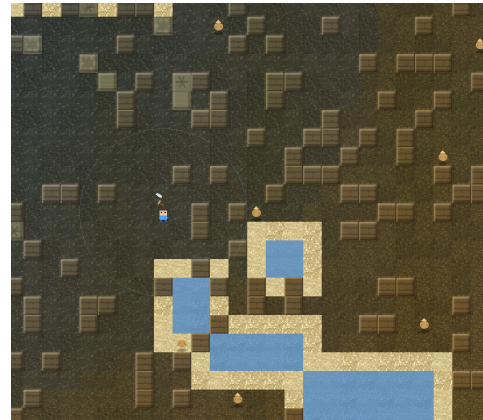
(a) Wald-Biom: hohe Baumdichte, geringe Wandhäufigkeit.



(b) Steppen-Biom: grasland-ähnlich mit leicht erhöhter Baumverteilung.



(c) Bergland-Biom: hohe Wand- und Erzdichte.



(d) Wüsten-Biom: baumfrei, erhöhte Steindichte.

Abbildung 4.2: Vergleich von vier der sieben Biome, eigene Darstellung (Screenshots des Prototyps).

Visuelle Biome-Transformation und weiche Übergänge

Obwohl das logische Biom (`biomeTag`) fest auf Chunk-Ebene definiert ist, vermeidet der Renderer harte, unästhetische Kanten zwischen unterschiedlichen Landschaftstypen. Die visuelle Aufbereitung im Render-Pfad des Clients erfolgt über zwei Stufen: Texturauswahl und prozedurales Farbmischen (*Tinting*).

```

1 // Auswertung gewichteter Biom-Einflüsse im Umfeld
2 BiomeWeights RenderEngine::biomeWeights(int worldX, int worldY)
3     const {
4         BiomeWeights weights{};
5         // Bilineares/Distanzbasiertes Abtasten der umliegenden Chunk-
           Biome
6         for (int dy = -1; dy <= 1; ++dy) {

```

```

6         for (int dx = -1; dx <= 1; ++dx) {
7             int cx = floorDiv(worldX + dx * kChunkSize, kChunkSize)
8                 ;
9             int cy = floorDiv(worldY + dy * kChunkSize, kChunkSize)
10                ;
11             int tag = world_.biomeTagForChunk(cx, cy);
12             float weight = calculateDistanceWeight(worldX, worldY,
13                 cx, cy);
14             weights.add(tag, weight);
15         }
16     }
17     return weights.normalize();
18 }
19
20 // Prozedurale Einfärbung (Tinting) für weiche Übergänge
21 Color RenderEngine::biomeTintAtTile(int worldX, int worldY) const {
22     BiomeWeights weights = biomeWeights(worldX, worldY);
23     Color finalColor = {0, 0, 0, 255};
24
25     for (const auto& [biomeTag, weight] : weights.entries) {
26         Color biomeBaseColor = kRuntimeBiomes[biomeTag].
27             paletteColor;
28         finalColor.r += static_cast<unsigned char>(biomeBaseColor.r
29             * weight);
30         finalColor.g += static_cast<unsigned char>(biomeBaseColor.g
31             * weight);
32         finalColor.b += static_cast<unsigned char>(biomeBaseColor.b
33             * weight);
34     }
35     return finalColor;
36 }

```

Quelltext 4.7: Gewichtete Biom-Mischung und Tinting im Render-Pfad (render_engine.cpp)

- **Textursynthese (dominantBiomes):** Um festzulegen, welches Haupt- und Neben-Sprite (Boden- und Wandstrukturen aus RuntimeBiome) gezeichnet werden, ermittelt dominantBiomes die beiden Biome mit den höchsten Gewichten im umliegenden Raster.
- **Gewichtete Einfärbung (biomeTintAtTile):** Für jede Kachel berechnet biomeWeights anhand der Distanz zu benachbarten Chunks kontinuierliche Gewichtungsfaktoren. Die Funktion biomeTintAtTile nutzt diese Faktoren, um die charakteristischen Farbwerte der Biom-Palette linear zu interpolieren. Das Ergebnis ist ein stufenloser, geschmeidiger Farbverlauf auf der Landoberfläche.

4.1.7 Prozedurale Struktur-Platzierung und dynamisches Gegner-Spawning

Die Besiedlung der prozeduralen Spielwelt teilt sich architektonisch in zwei Subsysteme: statische, biomabhängige Geländestrukturen (Landmarken), die direkt in das Raster der

Chunks integriert werden, sowie ein dynamisches, entitätsbasiertes Gegner-Spawnsystem (*Lazy Mob Spawning*), das über die Simulationsschleife gesteuert wird.

Statische Geländestrukturen in der Chunk-Generierung

Strukturen wie Ruinen, Felsformationen oder spezielle Biomsymbole sind keine eigenständigen Dynamic-Entities, sondern werden als spezielle Kacheltypen (`TileType::StructureGrassland`, `TileType::StructureMountain` etc.) während der Erzeugung eines Chunks deterministisch eingebrannt.

```

1 void World::generateChunkTiles(Chunk& chunk) {
2     // 1. Basis-Tiles und Biome auswerten
3     int biomeTag = biomeTagForChunk(chunk.cx, chunk.cy);
4
5     // Exclusion-Check: Spawn- und Exit-Chunks erhalten keine
6     // Strukturen
7     if (isSpawnOrExitChunk(chunk.cx, chunk.cy)) {
8         return;
9     }
10
11    // 2. Deterministische Chance fuer eine Struktur (~10%)
12    double structRoll = noise01(chunk.cx, chunk.cy, kStructureSalt)
13    ;
14    if (structRoll < 0.10) {
15        // Deterministische Platzierung innerhalb des 8x8 Chunks
16        int localX = static_cast<int>(noise01(chunk.cx, chunk.cy,
17            kStructXSalt) * kChunkSize);
18        int localY = static_cast<int>(noise01(chunk.cx, chunk.cy,
19            kStructYSalt) * kChunkSize);
20
21        TileType structTile = getStructureTypeForBiome(biomeTag);
22
23        // Stempeln des Musters im Chunk-Array
24        stampStructurePattern(chunk, localX, localY, structTile);
25    }
26 }
```

Quelltext 4.8: Biomabhängige Struktur-Platzierung bei der Chunk-Generierung in `world.cpp`

Wie in Listing 4.8 zu sehen ist, entscheidet ein deterministisches Rauschen mit einer Wahrscheinlichkeit von circa 10%, ob ein Chunk eine Landmarke erhält. Die Position und der konkrete Typ richten sich nach dem `biomeTag`. So erzeugt Biom 3 (Bergland) beispielsweise alpine Felsstrukturen, während Biom 2 (Wüste) Ruinen-Muster stempelt. Chunks mit Spieler-Spawn oder Ausgang werden explizit von der Struktur-Generierung ausgenommen.

Dynamisches Gegner-Spawning und Lokale Verfolgung

Im Gegensatz zu Terrain-Objekten entstehen Gegner (*Mobs*) nicht vorab bei der Chunk-Generierung, sondern werden lazily erzeugt, sobald der Spieler sich neuen Chunks nähert.

```

1 void Simulation::updateMobSpawns() {
2     // Radius um den Spieler in Chunk-Einheiten bestimmen
3     int chunkRadius = std::max(1, (observationRadius_ + kChunkSize
4         - 1) / kChunkSize + 1);
5     int playerCX = floorDiv(player_.x, kChunkSize);
6     int playerCY = floorDiv(player_.y, kChunkSize);
7
8     for (int cy = playerCY - chunkRadius; cy <= playerCY +
9         chunkRadius; ++cy) {
10        for (int cx = playerCX - chunkRadius; cx <= playerCX +
11            chunkRadius; ++cx) {
12            uint64_t key = chunkKey(cx, cy);
13            if (mobSpawnCheckedChunks_.insert(key).second) {
14                // Chunk wird zum ersten Mal geprueft: Versuche Mob
15                // -Spawn
16                trySpawnMobInChunk(cx, cy);
17            }
18        }
19    }
20
21    void Simulation::trySpawnMobInChunk(int cx, int cy) {
22        // Bis zu 10 Zufalls-Kandidaten im Chunk ausprobieren
23        for (int attempt = 0; attempt < 10; ++attempt) {
24            int wx = cx * kChunkSize + randomInt(0, kChunkSize - 1);
25            int wy = cy * kChunkSize + randomInt(0, kChunkSize - 1);
26
27            if (world_.isPassable(wx, wy) && !(wx == player_.x && wy ==
28                player_.y)
29                && !hasMobAt(wx, wy)) {
30
31                int biomeTag = world_.biomeTagForChunk(cx, cy);
32                MobVariant variant = mobVariantForBiomeTag(biomeTag);
33                mobs_.push_back(Mob{wx, wy, variant, MobState::Idle});
34                return; // Maximal ein Mob pro Chunk-Attempt
35            }
36        }
37    }
38 }

```

Quelltext 4.9: Lazy Mob-Spawning und lokale Bewegungslogik in simulation.cpp

- **Beobachtungsradius (chunkRadius):** Der Spawnbereich berechnet sich dynamisch anhand der Sichtweite (observationRadius_) des Spielers.

- **Redundanzvermeidung (mobSpawnCheckedChunks_):** Ein `std::unordered_set` speichert bereits evaluierte Chunks, damit nicht bei jedem Frame neue Gegner am selben Ort erzeugt werden.
- **Validierung und Biom-Adaption:** Ein Gegner wird nur auf passierbaren, unbesetzten Feldern außerhalb der Spielerposition platziert. Der Gegnertyp (`MobVariant`) richtet sich dabei über `mobVariantForBiomeTag` direkt nach den Eigenschaften des jeweiligen Bioms (zum Beispiel Wüsten- oder Eis-Varianten).
- **Begrenzte Verfolgungslogik:** Die KI der Gegner nutzt zwei Reichweiten (`detectRange` und `loseRange`). Ein Aggro-State wird nur getriggert, wenn der Spieler sich im Nahbereich (zum Beispiel in einem 7×7 -Umfeld) befindet. Bewegungen erfolgen schrittweise und nur auf unbesetzte, begehbare Felder.

4.1.8 Spieler-Bewegung, Kollisionsabfrage und Kampfdynamik

Die Interaktion des Spielers mit der prozedural erzeugten Spielwelt erfolgt über ein diskretes, grid-basiertes Koordinatensystem. Bewegung, Terrain-Restriktionen und Kampfhandlungen werden deterministisch innerhalb der Simulations-Engine ausgewertet, während spezifische Umgebungs-Effekte (wie Wasserverlangsamung) im Client-Render-Pfad moduliert werden.

Kollisionsprüfungen und Passierbarkeits-Matrix

Ein Bewegungsschritt entspricht exakt der Verschiebung um eine Einheit auf dem diskreten Kachelraster. Vor jeder Positionsaktualisierung evaluiert die Methode `tryMove()` das Ziel-Feld (x', y') .

```

1 // Bewegungsprüfung in simulation.cpp
2 bool Simulation::tryMove(int dx, int dy) {
3     int targetX = player_.x + dx;
4     int targetY = player_.y + dy;
5
6     // 1. Passierbarkeitsprüfung des Ziel-Tiles
7     if (!world_.isPassable(targetX, targetY)) {
8         return false;
9     }
10
11    // 2. Entitäts-Kollisionsprüfung (Blocking durch Gegner)
12    if (hasMobAt(targetX, targetY)) {
13        return false;
14    }
15
16    player_.x = targetX;
17    player_.y = targetY;
18    return true;
19 }
20
21 // Passierbarkeits-Matrix in object.cpp

```

```

22 bool World::isPassable(int x, int y) const {
23     TileType tile = tileAt(x, y);
24     switch (tile) {
25         case TileType::Empty:
26         case TileType::Exit:
27             return true; // Passierbar
28
29         case TileType::Wall:
30         case TileType::Tree:
31         case TileType::Resource:
32         case TileType::Workbench:
33         case TileType::StructureGrassland:
34         case TileType::StructureMountain:
35             return false; // Harte Blockade
36
37         default:
38             return false;
39     }
40 }

```

Quelltext 4.10: Diskrete Bewegungs- und Kollisionsprüfung in `simulation.cpp` und `object.cpp`

Wie in Listing 4.10 dargestellt, werden Hindernisse wie `Wall`, `Tree`, `Resource` oder stempelförmige Geländestrukturen als unpassierbar eingestuft. Bewegungen auf von Gegnern besetzte Felder werden ebenfalls abgewiesen, um eine physische Überschneidung von Entitäten zu unterbinden.

Client-seitige Terrain-Verlangsamung (Wasser-Mechanik)

Gewässer (`isLakeAt`) stellen im Weltmodell keine unpassierbare Barriere dar, sollen den Spieler jedoch beim Durchqueren spürbar ausbremsen. Anstatt die Core-Simulationslogik mit veränderlichen Geschwindigkeitszuständen zu verkomplizieren, setzt der Client im Render-Pfad eine Takt-Verlangsamung um:

- **Input-Modulation:** Betritt der Spieler ein als See markiertes Feld oder versucht, sich auf ein solches zu bewegen, fängt die Render-Engine die Eingabe ab und wandelt sie temporär in eine Warten-Aktion (`Wait`) um.
- **Verzögerungs-Timer:** Durch das Aufschalten eines Cooldown-Intervalls von ca. 0,40s verdoppelt sich die effektive Schritzeit auf Wasserfeldern. Dies simuliert das Waten durch tiefes Gewässer, ohne die diskrete Weltmechanik aufzuweichen.

Da diese Verlangsamung ausschließlich im Client umgesetzt ist, wirkt sie im headless betriebenen RL-Training nicht: Der Agent durchquert Seen dort ohne Zeitverlust, während ein menschlicher Spieler spürbar ausgebremst wird. Trainings- und Spielumgebung unterscheiden sich in diesem Punkt.

Flächenbasiertes Kampfsystem und Proximity Damage

Das Kampfsystem ist bewusst schlank strukturiert, um eine hohe Performanz und Vorhersehbarkeit im Runden- beziehungsweise Taktmodell zu gewährleisten.

```

1 void Simulation::useAction() {
2     // Abzug von Ausdauer/Energie
3     if (player_.energy < kAttackEnergyCost) return;
4     player_.energy -= kAttackEnergyCost;
5
6     // 3x3-Flächenangriff um die Spielerposition
7     for (int dy = -1; dy <= 1; ++dy) {
8         for (int dx = -1; dx <= 1; ++dx) {
9             int targetX = player_.x + dx;
10            int targetY = player_.y + dy;
11
12            if (Mob* mob = getMobAt(targetX, targetY)) {
13                mob->hp -= player_.damage;
14                if (mob->hp <= 0) {
15                    removeMob(mob);
16                }
17            }
18        }
19    }
20    updateMobIndex();
21 }

```

Quelltext 4.11: Flächenangriff des Spielers und Schadensabwicklung in `simulation.cpp`

1. **Spieler-Angriff (`useAction`):** Der Angriff wird flächendeckend in einer 3×3 -Umgebung um den Spieler ausgeführt. Alle im Nahbereich stehenden Gegner erleiden gleichzeitig Schaden (`player.damage`). Fällt die Lebensenergie eines Mobs auf ≤ 0 , wird dieser unmittelbar aus der Entitätsliste entfernt (Listing 4.11).
2. **Gegner-Schaden (*Proximity Damage*):** Gegner besitzen keine aufwändigen Angriffs-Animationen oder Fernkampf-Geschosse. Sobald sich ein aggressiver Mob in der direkten 3×3 -Nachbarschaft des Spielers befindet, erzeugt er über die Simulationsschritte hinweg automatischen Nahkampfschaden an den Trefferpunkten des Spielers.

Sichtfeld und dynamische Beleuchtung im Client

Zur Verstärkung des Entdeckungscharakters blendet die Render-Engine des Clients ein kreisförmiges Sichtfeld um die Spielfigur ein; Bereiche außerhalb des aktiven Sichtradius werden abgedunkelt dargestellt. Diese Verdunkelung ist rein clientseitig umgesetzt und wirkt sich nicht auf den Beobachtungsvektor des RL-Agenten aus, der unverändert ein quadratisches 15×15 -Fenster liefert (siehe Abschnitt 4.2.2).

4.2 Implementierung des RL-Trainings

Nachdem die technische Grundlage der Spielumgebung gelegt wurde, widmet sich dieser Abschnitt der Umsetzung des eigentlichen RL-Trainings. Es geht primär um das „Wie“: Im Mittelpunkt stehen konkrete Code-Architekturen, Werte und Implementierungsdetails. Die methodischen Gründe für diese Entscheidungen (das „Warum“) werden an dieser Stelle vorausgesetzt. Maßgeblich für alle hier genannten Werte und Logiken ist ausschließlich der implementierte Quellcode.

4.2.1 Technologie-Stack

Die gesamte Trainingspipeline ist in Python geschrieben. Um dennoch die Performance des C++-Simulationskerns zu nutzen, erfolgt der Zugriff über ein kompiliertes Modul. Tabelle 4.2 gibt einen kurzen Überblick über die verwendeten Werkzeuge.

Tabelle 4.2: Eingesetzte Werkzeuge des RL-Trainings

Werkzeug	Rolle im System	Variante
Python	Trainings- und Auswertungssprache	≥ 3.10
gymnasium	Schnittstelle zwischen Spielwelt und Framework	Standard-Env-API
Stable-Baselines3	RL-Framework (PPO, DQN, A2C)	–
sb3-contrib	Erweiterung für RecurrentPPO	–
pybind11	Brücke zum C++-Simulationskern	kompiliertes Modul
Rechenweg	Beschleunigung via CPU, MPS oder CUDA	Device-Parameter

Verwendet wird **sb3-contrib** als direkte Ergänzung zum Stable-Baselines3-Kernframework. Der Grund dafür ist pragmatisch: Das Standard-Framework bringt von Haus aus keine rekurrente Policy mit, die aber für den LSTM-Ansatz (RecurrentPPO) benötigt wird. Obwohl das Framework auch Algorithmen wie DQN und A2C anbietet, konzentriert sich das Training ausschließlich auf den Vergleich von PPO und RecurrentPPO. Um den Durchsatz zu maximieren, laufen konstant 16 Instanzen der Umgebung parallel; die Anzahl folgt der in der Praxis gebräuchlichen Konvention, parallele Environment-Instanzen als Zweierpotenz zu wählen, ohne dass die exakte Kernzahl der verwendeten Hardware im Rahmen dieser Arbeit gezielt abgestimmt wurde.

4.2.2 Anbindung der Spielumgebung und Aufbau der Beobachtung

Damit ein RL-Framework überhaupt mit der C++-Welt kommunizieren kann, ist ein standardisierter Übersetzer nötig. Diese Aufgabe übernimmt die Python-Klasse `StoneforgeWorldEnv`. Sie implementiert die bekannte `gymnasium`-Schnittstelle mit `reset()` und `step()`, wandelt die Aktionen des Agenten in Simulationsschritte um und baut aus dem Spielzustand den Beobachtungsvektor zusammen.

```

1 def _normalize(self, raw: list[int]) -> np.ndarray:
2     arr = np.asarray(raw, dtype=np.float32)

```



```

3     gs = self._gs
4     arr[:gs] /= 30.0                                # grid
        tiles
5     arr[gs] /= 10.0                                # hp
6     arr[gs+1] /= 100.0                              # energy
7     arr[gs+2] = np.clip(arr[gs+2], 0.0, 64.0) / 64.0 # inventory
8     arr[gs+3] /= 64.0                                # exitDx
9     arr[gs+4] /= 64.0                                # exitDy
10    step_frac = np.float32(self._step_count / self._max_steps)
11
12    if not self.use_visited_mask:
13        extras = []
14        if self.use_step_frac:
15            extras.append(step_frac)
16        # ... (use_visit_count, use_last_action_reward: in der v12-
            Konfiguration deaktiviert)
17
18        if not self.include_energy_inventory:
19            # Energie (gs+1) und Inventar (gs+2) entfernen: [grid /
                hp / exitDx / exitDy]
20            arr = np.concatenate([arr[:gs + 1], arr[gs + 3:]])
21        return np.concatenate([arr] + [
22            np.atleast_1d(np.float32(e)) if np.isscalar(e) else e
23            for e in extras
24        ])
25    # ... (CNN-Variante mit Visited-Mask: in der v12-Konfiguration
        nicht aktiv)

```

Quelltext 4.12: Aufbau des Beobachtungsvektors, python/stoneforge_env.py

Die in Listing 4.12 definierte Struktur gießt die Problemformulierung in Code:

- **observation_space:** Ein Box-Raum mit exakt 229 Dimensionen. Er besteht aus dem lokalen 15×15 -Raster (225 Kacheln) um den Agenten herum sowie vier Zusatzmerkmalen: Den aktuellen Trefferpunkten, den beiden normierten Richtungskomponenten zum Ausgang (`exitDx`, `exitDy`) und dem Anteil des verbrauchten Schrittbudgets. Vormalig geplante Werte wie Energie oder Inventar wurden als irrelevante, tote Merkmale bewusst entfernt. Die Trefferpunkte bleiben dagegen im Vektor erhalten, obwohl sie bei deaktivierten Gegnern (siehe unten) ebenso konstant und damit tot sind; sie wurden aus Gründen der Vergleichbarkeit mit früheren Modellständen nicht entfernt.
- **action_space:** Ein diskreter Raum (`Discrete(4)`) für die vier Himmelsrichtungen. Mining, Bauen und Kampf sind im RL-Betrieb vollständig deaktiviert.
- **Deaktivierte Gegner:** Beim Reset der Umgebung stempelt der Wrapper die Weltgenerierung explizit mit `disable_mobs=True`. Es werden daher während des Trainings keine Gegner erzeugt; der Agent kann also weder Schaden nehmen noch angreifen, obwohl der Reward-Term für Trefferschaden und die Trefferpunkte-Komponente der Beobachtung im Code erhalten bleiben.

- **Das wandernde Sichtfenster:** Der Agent sieht immer nur den Ausschnitt direkt um sich herum. Diese Einschränkung sorgt erst dafür, dass ein partiell beobachtbarer Entscheidungsprozess (POMDP) entsteht (siehe Kapitel 2.2).

Abbildung 4.3 fasst diesen Aufbau schematisch zusammen.

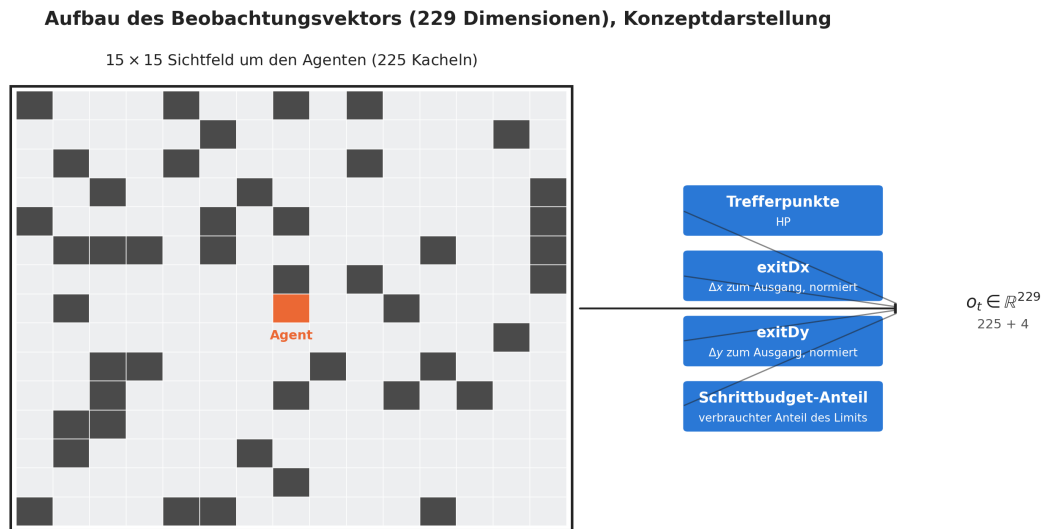


Abbildung 4.3: Aufbau des 229-dimensionalen Beobachtungsvektors, Konzeptdarstellung, eigene Darstellung.

4.2.3 Netzwerkarchitekturen: Asymmetrie zwischen LSTM- und MLP-Policy

Um beide Ansätze vergleichen zu können, erhalten sowohl die MLP- als auch die LSTM-Policy exakt den gleichen Beobachtungsvektor und Aktionsraum präsentiert. Beim Aufbau der Netze und den Batch-Größen zeigt der Quellcode jedoch eine bewusste Asymmetrie.

```

1 RPPO_KWARGS = dict(
2     policy="MlpLstmPolicy",
3     n_steps=256,
4     batch_size=8,          # batch=64 destabilisiert den Critic (
5                             CHANGELOG v2026-07-07.4)
6     n_epochs=10,
7     learning_rate=3e-4,
8     gamma=0.999, gae_lambda=0.95, clip_range=0.2,
9     ent_coef=0.05, vf_coef=0.5,
10    policy_kwargs=dict(n_lstm_layers=1, lstm_hidden_size=256,
11                        shared_lstm=False,
12                        enable_critic_lstm=True),
13    verbose=1, tensorboard_log="logs/tensorboard/",
14 )
15
16 # Gedächtnislose Kontrollgruppe für F2 (--algo ppo). Alles außer
17   den
18 # architekturspezifischen Parametern ist mit RPPO_KWARGS identisch.

```

```

16 # net_arch: [256,256] spiegelt lstm_hidden_size=256. Der SB3-
    # Default [64,64]
17 # würde die Baseline über die Kapazität benachteiligen statt ü
    # ber das Gedächtnis.
18 PPO_KWARGS = dict(
19     policy="MlpPolicy",
20     n_steps=256,
21     batch_size=256,
22     n_epochs=10,
23     learning_rate=3e-4,
24     gamma=0.999, gae_lambda=0.95, clip_range=0.2,
25     ent_coef=0.05, vf_coef=0.5,
26     policy_kwargs=dict(net_arch=[256, 256]),
27     verbose=1, tensorboard_log="logs/tensorboard/",
28 )

```

Quelltext 4.13: Instanziierung der Modelle mit asymmetrischen Parametern, `scripts/train_curriculum.py`

Wie im Code (Listing 4.13) erkennbar, unterscheiden sich die Verfahren fundamental in ihrer Dimensionierung:

- **lstm_hidden_size & Architektur:** Das LSTM nutzt eine rekurrente Schicht mit 256 Einheiten (`lstm_hidden_size=256`), die nicht zwischen Actor und Critic geteilt wird (`shared_lstm=False`). Das MLP-Netz nutzt hingegen zwei klassische Schichten à 256 Neuronen. Dies führt zu einer deutlichen Parameter-Asymmetrie: Das LSTM-Modell besitzt 1.038.917 Parameter, das MLP-Modell lediglich 250.629.
- **batch_size:** Das MLP nutzt einen Wert von 256, weil die 256×16 gesammelten Schritte damit in exakt 16 Minibatches je Epoche zerfallen; der Wert liegt bewusst über dem Framework-Standard von 64. Das LSTM nutzt hingegen einen stark reduzierten Wert von 8, was bei denselben 4096 gesammelten Schritten 512 Minibatches je Epoche ergibt. Dies ist eine technische Notwendigkeit für den LSTM-Critic: Die deutlich höhere Zahl an Gradientenschritten stabilisiert das Training, während ein größerer Batch von 64 den Critic spürbar destabilisierte (siehe Abschnitt 5.2.5).
- **n_epochs & n_steps:** Identisch bei beiden Verfahren. Jede Aktualisierung basiert auf 256 gesammelten Zeitschritten je der 16 parallelen Umgebungen, insgesamt also 4096 Schritten, die über 10 Epochen (`n_epochs=10`) trainiert werden.
- **Durchsatz:** Diese Architekturunterschiede spiegeln sich deutlich in der Rechengeschwindigkeit wider. Gemessen auf der in Abschnitt 5.1.5 genannten Entwicklungsmaschine erreicht MLP im Training einen Durchsatz von 5989 Schritten pro Sekunde, während das schwere LSTM-Netz bei 104 Schritten pro Sekunde limitiert.

4.2.4 Trainingspipeline und duales Curriculum-Gating

Das Training erfolgt in vier fließenden Phasen. Beim Phasenwechsel wird das Training nicht neu gestartet, das Modell trainiert nahtlos weiter. Der Wechsel in die nächste Phase (Gating) ist dabei nicht rein leistungsbasiert, sondern nutzt eine duale Abbruchbedingung.

```

1 PHASES = [
2     {"exit_min": 5, "exit_max": 12, "max_steps": 500_000, "
3       target_sr": 0.85,
4       "eval_min": 5, "eval_max": 12, "label": "Phase 1 (exit=5-12)
5       ",
6       "gate_metric": "stoch", "eval_max_steps": 4000},
7     {"exit_min": 12, "exit_max": 25, "max_steps": 500_000, "
8       target_sr": 0.70,
9       "eval_min": 12, "eval_max": 25, "label": "Phase 2 (exit
10      =12-25)",
11      "gate_metric": "stoch", "eval_max_steps": 4000},
12     {"exit_min": 25, "exit_max": 45, "max_steps": 1_000_000, "
13       target_sr": 0.70,
14       "eval_min": 35, "eval_max": 45, "label": "Phase 3 (exit
15       =25-45, eval 35-45)",
16       "gate_metric": "det", "eval_max_steps": 4000},
17     {"exit_min": 25, "exit_max": 45, "max_steps": 200_000, "
18       target_sr": 0.60,
19       "eval_min": 35, "eval_max": 45, "label": "Phase 4 (Greedy
20       Fine-Tune)",
21       "gate_metric": "det", "eval_max_steps": 4000, "
22       ent_coef_override": 0.0001},
23 ]
24
25 # Callback: stoppt eine Phase vorzeitig bei erreichter Ziel-SR,
26 # sonst laeuft
27 # model.learn(total_timesteps=phase["max_steps"]) bis zum
28 # Budgetlimit durch.
29
30 def _on_step(self) -> bool:
31     if self.n_calls % self.eval_freq == 0:
32         sr = self._run_eval()
33         if sr >= self.target_sr:
34             self.consecutive_successes += 1
35             if self.consecutive_successes >= 2:
36                 return False # Training stoppen, naechste Phase
37                               # starten
38         else:
39             self.consecutive_successes = 0
40     return True

```

Quelltext 4.14: Curriculum-Phasen mit dualen Abbruchbedingungen, scripts/train_curriculum.py

Die Tabelle und Listing 4.14 zeigen zwei Besonderheiten der Implementierung:

- **Das duale Gate:** Eine Phase endet, wenn *entweder* die Ziel-Erfolgsquote (Target-SR) in zwei aufeinanderfolgenden Evaluationen erreicht wird, *oder* das fest definierte Schrittbudget (Budget) aufgebraucht ist. In der Praxis zeigte sich, dass in Phase 3

und 4 stets das Schrittbudget den Phasenwechsel erzwang, da das Leistungs-Gate nicht mehr erreicht wurde. In den leichteren Phasen 1 und 2 setzte sich das Budget bei den LSTM-PPO-Läufen ebenfalls mehrheitlich durch: In Phase 1 erreichte nur einer der sieben Läufe das Leistungs-Gate vorzeitig, in Phase 2 gelang dies dreien, während die übrigen Läufe auch dort das jeweilige Schrittbudget vollständig ausschöpften.

- **Asymmetrische Evaluation:** In Phase 3 und Phase 4 trainiert der Agent auf einer Distanz von 25 bis 45 Schritten, wird aber auf einem künstlich verengten, schwierigeren Fenster von 35 bis 45 Schritten evaluiert. Das Gate soll damit gezielt die schweren Fälle der Distanzspanne prüfen, statt durch die im Training enthaltenen kürzeren, leichteren Distanzen künstlich beschönigt zu werden.
- **Doppelrolle der Metrik:** Die gemessene Erfolgsquote beendet nicht nur (potenziell) die Phase, sie fungiert gleichzeitig als Metrik für die Auswahl und Speicherung des besten Modell-Checkpoints.

Abbildung 4.4 zeigt den tatsächlichen Trainingsverlauf aller sieben LSTM-PPO-Läufe, getrennt nach Curriculum-Phase.

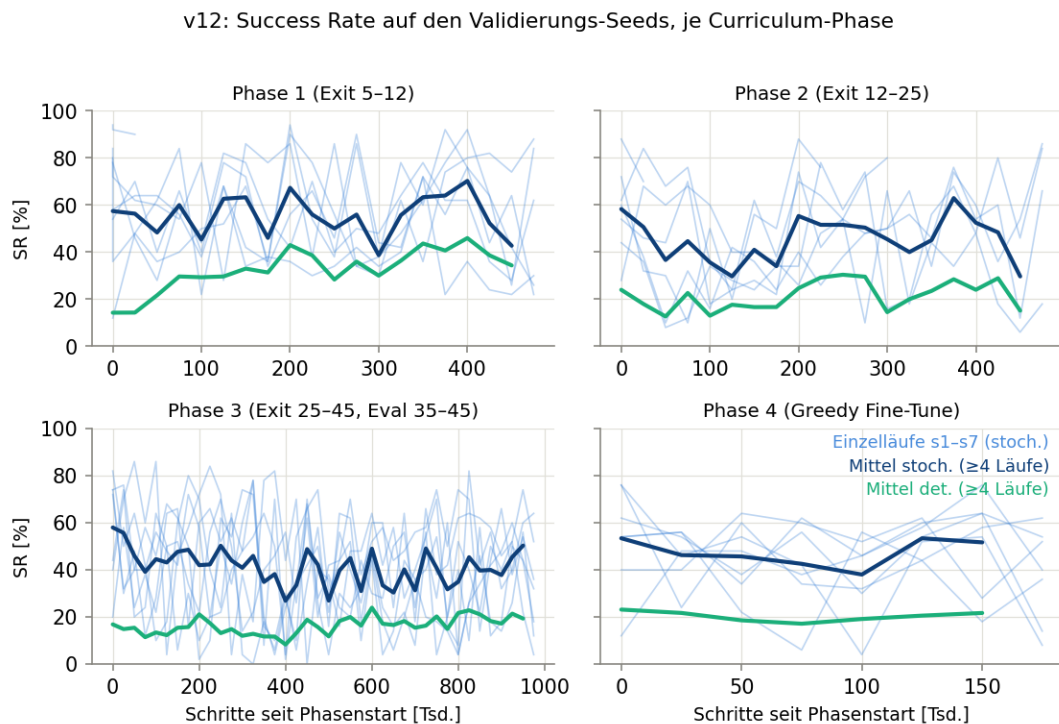


Abbildung 4.4: Erfolgsquote der sieben v12-Läufe über die Trainingsschritte, je Curriculum-Phase, eigene Darstellung.

Die Streuung der dünnen Einzellauf-Linien bleibt über alle vier Phasen hinweg beträchtlich, ein einzelner Lauf ist damit kein verlässlicher Indikator für das Verhalten des Verfahrens. In Phase 3 und 4 verharrt zudem auch das Ensemble-Mittel unterhalb der jeweiligen Ziel-Erfolgsquote, was den oben beschriebenen Befund bestätigt, dass in diesen Phasen durchgehend das Schrittbudget statt des Leistungs-Gates den Phasenwechsel auslöste.

Der Basiswert 0,05 selbst geht auf einen frühen Vergleich zurück: Bei einem niedrigeren Wert von 0,01 blieb die erklärte Varianz des Critics nahe null und die Erfolgsquote nach 50.000 Schritten bei 0%, während 0,05 die erklärte Varianz auf über 0,9 trieb und damit erst ein lernfähiges Value-Netz ermöglichte (siehe Abschnitt 5.2.5). Um dem Agenten den Übergang in schwerere Phasen zu erleichtern, wird der Entropie-Koeffizient (`ent_coef`) ausgehend von diesem Basiswert in Phase 3 über 500.000 Schritte stetig auf 0,001 abgesenkt (Annealing). In Phase 4 wird er zu Beginn der Phase zusätzlich um eine weitere Zehnerpotenz auf fix 0,0001 abgesenkt, da diese Phase gezielt als *Greedy Fine-Tune* konzipiert ist: Die Modellauswahl erfolgt hier anhand der deterministischen statt der stochastischen Erfolgsquote, wofür der Agent von einem explorativen nahezu vollständig in ein deterministisches Verhalten übergehen muss.

4.2.5 Reward-Shaping und Distanzfeld

Die Belohnungsstruktur ist überwiegend direkt in den performanten C++-Simulationskern eingewoben, mit Ausnahme einer im Python-Wrapper der Umgebung nachgelagerten Anti-Stuck-Strafe. Entgegen früheren Entwurfsstadien wurde die Strafen-Struktur stark entschlackt, da sich zeigte, dass sich aufsummierende Strafen (Penalty-Stacking) das Erkundungsverhalten des Agenten unterdrückten.

```

1 float reward = -0.01F;
2
3 if (newTileVisited) {
4     reward += 0.02F;
5 }
6 if (idleAction) {
7     reward -= 0.04F;
8 }
9 const int damage = std::max(0, hpBefore - hp_);
10 reward -= static_cast<float>(damage) * 0.5F;
11
12 if (positionLoop) {
13     reward -= 0.05F;
14 }
15
16 // Potential Based Reward Shaping (PBRs):  $\Phi(s) = -BFS(s) /$ 
17    $PBRs\_MAX\_DIST$ 
18 constexpr float PBRs_MAX_DIST = 128.0F;
19 constexpr float PBRs_GAMMA = 0.999F; // Shaping-Discount = RL-
20   Discount
21 constexpr float PBRs_BETA = 2.5F; // +0.01 netto pro Tile
22   Fortschritt bei d=40
23
24 const float phi_prev = -static_cast<float>(previousDistance) /
25   PBRs_MAX_DIST;
26 const float phi_cur = -static_cast<float>(currentDistance) /
27   PBRs_MAX_DIST;

```

```

23 const float shaping = PBRG_GAMMA * phi_cur - phi_prev;
24 reward += PBRG_BETA * shaping;
25
26 if (reachedExit) {
27     reward += 100.0F;
28 }
29 if (done_ && !reachedExit) {
30     reward -= 10.0F;
31 }
32 return reward;

```

Quelltext 4.15: Berechnung des Rewards und Shaping-Terms, `src/core/simulation.cpp`

Die konkreten Zahlenwerte aus Listing 4.15 sind hart im Code verankert: Ein normaler Schritt kostet $-0,01$, das Betreten einer neuen Kachel bringt $+0,02$, Stehenbleiben wird mit $-0,04$ bestraft. Diese Stehenbleiben-Strafe (`idleAction`) greift ausschließlich bei den Aktionen `Wait` und `Noop` des Simulationskerns; im RL-Binding sind ausschließlich die vier Bewegungsaktionen zugelassen (siehe Abschnitt 4.2.2), sodass diese Strafe während Training und Evaluation nie ausgelöst wird und als Rest aus dem spielbaren Client im Reward-Code verbleibt. Wiederholt der Agent eine Zwei-Schritt-Schleife, erhält er $-0,05$ (ein Wert, der von ehemals $-0,15$ abgeschwächt wurde, da die Kombination aus einem damaligen, inzwischen aus dem Reward-Code entfernten eskalierenden Wand-Penalty und der scharfen Loop-Strafe das Erkunden in engen Korridoren teurer machte als bloßes Stillstehen; die Erkennung der Schleife blieb erhalten, nur ihre Strafhöhe wurde moderater gesetzt).

Nimmt er Schaden, kostet das $-0,5$ pro Schadenspunkt. Der Terminal-Reward für das Erreichen des Ziels beträgt $+100,0$, ein Timeout durch das Episodenlimit (4000 Schritte) resultiert in $-10,0$. Zusätzlich verfolgt der Python-Wrapper der Umgebung die Besuchshäufigkeit jeder Position: Wird ein Feld zum 26. Mal oder öfter besucht, zieht das $0,03 \cdot \min(\text{Besuche}/25, 2,0)$ vom Reward ab, gedeckelt bei $-0,06$ ab dem 50. Besuch. Eine frühere Fassung setzte diese Schwelle noch bei 300 Besuchen an; die Verschärfung auf 25 beziehungsweise 26 Besuche greift damit deutlich früher in wiederholtes Aufsuchen derselben Position ein, ohne dass die konkreten neuen Zahlenwerte über eine eigene Parameterstudie hinaus abgesichert wurden.

Das Herzstück bildet jedoch das **Potential Based Reward Shaping (PBRG)**, implementiert nach der Formel $F(s, s') = \beta \cdot (\gamma \Phi(s') - \Phi(s))$:

- **Potenzialfunktion Φ :** Definiert als $-\text{BFS-Distanz}(s)/128,0$. Da γ passend zum restlichen RL-Setup bei $0,999$ liegt und der Faktor β auf $2,5$ skaliert ist, ergibt sich ein reines Shaping-Signal von $+0,0203$ je Feld Fortschritt (bei einer Distanz von 40), was inklusive der Schrittkosten von $-0,01$ einen Nettoeffekt von $+0,01$ ergibt. Der Diskontierungsfaktor γ liegt damit deutlich über dem in der PPO-Literatur gebräuchlichen Standardwert von $0,99$: Bei den bis zu 4000 Schritte langen Episoden dieser Arbeit würde der niedrigere Standardwert das Terminal-Reward über den Episodenverlauf auf einen verschwindend kleinen Diskontfaktor in der Größenord-

nung von 10^{-18} drücken, während bei $\gamma = 0,999$ noch rund 0,018 verbleiben. Der höhere Wert verhindert damit, dass der spärliche Erfolgs-Reward am Episodenende gegenüber den dichten PBRs-Zwischensignalen praktisch bedeutungslos wird.

- **Das Trainings-Distanzfeld (BFS):** Hier müssen zwei Systeme auseinandergehalten werden. Die nachträgliche Erreichbarkeitsprüfung bei der *Weltgenerierung* ist deaktiviert (die Lösbarkeit wird dort bereits durch die BFS-basierte Ausgangsplatzierung garantiert, siehe Abschnitt 4.1.4). Das Distanzfeld für das *Training* ist jedoch hochaktiv: Es wird bei jedem Episoden-Reset ausgehend vom Ausgang berechnet. Es dient sowohl dem Shaping als auch als Nenner für die spätere Pfadeffizienz-Metrik.
- **Puffergröße:** Die BFS berechnet eine Bounding-Box aus Spawn und Exit, erweitert um einen Puffer von 80 Feldern. Historisch führte ein kleinerer Puffer (20) kombiniert mit einem aggressiveren β (5,0) dazu, dass der Agent am Puffer-Rand falsches Shaping erhielt; β wurde daher nach dem Puffer-Fix im Code auf 2,5 halbiert.

4.2.6 Umsetzung der Baselines: Die wandblinde Kompass-Heuristik

Um die trainierten RL-Modelle bewerten zu können, braucht es ungelernete Referenzwerte (Baselines). Diese greifen direkt auf die Spiellogik zu und erfordern keine Trainingsphase.

```

1 class CompassPolicy:
2     """eps-verrauschter Greedy-Lauf entlang des Luftlinien-
        Kompasses."""
3
4     def __init__(self, eps: float, seed: int = 0) -> None:
5         self.eps = eps
6         self.rng = np.random.default_rng(seed)
7         self.name = f"Kompass-Zufallslauf (eps={eps:g})"
8
9     def reset(self) -> None:
10         return None
11
12     def __call__(self, obs, ctx):
13         if self.rng.random() < self.eps:
14             return int(self.rng.integers(4)), None
15         dx, dy = float(obs[IDX_DX]), float(obs[IDX_DY])
16         if abs(dx) > abs(dy):
17             return (RIGHT if dx > 0 else LEFT), None
18         return (DOWN if dy > 0 else UP), None

```

Quelltext 4.16: Aktionswahl der Referenzverfahren, `scripts/eval_baselines.py`

Die Baselines aus Listing 4.16 offenbaren ein wichtiges technisches Detail:

- **BFS-Optimum:** Dient nur der Messung der absoluten Mindestschritte (Pfadeffizienz) und wird nicht als Policy ausgeführt.
- **Random-Agent:** Wählt komplett blind und gleichverteilt eine der vier Richtungen.

- **ε -Kompass:** Mit der Wahrscheinlichkeit ε verhält er sich wie der Random-Agent. Andernfalls liest er schlicht die beiden Luftlinien-Richtungen ($\mathbf{dx}$, $\mathbf{dy}$) aus dem Beobachtungsvektor und geht einen Schritt auf der längeren Achse.

Dies macht die Kompass-Heuristik de facto **wandblind**. Sie nutzt keine Pfadfindungs-Logik (wie etwa den BFS-Gradienten) und kennt keine Hindernisse. Sie rennt stumpf gegen Wände, bis der Zufallsanteil ε sie wieder löst. Dass ein komplett kartenblindes Verfahren in der Evaluation derart stark abschneidet, ist eine zentrale Erkenntnis für die spätere Auswertung und erklärt zugleich, warum bei dieser Baseline im getesteten Bereich $\varepsilon \in [0,3; 0,9]$ mit steigendem Zufallswert ε die Erfolgsquote steigt und die Pfadeffizienz zugleich sinkt. Dieser Zusammenhang gilt jedoch nicht uneingeschränkt: Bei $\varepsilon = 1$ entspricht die Heuristik exakt dem Random-Agent, dessen Erfolgsquote von 5,2 % (siehe Abbildung 5.2) weit unter dem Wert bei $\varepsilon = 0,9$ liegt, sodass die Kurve zwischen beiden Randwerten einbrechen muss.

4.2.7 Vollständige Hyperparameter-Konfiguration

Um die vollständige Reproduzierbarkeit zu gewährleisten, sind in Tabelle 4.3 alle verbleibenden, fest im Code verankerten Hyperparameter für den Optimierer (PPO) sowie die generellen Testbedingungen aufgeführt. Für die dort nicht gesondert diskutierten Optimierer-Parameter (`learning_rate`, `gae_lambda`, `clip_range`, `vf_coef`) werden die etablierten Standardwerte aus Schulman et al. [19] beziehungsweise die Framework-Defaults übernommen, da die Trainingshistorie (siehe Tabelle 5.2) keinen Anlass zur Abweichung zeigte. Episoden werden regulär nach einem Limit von 4000 Schritten gestoppt. Zusätzlich erzwingt das System einen Early-Stop, wenn der Agent 256 Schritte lang keine positive Belohnung sammelt; auch dieser Schwellenwert folgt der Zweierpotenz-Konvention verwandter Zähl- und Batch-Parameter dieser Arbeit und wurde nicht durch eine eigene Ablationsstudie gegen benachbarte Werte abgesichert. Um Varianzen durch die Generierung auszugleichen, werden die Ergebnisse über sieben unterschiedliche Zufallssaaten (Seeds) gemittelt.

Tabelle 4.3: Allgemeine Hyperparameter des PPO-Optimierers

Parameter	Wert
<code>learning_rate</code>	$3 \cdot 10^{-4}$
<code>gamma</code> (γ)	0,999
<code>gae_lambda</code>	0,95
<code>clip_range</code>	0,2
<code>ent_coef</code> (Basis)	0,05
<code>vf_coef</code>	0,5
Max. Timesteps gesamt	2.200.000
Episoden- /Evaluationslimit	4000 Schritte

5 Evaluation

5.1 Evaluation der Weltgenerierung

Dieses Kapitel evaluiert die entwickelte prozedurale Weltengenerierung hinsichtlich ihrer visuellen Qualität, der logischen Kohärenz des Terrains sowie bekannter Artefakte der Darstellung. Als primäre Datengrundlage dient eine Laufzeitanalyse des prototypischen Softwareclients.

5.1.1 Visuelle Ergebnisse und Biome-Gradients

Abbildung 5.1 zeigt eine repräsentative Spielszene der generierten Welt aus der Vogelperspektive (*Top-Down-Perspektive*). Anhand der Visualisierung lassen sich die im Kapitel 4 beschriebenen Algorithmen in ihrer praktischen Zusammenwirkung überprüfen.

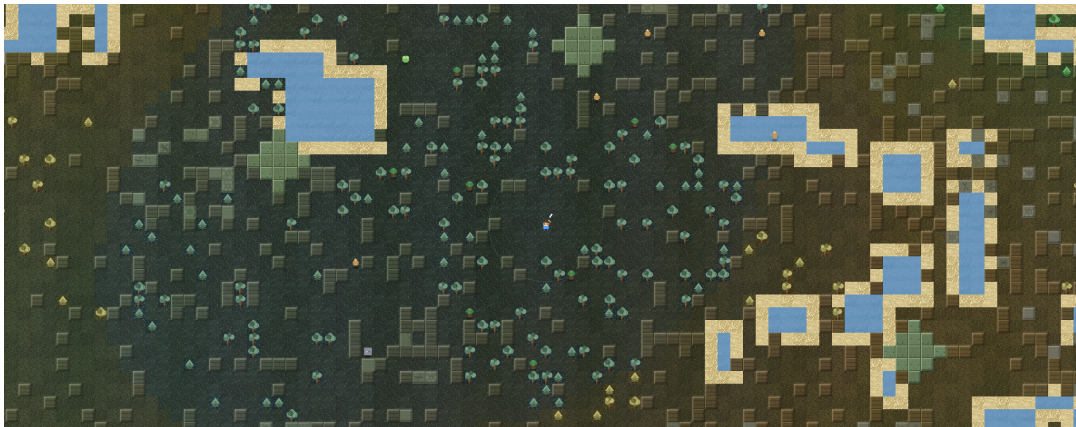


Abbildung 5.1: Generierte Spielwelt mit dynamischer Beleuchtung, Seenstrukturen und biomabhängigen Kacheltypen, eigene Darstellung (Screenshot des Prototyps, Debug-Overlay entfernt).

5.1.2 Synthese der Spielelemente

In Abbildung 5.1 werden die Kernmechanismen der prozeduralen Erzeugung deutlich sichtbar:

- **Beobachtungsradius und Sichtfeld:** Der zentrale Bereich um die Spielfigur wird durch ein kreisförmiges Sichtfeld (*Field of View* / Sichtradius) hervorgehoben. Bereiche außerhalb des aktiven Beobachtungsradius werden abgedunkelt, was den Entdeckungscharakter verstärkt.
- **Organische Gewässer und Strand-Overlay:** Die durch niederfrequentes Rauschen erzeugten Seen treten als zusammenhängende, abgerundete Wasserflächen auf.

Das im Render-Pfad verankerte Strand-Overlay erzeugt an den Begehrbarkeitsgrenzen zuverlässig helle Sandstrukturen.

- **Vegetation und Wandelemente:** Die Verteilung von Bäumen (`TileType::Tree`) und unpassierbaren Hindernissen (`TileType::Wall`) variiert entsprechend der Schwellenwerte (`wallThreshold`, `treeThreshold`) der jeweiligen Biome.
- **Landmarken und Strukturen:** Vereinzelt hervorgehobene symmetrische Fliesenmuster (zum Beispiel die quadratischen Rauten-Strukturen im oberen Zentrum) belegen das erfolgreiche Stempeln deterministischer Muster bei der Chunk-Initialisierung.

5.1.3 Artefaktanalyse: Das Phänomen des Biome-Swallowing

Trotz der optisch ansprechenden, stufenlosen Farbübergänge durch die gewichtete Biomischung (`biomeTintAtTile`) deckt die praktische Evaluation eine systematische Schwachstelle des gewählten *Biome Blending*-Ansatzes auf.

Da die Gewichtung der Biomfarben über ein 3×3 -Chunk-Umfeld berechnet wird (`biomeWeights`), besitzen flächenmäßig dominante Nachbarbiome eine überproportionale Strahlkraft auf dazwischenliegende, kleinere Biome. Dieser Effekt wird als **Biome-Swallowing** (Verschlucken von Kleinbiomen) bezeichnet:

Isolierte Phänomenologie: Besitzt ein einzelner Chunk (zum Beispiel eine 8×8 -Wüste mit orangener Grundpalette) eine räumliche Ausdehnung von nur einem Chunk (1×1) und liegt unmittelbar zwischen zwei ausgedehnten Biomen wie einem dichten Wald und einer großen Steppe, so dominiert der mathematische Mittelwert der 3×3 -Nachbarschaft.

In der Praxis führt dies dazu, dass die Wüste in der Logik (`biomeTag == 2`) zwar die korrekten Hindernis-Schwellenwerte und Baumverbote besitzt, visuell jedoch durch die Farbinformationen der umliegenden Wald- und Steppen-Chunks grünlich überfärbt wird. Der isolierte Wüsten-Chunk wird farblich „verschluckt“.

5.1.4 Lösungsansätze für kleine Biome

Um dieses Artefakt in zukünftigen Iterationen zu vermeiden, ohne auf weiche Farbübergänge verzichten zu müssen, kommen zwei Ansätze infrage:

1. **Mindestausdehnung von Biomen (*Clustering*):** Bei der Biom-Zuweisung auf Ebene des Chunk-Rasters muss durch eine zelluläre Automaten-Glättung sichergestellt werden, dass Biome stets in zusammenhängenden Clustern von mindestens 2×2 Chunks auftreten (siehe Kapitel 2.1.3).
2. **Nicht-lineare Interpolations-Gewichtung:** Das Gewicht des eigenen Chunks im Zentrum muss bei der Farberechnung deutlich höher gewichtet werden (zum Beispiel Faktor 0,6 für den eigenen Chunk und nur 0,4 verteilt auf die 8 Nachbarn), um die visuelle Eigenständigkeit kompakter Biome zu sichern.

5.1.5 Performance-Evaluation und Benchmarking

Zur empirischen Überprüfung der Laufzeit- und Speichereffizienz wurde eine automatisierte Benchmark-Routine im Headless-Runner (siehe Abschnitt 4.1.1) implementiert. Über einen dedizierten CLI-Parameter (`--benchmark-world`) führt das System eine isolierte Leistungsmessung durch, ohne vom Grafik-Rendering beeinträchtigt zu werden. Gemessen wurde auf derselben Entwicklungsmaschine wie das RL-Training, einem Apple M1 Pro (CPU). Tabelle 5.1 fasst die ermittelten Kennzahlen zusammen.

Tabelle 5.1: Übersicht der Performancemessungen und Benchmarks des Generierungs- und Simulationssystems.

Metrik / Messbereich	Parameter / Stichprobe	Ergebnis
Chunk-Generierungszeit	$N = 2048$ Chunks (8×8)	$\varnothing 13,12 \mu\text{s}$ (p95: $18,50 \mu\text{s}$)
Simulationsschritt (<i>Step Proxy</i>)	$N = 5000$ Ticks	$\varnothing 0,0137 \text{ ms}$ (p95: $0,02 \text{ ms}$)
Speicherbedarf (Sparse vs. Dicht)	2048 geladene Chunks	162 KiB (<code>unordered_map</code>) vs. 130 KiB
Determinismus-Abweichung	100 Seeds / 518.400 Tiles	0,0000 % (0 Mismatches)

Generierungsgeschwindigkeit und Lazy Loading

Die Messung der Erzeugungsdauer einzelner 8×8 -Chunks über 2048 Stichproben ergibt eine durchschnittliche Generierungszeit von nur $13,12 \mu\text{s}$ pro Chunk bei einem 95-Perzentil (p95) von $18,50 \mu\text{s}$.

Diese geringe Erzeugungsdauer bestätigt die Effizienz des mehrstufigen Noise-Ansatzes und der optimierten Tile-Zuweisung. Dank der Mikrosekunden-Latenz kann das System neue Chunks beim Betreten durch den Spieler erzeugen (*Lazy Loading*), ohne dass Ruckler (*Frame Drops*) im Render-Thread wahrnehmbar sind.

Simulationsschritt und Frame-Proxy

Um die Ausführungsgeschwindigkeit der Kern-Logik (Pfadfindung, Sichtweitenberechnung, Mob-KI und Kollision) zu bestimmen, wurden 5000 aufeinanderfolgende Simulationsschritte evaluiert:

- **Durchschnittliche Step-Dauer:** $0,0137 \text{ ms}$ (p95 = $0,02 \text{ ms}$).
- **Durchsatz-Proxy:** Rein logisch bewältigt die Engine theoretisch ca. 73.000 Simulationsschritte pro Sekunde (*Step Proxy*).

Das System bietet damit reichlich Spielraum für die grafische Darstellung im Client und ermöglicht selbst auf leistungsschwacher Hardware eine flüssige Bildrate bei konstanten 60 FPS.

Speichereffizienz der Hash-Map-Architektur

Ein zentraler Architekturentwurf der Spielwelt ist die Speicherung der Chunks in einer `std::unordered_map<uint64_t, Chunk>`. Bei 2048 aktiven Chunks (bei 8×8 Fliesen pro Chunk insgesamt 131 072 Fliesen) beläuft sich der Speicherverbrauch der Hash-Map inklusive Dynamic-Overhead auf ca. 162 KiB.

Im Vergleich dazu würde ein zweidimensionales, dicht besetztes Array derselben maximalen Spannen etwa 130 KiB belegen. Der geringfügige Speicher-Overhead von ≈ 32 KiB für die Hash-Metadaten wird durch den Vorteil aufgewogen, dass die Welt theoretisch unendlich ausgedehnt sein kann, ohne unbesetzte Weltbereiche vorab im Arbeitsspeicher allozieren zu müssen.

Determinismus und Reproduzierbarkeit

Für prozedurale Welten ist die exakte Reproduzierbarkeit essenziell. Zur Verifikation wurden 100 unterschiedliche Zufallssamen (*Seeds*) evaluiert und insgesamt 518.400 einzelne Kacheln zwischen mehrfachen Generierungsläufen verglichen:

$$\text{Abweichungsrate} = \frac{\text{Anzahl Mismatches}}{\text{Gesamtzahl verglichener Tiles}} = \frac{0}{518.400} = 0,0000 \%$$

Die Fehlerrate liegt exakt bei 0,00 %. Dies beweist, dass sowohl das Noise-Cascading für Höhen und Biome als auch die BFS-basierte Ausgangsplatzierung vollständig deterministisch arbeiten und keine unkontrollierten Nebeneffekte oder Zustandsartefakte vorliegen.

5.1.6 Kritische Diskussion und Limitationen

Die empirischen und visuellen Befunde belegen, dass die entwickelte Architektur für prozedurale Welten eine performante, deterministische und visuell ansprechende Spielumgebung bereitstellt. Dennoch ergeben sich aus den gewählten Entwurfsentscheidungen spezifische Kompromisse und methodische Grenzen, die im Folgenden kritisch gewürdigt werden.

Vorteile der Architektur

- **Nahtlose Performanz und Ladezeitfreiheit:** Durch die Kombination aus Mikrosekunden-Chunk-Generierung ($\varnothing 13,12 \mu s$) und bedarfsgesteuertem *Lazy Loading* entstehen im Spielverlauf keinerlei spürbare Ladezeiten oder Unterbrechungen (*Stuttering*), wenn der Spieler neue Kartenbereiche erschließt.
- **Garantierte Pfad-Topologie:** Die mathematisch strikte, BFS-basierte Platzierung des Ausgangs garantiert zu 100 %, dass zwischen Startpunkt (`spawn_`) und Ziel (`exit_`) stets ein begehbarer Pfad existiert. Unlösbare Welt-Instanzen werden dadurch deterministisch ausgeschlossen.
- **Sparsamer Speicherbedarf bei bedarfsgesteuertem Laden:** Dank der spärlich besetzten Hash-Map-Architektur (`std::unordered_map`) reserviert die Welt Speicher ausschließlich für tatsächlich generierte Chunks, anstatt riesige 2D-Arrays vor-

ab im RAM zu reservieren; bei 2048 geladenen Chunks liegt der gemessene Bedarf bei ≈ 162 KiB (siehe Tabelle 5.1).

Identifizierte Schwachstellen und Limitationen

- **Geometrische Künstlichkeit des Manhattan-Carvers:** Da der Pfad-Einkerber Kacheln entlang rechtwinkliger Koordinatendifferenzen aufbricht, neigt die Logik zur Bildung auffälliger „L-Shapes“ (rechtwinklige Gänge). In offenem Terrain (zum Beispiel Wiese oder Wüste) wirken diese harten 90° -Achsenbrüche stellenweise unnatürlich und brechen die organische Anmutung der Noise-Generierung.
- **Beschränkung auf flaches 2D-Raster:** Das System arbeitet auf einem rein zweidimensionalen Gitter. Echte Höhenunterschiede, Plateaus, Höhlensysteme auf mehreren Ebenen oder Verdeckungslogiken (zum Beispiel Rampen oder Brücken) können ohne Erweiterung um ein Z -Achsen-gitter oder Iso-Layer nicht abgebildet werden.
- **Unbegrenztes RAM-Wachstum ohne Chunk-Unloading:** Obwohl der Einstiegsspeicherbedarf minimal ist, besitzt der aktuelle Chunk-Cache keinen Entlademechanismus (zum Beispiel *Least Recently Used* / LRU-Unloading). Besucht ein Spieler in einer extrem langen Sitzung hunderttausende Chunks, wächst die `unordered_map` kontinuierlich an.
- **Farb-Absorption bei Kleinbiomen (*Biome-Swallowing*):** Wie in Abschnitt 5.1.3 analysiert, führt das räumlich gewichtete *Biome Blending* über 3×3 Chunks dazu, dass isolierte Einzel-Chunks (1×1) farblich von ihren dominanten Nachbarbiomen überstrahlt werden.

5.2 Evaluation des RL-Trainings

Dieser Abschnitt evaluiert die trainierten RL-Verfahren hinsichtlich Erfolgsquote, Pfadefizienz und Robustheit gegenüber den ungelerten Referenzpunkten aus Abschnitt 3.2.4. Datengrundlage sind sieben unabhängige Trainingsläufe je Modell, ausgewertet auf dem in Abschnitt 3.2.5 definierten Testset A.

5.2.1 Erfolgsquote der trainierten Verfahren

Die Erfolgsquote ist die zentrale Metrik zur Beurteilung, ob ein Verfahren die Navigationsaufgabe überhaupt löst. Abbildung 5.2 stellt die beiden trainierten Modelle den ungelerten Referenzpunkten gegenüber; das LSTM-PPO-Modell erscheint dabei zusätzlich in der deterministischen Auswertung.

Das LSTM-PPO-Modell erreicht in der stochastischen Auswertung eine mittlere Erfolgsquote von 64,6 % auf Testset A. Über die sieben Durchläufe hinweg liegt die Standardabweichung bei 10,1 Prozentpunkten, die Einzelwerte schwanken zwischen 51,6 % und 76,8 %. Auf Holdout B fällt das Ergebnis mit $68,8 \% \pm 15,5$ höher aus, was für eine gute Generalisierung auf ungesehene Seeds spricht. Im deterministischen Modus bricht die Leistung massiv ein: Die Quote fällt auf magere $29,1 \% \pm 8,0$ auf Testset A und $32,6 \% \pm 12,7$

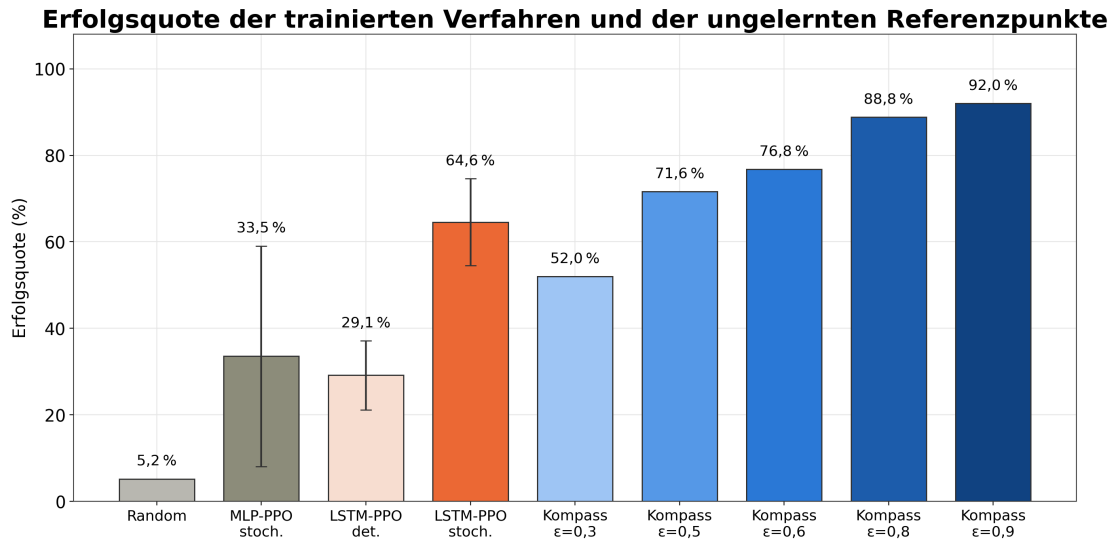


Abbildung 5.2: Evaluation – Erfolgsquote der trainierten Verfahren und der Referenzpunkte auf Testset A, eigene Darstellung.

auf Holdout B. Diese Werte stammen aus einer eigens auf denselben Testset-Seeds durchgeführten, einmaligen deterministischen Zusatzauswertung; das kanonische Eval-Skript reproduziert diesen Modus allerdings nicht mehr, weshalb die Ergebnisse hier nachrichtlich referenziert werden.

Beim MLP-PPO-Verfahren fällt das Bild schwächer aus als beim LSTM. Unter dem identischen Standardprotokoll (siehe Abschnitt 3.2.5) erreicht das Modell stochastisch ausgewertet auf Testset A $33,5\% \pm 25,5$ und auf Holdout B $35,8\% \pm 30,5$. Die Streuung zwischen den sieben Seeds fällt damit mehr als doppelt so hoch aus wie beim LSTM, die Einzelwerte auf Testset A reichen von $4,0\%$ bis $65,2\%$. Eine deterministische Zusatzmessung auf den Testset-Seeds liegt für das MLP-PPO-Modell nicht vor; der deterministische Modus wird daher nur für das LSTM-PPO-Modell berichtet (siehe Abschnitt 3.2.5).

Der Systemvergleich fällt damit eindeutig aus: Die rekurrente LSTM-Architektur schlägt das gedächtnislose MLP-Netz deutlich. Der Vorsprung beträgt rund 31 Prozentpunkte, auch wenn die starken Schwankungen des MLP-Modells für Überschneidungen einzelner Läufe sorgen. Das LSTM bleibt dennoch das klar überlegene Verfahren, wobei Abschnitt 5.2.5 diesen Vorsprung noch einordnet.

Bei nur sieben Läufen je Architektur und einer Standardabweichung von 25,5 Prozentpunkten beim MLP-Modell stellt sich die Frage, ob dieser Unterschied statistisch belastbar ist oder in der Streuung der kleinen Stichprobe verschwindet. Die zugrunde liegenden sieben Einzelwerte je Verfahren und Testset sind in Anhang C aufgeführt. Ein Welch-Test auf die sieben Erfolgsquoten je Verfahren (Testset A, stochastisch) weist den Unterschied als statistisch signifikant aus ($t(7,8) = 2,99$, $p = 0,018$); ein verteilungsfreier Mann-Whitney-U-Test verfehlt die 5-Prozent-Schwelle dagegen knapp ($U = 40$, $p = 0,053$). Die Effektstärke fällt mit Cohens $d = 1,6$ groß aus.

Auf Holdout B liegen beide Tests im statistisch signifikanten Bereich (Welch: $p = 0,031$; Mann-Whitney: $U = 41$, $p = 0,038$). Die einzelnen 95 %-Konfidenzintervalle auf Testset A überschneiden sich zwar teilweise (LSTM: $[55,3; 73,9]$, MLP: $[9,9; 57,1]$), was bei einem Ver-

gleich zweier einzelner Intervalle leicht als Beleg gegen Signifikanz gelesen werden könnte; das Konfidenzintervall der Differenz selbst schließt die Null jedoch nicht ein ($[7,0; 55,0]$). Der gemessene Vorsprung des LSTM ist damit trotz der geringen Stichprobengröße nach gängigen Kriterien statistisch abgesichert; einzig der verteilungsfreie Test auf Testset A bleibt knapp oberhalb der 5-Prozent-Schwelle.

5.2.2 Pfadeffizienz derselben Verfahren

Nur am Ausgang anzukommen, reicht als Bewertungskriterium nicht aus, wichtig ist auch der Weg dorthin. Abbildung 5.3 zeigt daher die Pfadeffizienz. Sie wird pro erfolgreicher Episode als Verhältnis zwischen dem per Breitensuche (siehe Abschnitt 2.1.4) ermittelten kürzesten Weg dieser Episode und den vom Agenten tatsächlich gelaufenen Schritten berechnet und anschließend über alle erfolgreichen Episoden gemittelt. Da dieser Mittelwert der Verhältnisse nicht dem Verhältnis der Mittelwerte entspricht, lässt sich die berichtete Pfadeffizienz nicht durch Division der weiter unten genannten Durchschnittswerte für BFS-Optimum und Schrittzahl nachrechnen. Ein Wert von 1,0 markiert das Optimum.

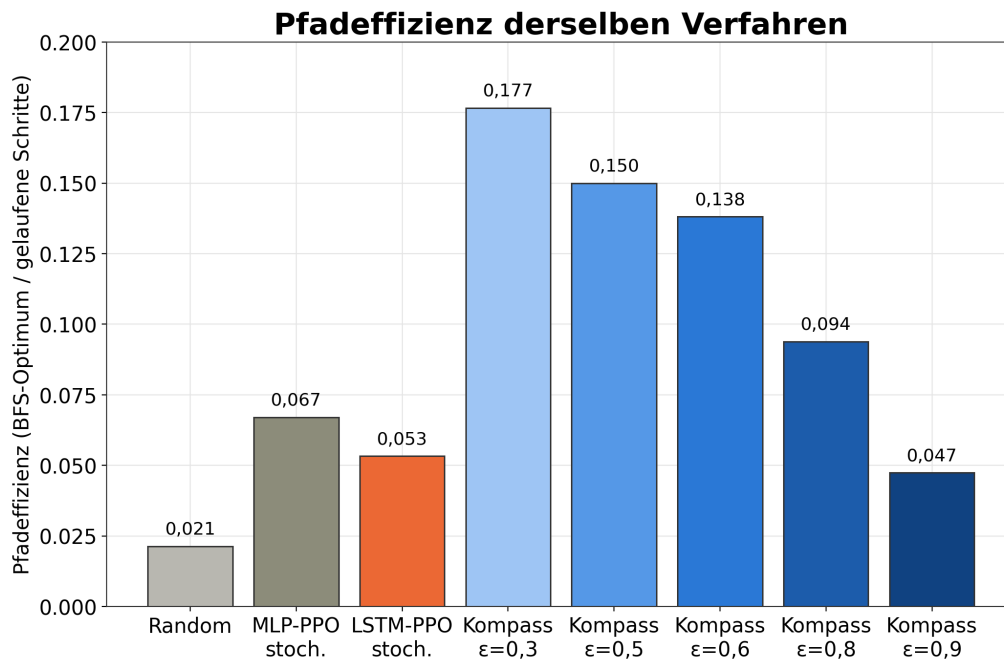


Abbildung 5.3: Evaluation – Pfadeffizienz der Verfahren in derselben Achsenordnung, eigene Darstellung.

Das BFS-Optimum liegt auf dem verwendeten Testset im Schnitt bei 40,1 Schritten. Das trainierte LSTM-Modell benötigt für eine erfolgreiche Episode dagegen durchschnittlich 1425 Schritte, bei einer gemittelten Pfadeffizienz von lediglich 0,053. Der Agent läuft damit im Schnitt einen Weg, der mehr als 35-mal so lang ist wie nötig. Abbildung 5.4 macht diesen Unterschied an einer einzelnen Episode aus Testset A sichtbar.

Der abgebildete Lauf erreicht mit 456 gegenüber 43 BFS-optimalen Schritten eine Pfadeffizienz von 0,094 und liegt damit über dem berichteten Mittel von 0,053; die wiederholten Schleifen im eingezeichneten Pfad zeigen aber bereits an dieser einzelnen, vergleichsweise günstigen Episode das charakteristische Herumirren, das die niedrige mittlere Pfadeffizienz

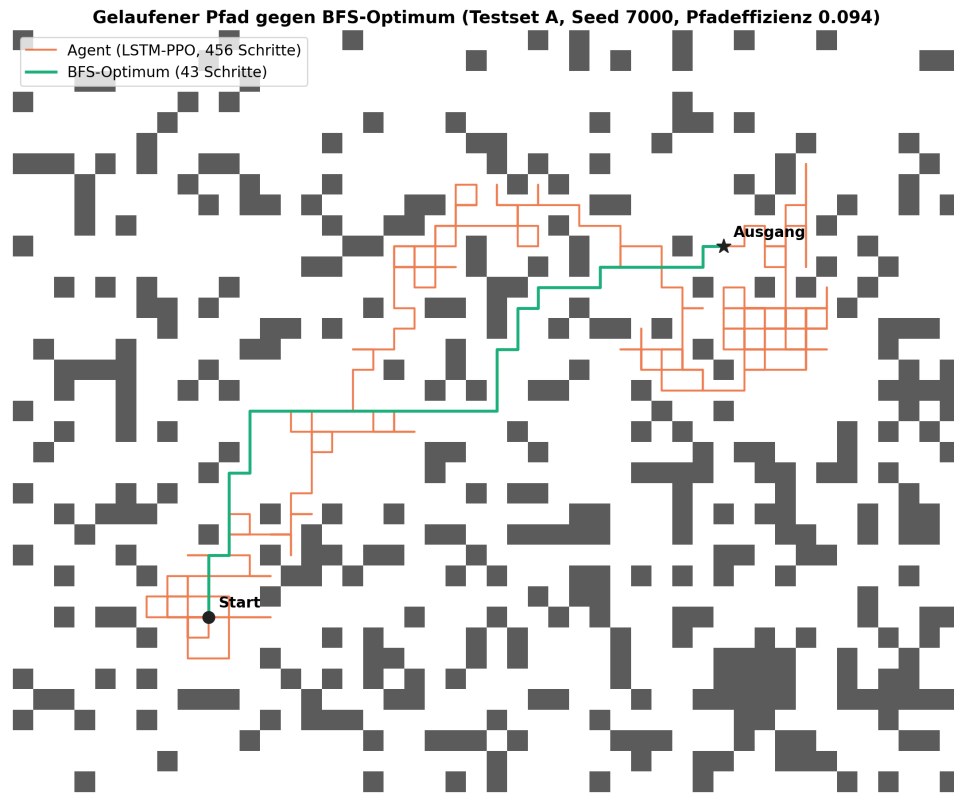


Abbildung 5.4: Evaluation – gelaufener Agentenpfad gegen BFS-Optimum auf Seed 7000 (Testset A), eigene Darstellung.

erklärt. Für das MLP-Modell liegt ein vergleichbarer Wert vor: Bei durchschnittlich 1084 Schritten je erfolgreicher Episode liegt die gemittelte Pfadefizienz bei 0,067 und damit höher als beim LSTM. Dieser Befund ist mit Vorsicht zu lesen, da die Kennzahl nur über die erfolgreichen Episoden gemittelt wird, wie der folgende Absatz zum Selektionseffekt zeigt: Bei einer Erfolgsquote von nur 33,5 % löst das MLP-Modell bevorzugt die leichten Seeds, was die Effizienz künstlich anhebt.

Aufschlussreich wird der Blick auf die Referenzverfahren aus Abschnitt 3.2.4, deren vollständige Ergebnistabelle in Anhang C aufgeführt ist. Der vollständig zufällig agierende Random-Agent benötigt in seinen seltenen erfolgreichen Episoden im Schnitt 2251,9 Schritte, bei einer gemittelten Pfadefizienz von lediglich 0,021. Der ungelernte Kompass-Agent mit einem Zufallsanteil von $\varepsilon = 0,3$ erreicht eine Effizienz von 0,177 und ist damit rund 3,3-mal so wegeffizient wie das trainierte LSTM-Netz, auch wenn er dabei eine rund 13 Prozentpunkte niedrigere Erfolgsquote in Kauf nimmt. Das überraschendere Ergebnis liefert der $\varepsilon = 0,8$ -Kompass: Er übertrifft das trainierte Modell nicht nur bei der Erfolgsquote, sondern mit einer Pfadefizienz von 0,094 gleichzeitig auch bei der Weglänge.

Ein leichter Selektionseffekt ist hierbei zu berücksichtigen: In die Pfadefizienz fließen nur jene Episoden ein, bei denen der Ausgang gefunden wurde. Verfahren, die oft scheitern, wobei die gescheiterten Versuche meist sehr lange Irrwege sind, stehen dadurch statistisch etwas besser da, als sie eigentlich sind. Das erklärt den starken Kompass jedoch nicht vollständig. Die bittere Erkenntnis bleibt: Das Training führt zwar ans Ziel, aber alles andere als auf direktem Weg.

Ein plausibler Mitverursacher liegt in der Reward-Struktur selbst: Der Neuland-Bonus von $+0,02$ übersteigt die Schrittkosten von $-0,01$ (siehe Abschnitt 4.2.5), sodass das Betreten einer noch unbesuchten Kachel unabhängig von ihrer Richtung zum Ziel netto positiv entlohnt wird. Dieser Anreiz zum flächigen Erkunden könnte die stark unterdurchschnittliche Wegeffizienz des LSTM mit erklären, ist im Rahmen dieser Arbeit jedoch nicht durch eine eigene Ablationsstudie belegt.

5.2.3 Die Determinismus-Lücke

Das LSTM-PPO-Modell wurde in zwei Modi ausgewertet: deterministisch, wobei stets die wahrscheinlichste Aktion gewählt wird, und stochastisch, wobei die Aktion aus der Wahrscheinlichkeitsverteilung gezogen wird. Der Unterschied zwischen beiden Auswertungen ist enorm, wie Abbildung 5.5 für jeden einzelnen der sieben Läufe zeigt.

Determinismus-Lücke des LSTM-PPO: dasselbe Modell, zwei Auswertungsmodi

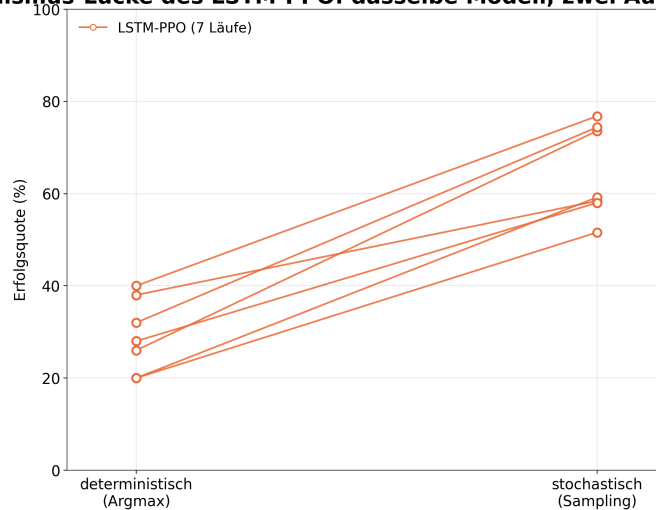


Abbildung 5.5: Evaluation – Erfolgsquote der sieben LSTM-PPO-Läufe in deterministischer und stochastischer Auswertung, eigene Darstellung.

Zwischen den beiden Modi klafft eine Lücke von durchschnittlich 35,5 Prozentpunkten (64,6 % gegenüber 29,1 %, siehe Abschnitt 5.2.1). Dieses Phänomen tritt konsequent in jedem der sieben Läufe auf: Kein einziger Lauf schneidet deterministisch besser ab als stochastisch.

Das ist ein unbequemer Befund. Die gemessene Erfolgsquote verdankt sich zu einem beträchtlichen Teil dem reinen Zufall beim Ziehen aus der Verteilung, nicht einer verlässlichen, gelernten Route. Bleibt der Zufall aus, bleibt der Agent oft in Sackgassen stecken.

Eine ergänzende Trajektorienanalyse stützt diese Vermutung, ohne sie vollständig zu bestätigen. Für zwanzig Seeds aus Testset A (ein einzelner Trainingslauf, Seed 1, Distanz 35–45, Schrittlimit 1500 statt der vollen 4000, aus Zeitgründen) richtet sich die deterministische Politik nur in 16,2 % der Schritte nach der BFS-optimalen Richtung, die stochastische in 23,4 %. Von den 13 gescheiterten deterministischen Episoden erschöpfen 9 das volle Schrittlimit, ohne durch einen erkennbaren Abbruch früher zu enden; in den übrigen 4 Episoden griff der in Abschnitt 4.2.7 beschriebene Early-Stop nach 256 Schritten ohne positiven Reward. Bei der Mehrheit der gescheiterten Episoden greift dieser Mechanismus

jedoch nicht: Da das PBRS-Shaping bei jeder Verringerung der BFS-Distanz einen positiven Einzelreward liefert, genügt bereits gelegentlicher, unsystematischer Fortschritt, um den Zähler immer wieder zurückzusetzen. Der Agent bewegt sich bis zum Limit weiter, ohne dem Ziel näherzukommen.

Einzelne Positionen werden dabei extrem häufig erneut besucht, in einem Fall 161-mal innerhalb von 1500 Schritten. Überraschend liegt die durchschnittliche Zahl mehrfach besuchter Positionen jedoch beim stochastischen Modus höher (90,3 gegenüber 58,5): Der Zufallsanteil erzeugt viele kurze Vor-und-Zurück-Bewegungen, während die deterministische Politik seltener, dafür aber in einzelnen Positionen deutlich hartnäckiger feststeckt. Die einfache Erklärung „Zufall befreit aus der Sackgasse“ trifft den Befund damit nur teilweise; robuster ist die geringere Optimalausrichtung der deterministischen Politik.

5.2.4 Einordnung gegenüber den Referenzpunkten

Um das Geleistete ins Verhältnis zu setzen, trägt Abbildung 5.6 die beiden Hauptmetriken, Erfolgsquote und Pfadefizienz, gegeneinander auf.

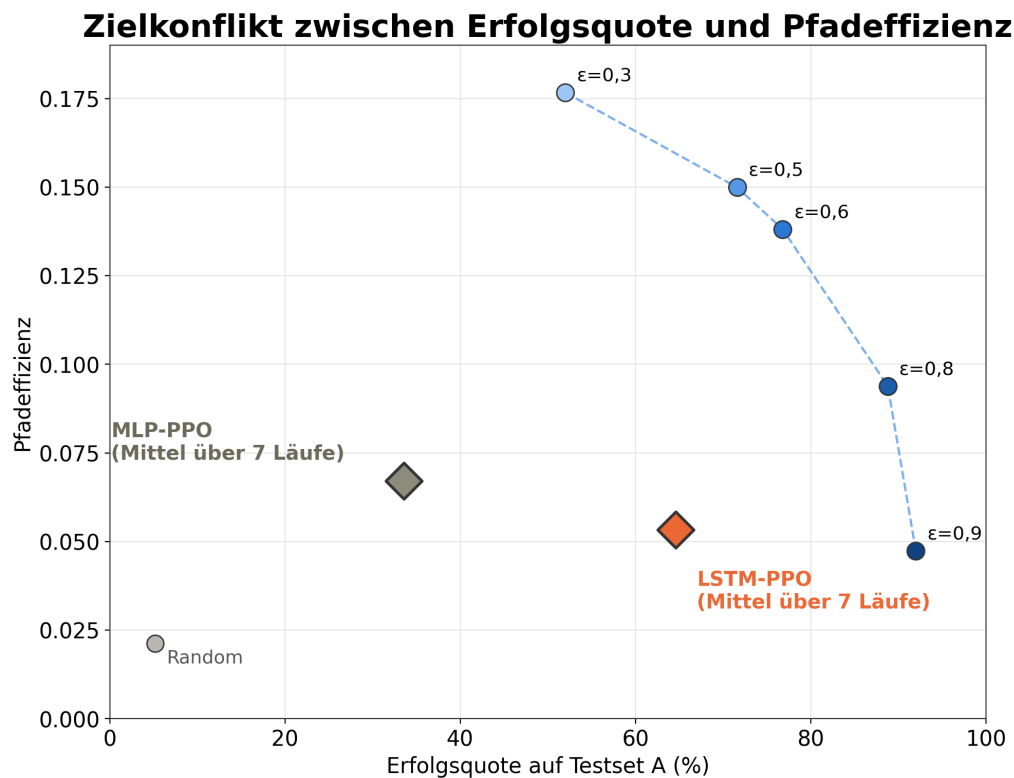


Abbildung 5.6: Evaluation – Erfolgsquote gegen Pfadefizienz für alle Verfahren, eigene Darstellung.

Die fünf Varianten der Kompass-Heuristik aus Abschnitt 3.2.4 ziehen im Diagramm eine zusammenhängende Kurve: Im getesteten Bereich steigt mit höherem Zufallsanteil ϵ die Erfolgsquote, während die Pfadefizienz sinkt. Bei $\epsilon = 0,9$ steigt die Erfolgsquote noch einmal auf 92,0 %, die Pfadefizienz sinkt dabei jedoch auf 0,047 und bleibt damit unter dem LSTM-Wert von 0,053. Auffällig ist die Position des trainierten Modells: Es liegt vollständig unterhalb dieser Heuristik-Kurve.

Besonders hart fällt der Vergleich mit der Kompass-Kurve insgesamt aus. Von den fünf getesteten Varianten schlagen $\varepsilon = 0,5$, $0,6$ und $0,8$ das LSTM-Netzwerk in *beiden* Metriken gleichzeitig: bei $\varepsilon = 0,8$ mit der größten Differenz, einer Erfolgsquote von $88,8\%$ gegenüber $64,6\%$ und einer Effizienz von $0,094$ gegenüber $0,053$. Nur bei $\varepsilon = 0,9$ reicht es, wie in Abbildung 5.6 zu sehen, wegen der knapp unterschrittenen Effizienz nicht zur Dominanz in beiden Metriken (siehe Abschnitt 5.2.2). Das wiegt umso schwerer angesichts dessen, was der Kompass eigentlich ist: eine primitive Heuristik, die nur Luftlinien-Richtungen kennt, Wände ignoriert und stumpf gegen Hindernisse rennt, bis der Zufall sie befreit.

Dass ein kartenblindes Verfahren ohne Trainingsphase zuverlässiger und direkter ins Ziel findet als ein Agent, der über bis zu 2,2 Millionen Zeitschritte hinweg optimiert wurde, ist die zentrale Pointe dieser Auswertung. Das ändert nichts daran, dass das LSTM besser lernt als das MLP. Für die Beurteilung, wie leistungsfähig Reinforcement Learning für dieses spezifische Problem tatsächlich ist, bleibt es jedoch ein starker Dämpfer.

5.2.5 Wirksamkeit der Hyperparameter

Abseits der großen Metriken liefert der Blick in die Trainingshistorie Erkenntnisse darüber, welche Parameter das Training tatsächlich beeinflussten und welche nicht. Tabelle 5.2 fasst die belegten Stellschrauben zusammen, die vollständige Hyperparameter-Konfiguration findet sich in Tabelle 4.3.

Tabelle 5.2: Belegte Wirkung einzelner Parameterentscheidungen

Parameter	Gewählt	Belegte Wirkung
ent_coef	0,05	Ein Versuch mit 0,01 scheiterte komplett; der Agent fand den Ausgang nicht mehr. Dies war der Parameter mit dem größten messbaren Effekt.
n_steps	256	Bei 512 verschwanden die Gradienten im Backpropagation-Through-Time-Verfahren und das Lernsignal brach zusammen.
batch_size	8 (LSTM)	Ein Wert von 64 destabilisierte den Critic spürbar (84% Erfolgsquote bei 8 gegenüber 48% bei 64, jeweils bei 150.000 Schritten).
Curriculum-Gate	doppelt	Der Wechsel von einem reinen Zeit-Limit auf ein Gate aus Leistung <i>oder</i> Budget behob drastische Leistungseinbrüche.
PBRs-Puffer	80	Ein zu kleiner Puffer (20) führte außerhalb der Zone zu falschem Shaping und zerstörte die gelernte Strategie.
vf_coef	0,5	Eine Erhöhung auf 1,0 wurde getestet, brachte aber nachweislich keine Verbesserung.

Aus diesen Daten lassen sich zwei Erkenntnisse ableiten. Erstens wird das Trainingsergebnis von wenigen Parametern dominiert: Der Entropie-Koeffizient, also der Zwang zum Ausprobieren neuer Aktionen, ist essenziell, während viele andere Werte kaum messbare Veränderungen brachten. Zweitens halfen erfolgreiche Anpassungen fast ausschließlich da-

bei, die Exploration zu fördern oder instabile Gradienten zu stabilisieren, nicht aber dabei, das Netz an sich leistungsfähiger zu machen.

Eine Leerstelle bleibt die Netzgröße (siehe Abschnitt 4.2.3). Die rekurrente Schicht mit 256 Einheiten wurde nie in einer sauberen Ablationsstudie gegen andere Größen getestet. Ein Versuch mit 512 Einheiten sah zwischenzeitlich vielversprechend aus, lief jedoch nie durch das Testset. Welche Größe hier tatsächlich optimal wäre, lässt sich aus den vorliegenden Daten nicht seriös beantworten.

Damit eng verwandt ist eine Einschränkung der Vergleichbarkeit zwischen den beiden Architekturen selbst. Wie in Abschnitt 4.2.3 dargestellt, besitzt das LSTM-Modell mit 1.038.917 Parametern gut viermal so viele trainierbare Gewichte wie das MLP-Modell mit 250.629 Parametern, da beide Netze mit dem in der Bibliothek vorgesehenen Standardaufbau trainiert wurden und nicht mit kapazitätsangeglichenen Architekturen. Der in Abschnitt 5.2.1 gemessene Vorsprung des LSTM ließe sich daher nicht allein auf die Rekurrenz zurückführen, sondern könnte teilweise auch aus der schlicht größeren Netzkapazität stammen. Ein Modellvergleich mit angeglicher Parameterzahl müsste in einer eigenen Ablationsstudie erfolgen, was im Rahmen dieser Arbeit aus Zeitgründen ausgeklammert werden soll. Der zentrale Befund, dass die Kompass-Heuristik beide trainierten Verfahren bei Erfolgsquote und Pfadeffizienz übertrifft (siehe Abschnitt 5.2.4), bleibt von dieser Einschränkung unberührt, da er unabhängig vom internen Architekturvergleich zwischen LSTM und MLP gilt.

Zum Schluss zeigt das Curriculum (siehe Abschnitt 4.2.4) ungeschminkt die Grenzen des Modells: In den Trainingsphasen 3 und 4 sollte der Agent bestimmte Erfolgsquoten von 70 % und 60 % erreichen, um die Phase regulär abzuschließen. In keinem der sieben Läufe ist das gelungen. Die Phasen endeten ausschließlich, weil das feste Schrittbudget aufgebraucht war. Die gewünschte Zielleistung lag außerhalb dessen, was die Architektur auf dieser Distanz leisten konnte.

6 Fazit und Ausblick

6.1 Diskussion der Ergebnisse

Am Ende der Arbeit steht die Rückkehr zur eingangs formulierten Forschungsfrage aus Abschnitt 1.3: wie eine prozedural generierte 2D-Spielwelt aufgebaut sein muss und welches RL-Verfahren darin die besten Navigationsergebnisse liefert. Im Folgenden werden die beiden Teilfragen zunächst getrennt betrachtet. Anschließend werden sie in Abschnitt 6.2 zu einem Gesamtbild zusammengeführt.

Teilfrage 1: Aufbau, Performanz und Lösbarkeit der Spielwelt

Die erste Teilfrage betraf den Aufbau einer performanten, seedbasierten und lösbaren Spielwelt. Die Antwort lässt sich an drei Punkten festmachen:

1. **Strukturelle Lösbarkeit statt Parameter-Testen:** Die Lösbarkeit der erzeugten Instanzen hängt nicht an einem feingetunten, fehleranfälligen Parameterfenster, sondern an einer harten strukturellen Garantie. Bei jeder Weltinitialisierung wird der Ausgang ausschließlich an Positionen platziert, die eine Breitensuche (BFS) vom Startpunkt aus als erreichbar bestätigt. Zusätzlich steht der achsparallele Manhattan-Carver als abschaltbarer Fallback bereit (siehe Abschnitt 4.1.4). Unlösbare Weltinstanzen sind dadurch ausgeschlossen, egal wie dicht die restliche Wandstruktur generiert wird.
2. **Abwechslung durch biomabhängige Parameter:** Die Konfigurationsparameter (wie die biomspezifischen Waddichten zwischen 7 % und 25 %) steuern demnach nicht das Ob der Lösbarkeit, sondern die Topologie: Sie bestimmen, wie umwegig und variantenreich die Pfade abseits der Hauptroute ausfallen, und erzeugen so die gewünschte visuelle und mechanische Varianz zwischen den Biomen.
3. **Performanz und Determinismus als Trainingsgrundlage:** Mit einer durchschnittlichen Generierungsdauer von $13,12 \mu\text{s}$ pro Chunk, einem Durchsatz von rund 73.000 Simulationsschritten pro Sekunde (siehe Tabelle 5.1) und einer gemessenen Determinismus-Abweichung von exakt 0,0000 % (siehe Abschnitt 5.1.5) erfüllt die Weltarchitektur die Anforderungen, die ein paralleles RL-Training an seine Umgebung stellt: reproduzierbare Welten über Seeds und genug Durchsatz, damit nicht die Simulation zum Engpass wird.

Damit ist die erste Teilfrage beantwortet: Eine faire und lösbare Welt entsteht nicht durch vorsichtiges Austarieren der Generierungsparameter, sondern durch die strukturelle Garantie bei der Platzierung des Ausgangs. Die Parameter formen die Welt, die Breitensuche sichert sie ab.

Teilfrage 2: Wahl des Trainingsverfahrens

Die zweite Teilfrage suchte die Kombination aus Policy-Architektur und Trainingsverfahren, die die höchste Erfolgsquote und die besten Wege zum Ausgang erreicht. Bei der Erfolgsquote fällt die Antwort klar aus: Es ist das rekurrente LSTM-PPO-Netz, trainiert über das leistungsorientierte Vier-Phasen-Curriculum mit kleinen Batches und hohem Entropie-Koeffizienten (siehe Abschnitt 3.2.3). Diese Kombination kommt stochastisch ausgewertet im Mittel auf 64,6 % ($\pm 10,1$), das MLP-PPO-Modell unter identischem Curriculum nur auf 33,5 % ($\pm 25,5$) (siehe Abbildung 5.2); der Unterschied ist signifikant ($p = 0,018$, siehe Abschnitt 5.2.1). Mit der partiellen Beobachtbarkeit kommt das LSTM-Netz im Mittel deutlich besser zurecht; zwar erreichen die besten MLP-Läufe das Niveau schwächerer LSTM-Läufe, am Gesamtbild ändert diese Überschneidung jedoch nichts.

Einen Dämpfer setzt die Determinismus-Lücke: Ohne Zufallsanteil fällt das LSTM-PPO-Modell von 64,6 % auf 29,1 % zurück, und zwar in jedem einzelnen der sieben Läufe (siehe Abbildung 5.5). Daran hat auch die vierte Curriculum-Phase nichts geändert, deren Greedy Fine-Tuning genau diese Lücke schließen sollte (siehe Abschnitt 3.2.3). Wie in Abschnitt 5.2.5 eingeordnet, ist der Vorsprung gegenüber dem MLP zudem nicht sauber von der reinen Netzkapazität zu trennen, da das LSTM gut viermal so viele Parameter besitzt wie das MLP.

Die zweite Hälfte der Frage, die nach den besten Wegen zum Ausgang, muss dagegen negativ beantwortet werden: Keine der trainierten Kombinationen findet gute Wege (siehe Abbildung 5.3). Für eine erfolgreiche Episode braucht das beste trainierte Modell im Schnitt 1425 Schritte, das rechnerische Optimum liegt bei 40,1 Schritten. Die gemittelte Pfadineffizienz von 0,053 wird selbst von trivialen Referenzheuristiken um mehr als das Dreifache übertroffen. Die gesuchte Kombination gewinnt also die Erfolgsquote, die besten Wege findet sie nicht.

Auf der Seite des Trainingsverfahrens dominieren wenige Einstellungen das gesamte Training (siehe Abschnitt 5.2.5). Der Entropie-Koeffizient entscheidet darüber, ob der Agent das Ziel überhaupt jemals sieht: Ein zu niedriger Wert lässt ihn das Navigieren komplett verlernen. Ausschlaggebend waren durchgehend Exploration und Gradientenstabilität, nicht die Kapazität oder Tiefe der neuronalen Netze.

6.2 Fazit

6.2.1 Beitrag der Weltgenerierung

Im Rahmen dieser Arbeit wurde ein vollständiges, performantes System zur prozeduralen Generierung unendlicher 2D-Spielwelten auf Basis von C++ und Raylib entworfen und implementiert.

- **Synthese aus Rauschen und Pfadgarantie:** Durch die Kombination mehrstufiger *Value-Noise*-Gitter mit der BFS-basierten Ausgangsplatzierung konnte der Zielkonflikt zwischen organisch wirkenden Landschaften und deterministisch garantierter

Begehbarkeit gelöst werden. Unlösbare Welt-Instanzen werden systematisch ausgeschlossen.

- **Hohe Speicher- und Laufzeiteffizienz:** Die Architektur nutzt eine bedarfsgesteuerte Chunk-Verwaltung (*Lazy Loading*) mit einer 8×8 -Kachelstruktur in Kombination mit einer spärlich besetzten Hash-Map-Struktur. Die Benchmarks bestätigen eine Chunk-Generierungszeit von durchschnittlich $13,12 \mu\text{s}$ und einen Speicherbedarf von ca. 162 KiB bei 2048 geladenen Chunks.
- **Visuelle und funktionale Kohärenz:** Das Entkoppeln der logischen Datenhaltung von der visuellen Darstellung ermöglichte dynamische Effekte wie stufenloses *Biome Blending* sowie das nahtlose Ableiten von Strand-Overlays an Seen-Rändern im Render-Pfad, ohne den Speicheraufwand pro Chunk zu erhöhen.
- **Vollständiger Determinismus:** Mit einer Abweichungsquote von 0,0000 % bei über 500.000 verglichenen Kacheln konnte die exakte Reproduzierbarkeit der Weltgenerierung über individuelle Welt-Seeds nachgewiesen werden.

Die gestellten Anforderungen an Verlässlichkeit und Performanz wurden damit erfüllt; bei der Ästhetik bleiben die in Abschnitt 5.1.6 benannten Artefakte, das Biome-Swallowing und die rechtwinkligen L-Shapes des Manhattan-Carvers, bestehen.

6.2.2 Beitrag des RL-Trainings

Im Rahmen dieser Arbeit wurde eine vollständige Trainings- und Evaluationspipeline für das Navigationsproblem auf der in Abschnitt 6.2.1 beschriebenen Umgebung aufgebaut.

- **Funktionierendes Curriculum mit dualem Gate:** Sieben unabhängige LSTM-PPO- und sieben unabhängige MLP-PPO-Läufe wurden über bis zu 2,2 Millionen Zeitschritte hinweg leistungs- und budgetbasiert trainiert und unter einem einheitlichen, reproduzierbaren Protokoll ausgewertet (siehe Abschnitt 3.2.5).
- **Rekurrenz-Vorteil statistisch abgesichert:** LSTM-PPO übertrifft MLP-PPO in der Erfolgsquote statistisch signifikant (Welch-Test, Mann-Whitney-U-Test, Cohens $d = 1,6$), auch wenn sich dieser Vorsprung nicht sauber von der höheren Parameterzahl des LSTM trennen lässt (siehe Abschnitt 5.2.5).
- **Determinismus-Lücke identifiziert:** Das LSTM-PPO-Modell verliert im deterministischen Modus einen Großteil seiner Erfolgsquote und fällt von 64,6 % auf 29,1 %. Der gelernte Vorteil beruht damit teilweise auf stochastischem Sampling und nicht allein auf einer verlässlichen Route (siehe Abschnitt 5.2.3).
- **Baseline-Vergleich als methodischer Befund:** Die einfache ε -Kompass-Heuristik übertrifft beide trainierten Modelle bei $\varepsilon = 0,8$ in Erfolgsquote und Pfadeffizienz gleichzeitig, was die Grenzen des gewählten RL-Ansatzes auf dieser konkreten Aufgabenstellung offenlegt (siehe Abschnitt 5.2.4).

Abbildung 6.1 stellt das LSTM-PPO-Endergebnis den vorab festgelegten Zielkriterien von mindestens 70 % auf Testset A und mindestens 60 % auf Holdout B gegenüber, deren Zahlenwerte aus den in Abschnitt 4.2.4 beschriebenen Ziel-Erfolgsquoten der Phasen 3

und 4 übernommen wurden. Dort galten sie als deterministische Gate-Kriterien auf den Validierungs-Seeds; hier werden sie als nominale Schwellen auf die stochastische Hauptmetrik aus Abschnitt 3.2.5 angelegt, unabhängig von der ursprünglichen Gate-Modalität der Phasen.

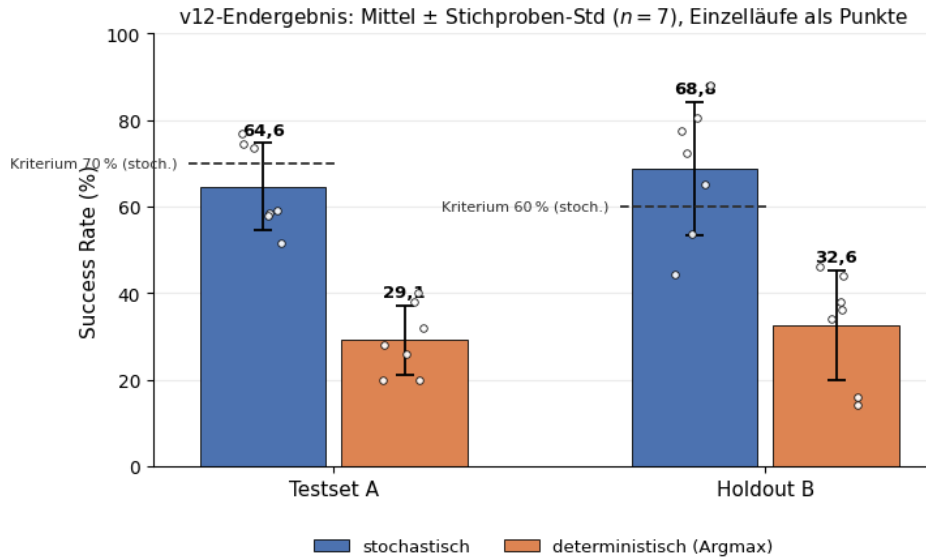


Abbildung 6.1: Erfolgsquote des LSTM-PPO-Modells ($n=7$) gegen das Zielkriterium, Testset A und Holdout B, eigene Darstellung.

Das Kriterium für Testset A ($\geq 70\%$) wird mit $64,6\%$ verfehlt, während das Kriterium für Holdout B ($\geq 60\%$) mit $68,8\%$ erreicht wird; die eingezeichneten Streupunkte der sieben Einzelläufe machen zugleich sichtbar, dass beide Mittelwerte auf einer erheblichen Schwankungsbreite beruhen. Die zusätzlich gezeigten deterministischen Werte fallen deutlich niedriger aus, unterliegen jedoch keinem eigenen Zielkriterium.

Die Zielsetzung, verschiedene RL-Verfahren auf der Navigationsaufgabe zu trainieren und ihre Leistungsfähigkeit gegen ungelernete Referenzen einzuordnen, wurde damit erreicht. Ob Reinforcement Learning der geeignete Ansatz für diese konkrete Aufgabenstellung ist, bleibt dagegen offen und wird in Abschnitt 6.2.3 eingeordnet.

6.2.3 Gesamtbild

Beide Projektteile, die Entwicklung der prozeduralen Welt und das Training der Agenten, sind für sich genommen funktional und abgeschlossen. Der eigentliche Ertrag der Arbeit liegt jedoch nicht in diesen Einzelergebnissen, sondern in ihrem Zusammenspiel.

Der auffälligste Befund der Evaluation ist, dass eine ungelernete, primitive Heuristik das aufwendig trainierte RL-Modell in beiden Metriken deklassiert: Der $\varepsilon = 0,8$ -Kompass erreicht $88,8\%$ Erfolgsquote gegenüber $64,6\%$ und eine Pfadeffizienz von $0,094$ gegenüber $0,053$ (siehe Abbildung 5.6). Diese Heuristik wertet lediglich die Richtungskomponenten zum Ziel aus und geht einen Schritt auf der längeren Achse (siehe Abschnitt 3.2.4). Sie ist „wandblind“, nutzt keine Pfadfindung und kennt das Layout der Welt nicht.

Genau hier greifen die beiden Teile der Arbeit ineinander. Der Ausgang wird stets über eine Breitensuche vom Spawnpunkt aus platziert, wodurch die generierte Welt strukturell auf kurze, direkte Wegverbindungen hin ausgelegt ist (siehe Abschnitt 4.1.4). Die simple Kompass-Heuristik bewegt sich rein richtungsbasiert auf das Ziel zu. Naheliegend, aber noch nicht experimentell belegt, ist die Vermutung, dass die auf Erreichbarkeit optimierte Weltgenerierung genau diejenige einfache Wegstruktur begünstigt, die das ungelernete Verfahren instinktiv ausnutzt (siehe Abschnitt 6.3.2).

Daraus ergibt sich eine methodische Frage für weiterführende Arbeiten: Sollte sich diese Vermutung bestätigen, enthielte eine Trainingsumgebung, deren Lösbarkeit über geometrisch regelmäßige Konstruktionsprinzipien erzwungen wird, systembedingte Abkürzungen. Ein Vergleich zwischen gelernten und ungelerten Verfahren würde dann nicht mehr nur die „Intelligenz“ des Reinforcement Learnings messen, sondern auch, wie gut sich solche strukturellen Eigenschaften der Welt ausnutzen lassen.

Für die anfängliche Fragestellung bedeutet das: LSTM-PPO bleibt der Gewinner unter den Deep-Learning-Verfahren, die Umgebung erfüllt ihre Performanzanforderungen. Bewiesen ist damit jedoch nicht, dass RL diese spezifische Navigationsaufgabe gut löst. Belegt ist stattdessen, dass RL unter diesen Rahmenbedingungen schlechter abschneidet als ein triviales Skript. Ob dahinter ein struktureller Vorteil der Kompass-Heuristik durch die Bauart der Weltgenerierung steht, bleibt eine offene, in Abschnitt 6.3.2 benannte Frage.

6.3 Ausblick

Aus den Erkenntnissen und methodischen Einschränkungen beider Projektteile ergeben sich Ansatzpunkte für weiterführende Arbeiten, im Folgenden getrennt nach Weltgenerierung und Reinforcement Learning betrachtet.

6.3.1 Weltgenerierung

Aus den in Abschnitt 5.1.6 identifizierten Limitationen sowie den erweiterten Möglichkeiten prozeduraler Spielwelten leiten sich folgende Ansatzpunkte für zukünftige Entwicklungen ab:

1. **Dynamisches Chunk-Unloading (LRU-Cache):** Um bei extrem langen Spielsitzungen ein unbegrenztes Wachstum der Chunk-Map zu verhindern, sollte der Speicher um eine *Least-Recently-Used* (LRU) Entladestrategie erweitert werden. Nicht mehr sichtbare Chunks außerhalb des Spieler-Umkreises können so sicher aus dem RAM entfernt oder auf Datenträger ausgelagert werden.
2. **Verbesserung der Pfad-Ästhetik (*A*-Guided Carving*):** Der aktuelle Manhattan-Carver neigt stellenweise zu rechtwinkligen „L-Shapes“. Eine Integration eines A*-Algorithmus mit geglätteter Kostenfunktion (*Bézier-Kurven* oder *Diagonalschneiden*) könnte die Wegführung noch organischer in das Noise-Terrain einbetten.
3. **Nicht-lineares Biome-Blending:** Zur Vermeidung des *Biome-Swallowing*-Effekts bei 1×1 -Kleinbiomen soll die Farbmischungsmatrix dahingehend angepasst werden,

dass der zentrale Chunk ein höheres Eigengewicht erhält oder Biome vorab über zelluläre Automaten geclustert werden.

4. **Vertikalität und Multi-Layer-Welten:** Die Erweiterung des 2D-Rasters um eine vereinfachte Höhendimension (Z -Ebene) würde das Hinzufügen von unterirdischen Höhlensystemen, mehrstöckigen Dungeons oder fließenden Gewässern (Flüsse) ermöglichen.

6.3.2 Reinforcement Learning

1. Isolierung der Pfadgarantie: Die Hypothese, dass die Kompass-Heuristik von der auf Erreichbarkeit optimierten, tendenziell direkten Ausgangsplatzierung profitiert, müsste experimentell bewiesen werden. Zukünftige Arbeiten sollten die Lösbarkeit über organische, irreguläre Pfadfindungsalgorithmen sicherstellen, da sich der wahre Leistungsunterschied zwischen Heuristik und RL-Agent erst isolieren lässt, wenn die Umgebung keine offensichtliche Geometrie mehr aufweist (vgl. Abschnitt 4.1.4).

2. Systematische Trajektorienanalyse über alle Modelle: Eine Stichprobe von zwanzig Seeds an einem einzelnen Modell (Abschnitt 5.2.3) zeigt bereits eine geringere BFS-Optimalausrichtung im deterministischen Modus (16,2 % gegenüber 23,4 %) und vereinzelt extreme Positions-Wiederholungen, wobei die anfängliche Vermutung reiner Zweischritt-Schleifen zu einfach war. Offen bleibt eine systematische Untersuchung über alle sieben Modelle und das volle Schrittbudget von 4000 statt der hier aus Zeitgründen verwendeten 1500 Schritte, sowie eine genauere Klassifikation der beobachteten Fest-Steck-Muster.

3. Echte Schwierigkeitskalibrierung: Der direkte Einfluss der Welt-Parameter, etwa Waddichte, Chunk-Größe und Biom-Verteilung, auf die RL-Metriken bleibt in dieser Arbeit ausgeklammert, weil der Fokus auf dem Vergleich der Trainingsverfahren lag und nicht auf einer Parameterstudie der Weltgenerierung. Eine systematische Variation dieser Werte gegenüber der erreichten Erfolgsquote könnte die erste Teilfrage von einer reinen Aussage über Lösbarkeit zu einer Metrik für Schwierigkeit weiterentwickeln.

4. Kapazitätsangeglichene Ablationsstudie: Wie in Abschnitt 5.2.5 benannt, unterscheiden sich LSTM- und MLP-Modell nicht nur in der Rekurrenz, sondern auch in der Parameterzahl. Zukünftige Arbeiten müssten beide Architekturen auf eine vergleichbare Parameterzahl bringen, um den Anteil der Rekurrenz am gemessenen Vorsprung sauber vom Anteil der reinen Netzkapazität zu trennen.

Zum Schluss lässt sich das Ergebnis der Arbeit in drei Sätzen festhalten. Die entwickelte Weltgenerierung liefert eine performante, deterministische und garantiert lösbare Trainingsumgebung. Das Reinforcement Learning navigiert darin mit Gedächtnis zuverlässiger als ohne, bleibt aber hinter einer trivialen Kompass-Heuristik zurück. Ob dieser Rückstand an den Verfahren selbst oder an der Bauart der Welt liegt, könnte erst eine Umgebung zeigen, deren Lösbarkeit nicht über direkte Wege erzwungen wird; sie zu bauen wäre der naheliegende nächste Schritt.

Literaturverzeichnis

- [1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009.
- [2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.
- [3] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & BabyAI: Core architectures for RL in environments with sparse rewards and partial observability. *arXiv preprint arXiv:2306.13831*, 2023. URL <https://arxiv.org/abs/2306.13831>.
- [4] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 2048–2056. PMLR, 2020. URL <https://proceedings.mlr.press/v119/cobbe20a.html>.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 3 edition, 2009.
- [6] Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021. URL <https://arxiv.org/abs/2109.06780>.
- [7] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *2015 AAAI Fall Symposium Series*, 2015.
- [8] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [9] Lawrence Johnson, Georgios N. Yannakakis, and Julian Togelius. Cellular automata for real-time generation of infinite cave levels. In *Proceedings of the 2010 Workshop on Procedural Content Generation in Games (PCGames '10)*. ACM, 2010. URL <https://doi.org/10.1145/1814256.1814266>.
- [10] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- [11] Ahmed Khalifa, Philip Bontrager, Sam Earle, and Julian Togelius. PCGRL: Procedural content generation via reinforcement learning. In *Proceedings of the AAAI*

- Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 16, pages 95–101, 2020. URL <https://arxiv.org/abs/2001.09212>.
- [12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
 - [13] Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prok. POPGym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2303.01859>.
 - [14] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *The Journal of Machine Learning Research (JMLR)*, 21(1):7382–7431, 2020.
 - [15] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, volume 99, pages 278–287, 1999.
 - [16] Ken Perlin. An image synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3):287–296, 1985. URL <https://doi.org/10.1145/325165.325247>.
 - [17] Inigo Quilez. Warping - Domain distortion. URL <https://iquilezles.org/articles/warp/>. Online-Artikel, ohne festes Publikationsdatum, Zugriff am 13.08.2026.
 - [18] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Scott Ernest, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
 - [19] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
 - [20] Guy L. Steele, Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA '14)*, pages 453–472. ACM, 2014. URL <https://doi.org/10.1145/2660193.2660195>.
 - [21] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT press, Cambridge, Massachusetts, 2 edition, 2018.
 - [22] Mark Towers, J. K. Terry, Ariel Kwiatkowski, John U. Balis, Gianluca Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Andrew Tan Jin Shen,

and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2023. URL <https://github.com/Farama-Foundation/Gymnasium>. Version 0.29.1.

A Konfigurationsauszug

Listing A.1 zeigt den für die Weltgenerierung relevanten Ausschnitt der ausgelieferten Konfigurationsdatei, auf den Tabelle 3.1 Bezug nimmt.

```
1 "worldgen": {
2     "exitMinDistance": 35,
3     "exitMaxDistance": 45,
4     "forceGuaranteedPath": false,
5     "guaranteedPathFallback": false,
6     "spawnClearRadius": 2,
7     "exitClearRadius": 1,
8     "procedural": {
9         "enableCellularSmoothing": false,
10        "cellularIterations": 2,
11        "cellularBirthMinNeighbors": 5,
12        "cellularSurvivalMinNeighbors": 4,
13        "enableFloodFillValidation": false,
14        "validationRadiusChunks": 8,
15        "enableMacroGraphPrecheck": false,
16        "macroGraphRadiusChunks": 16
17    }
18 }
```

Quelltext A.1: Weltgenerierungs-Konfiguration (Auszug)

B Autorschaftsübersicht

Tabelle B.1 ordnet jedes Kapitel beziehungsweise jeden Abschnitt der Arbeit einer Person zu oder weist ihn als gemeinsam verfasst aus, ergänzend zur Erklärung im Vorspann.

Tabelle B.1: Zuordnung der Kapitel und Abschnitte der Arbeit

Kapitel	Titel	Person
1	Einleitung	gemeinsam
2	Grundlagen	gemeinsam
3.1	Methodik der Weltgenerierung	Laurin
3.2	Methodik des RL-Trainings	Florian
4.1	Implementierung der Weltgenerierung	Laurin
4.2	Implementierung des RL-Trainings	Florian
5.1	Evaluation der Weltgenerierung	Laurin
5.2	Evaluation des RL-Trainings	Florian
6.1	Diskussion der Ergebnisse	gemeinsam
6.2.1	Beitrag der Weltgenerierung	Laurin
6.2.2	Beitrag des RL-Trainings	Florian
6.2.3	Gesamtbild	gemeinsam
6.3.1	Ausblick Weltgenerierung	Laurin
6.3.2	Ausblick Reinforcement Learning	Florian
A	Konfigurationsauszug	Laurin
B	Autorschaftsübersicht	gemeinsam
C	Rohdaten der statistischen Auswertung	Florian

C Rohdaten der statistischen Auswertung

Dieser Anhang stellt die Einzelwerte bereit, auf denen die in Kapitel 5.2 berichteten Mittelwerte, Standardabweichungen und Signifikanztests beruhen, sowie die vollständige Ergebnistabelle der ungelernten Referenzverfahren aus Abschnitt 5.2.2.

Tabelle C.1: LSTM-PPO: Erfolgsquote je Lauf, Testset A und Holdout B, stochastisch und deterministisch

Seed	A stoch. (%)	A det. (%)	B stoch. (%)	B det. (%)
1	58,4	38,0	65,2	46,0
2	73,6	26,0	77,6	44,0
3	74,4	32,0	88,0	36,0
4	76,8	40,0	80,4	38,0
5	59,2	20,0	72,4	34,0
6	51,6	20,0	53,6	14,0
7	58,0	28,0	44,4	16,0
Mittel $\pm$ Std	64,6 $\pm$ 10,1	29,1 $\pm$ 8,0	68,8 $\pm$ 15,5	32,6 $\pm$ 12,7

Tabelle C.2: MLP-PPO: Erfolgsquote je Lauf, Testset A und Holdout B, stochastisch

Seed	A stoch. (%)	B stoch. (%)
1	18,8	17,6
2	53,6	62,8
3	22,0	20,8
4	4,0	2,0
5	60,8	70,8
6	10,4	7,6
7	65,2	69,2
Mittel $\pm$ Std	33,5 $\pm$ 25,5	35,8 $\pm$ 30,5

Tabelle C.3: Ungelernte Referenzverfahren: Erfolgsquote, mittlere Schrittzahl und Pfadeffizienz, Testset A und Holdout B

Verfahren	Testset A			Holdout B		
	SR (%)	Schritte	Effizienz	SR (%)	Schritte	Effizienz
Random	$5,2 \pm 2,7$	2251,9	0,021	$10,4 \pm 1,7$	2031,2	0,025
Kompass $\varepsilon = 0,3$	$52,0 \pm 4,2$	481,0	0,177	$50,4 \pm 2,6$	583,9	0,156
Kompass $\varepsilon = 0,5$	$71,6 \pm 3,8$	474,3	0,150	$70,8 \pm 4,8$	520,5	0,148
Kompass $\varepsilon = 0,6$	$76,8 \pm 7,0$	482,8	0,138	$80,4 \pm 4,6$	518,3	0,144
Kompass $\varepsilon = 0,8$	$88,8 \pm 5,2$	672,5	0,094	$87,6 \pm 1,7$	606,3	0,092
Kompass $\varepsilon = 0,9$	$92,0 \pm 5,1$	1230,3	0,047	$91,6 \pm 4,3$	1151,3	0,047

Alle stochastischen Werte sowie die Werte der Referenzverfahren sind Mittelwerte über fünf unabhängige Wiederholungen mit je eigenem Politik-RNG-Seed (siehe Abschnitt 3.2.5); die deterministischen Spalten entstammen dagegen einer einmaligen Zusatzmessung je Lauf auf denselben Testset-Seeds, erkennbar am 2-Prozent-Raster der Einzelwerte. Die Standardabweichungen bei Random und den Kompass-Varianten beziehen sich auf die Streuung zwischen den fünf Wiederholungen, nicht auf die Streuung zwischen einzelnen Modell-Seeds wie bei LSTM-PPO und MLP-PPO.